Gene Regulatory Network-Guided Causal Generative Modeling of Single-Cell Transcriptomics

A thesis submitted to McGill University in partial fulfillment of the requirements for the degree of Master of Science.

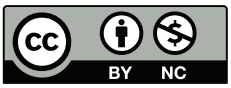
Yazdan Zinati

January 2026

Faculty of Engineering
Department of Electrical & Computer Engineering
McGill University, Montréal

Supervisor: Amin Emad

© 2026 Yazdan Zinati



Gene Regulatory Network-Guided Causal Generative Modeling of Single-Cell Transcriptomics
© 2026 by Yazdan Zinati is licensed under CC BY-NC 4.0. To view a copy of this license, visit
https://creativecommons.org/licenses/by-nc/4.0/

Abstract

Precisely controlled programs of gene expression are essential for cellular function and organismal viability across all forms of life. Gene regulatory networks (GRNs) provide a high-level mathematical representation of transcriptional regulation by describing the interactions between genes and their regulatory elements, such as transcription factors, across diverse cellular states and contexts. Through this lens, GRNs offer biological insights into processes such as morphogenesis, disease progression, stress adaptation, and response to environmental stimuli. High-throughput profiling of gene expression, particularly single-cell RNA sequencing, has fueled the development of data-driven methods for reconstructing GRNs, sometimes in combination with other data modalities or omics layers. However, in the absence of comprehensive experimentally validated networks corresponding to the exact biological system and experimental conditions under study, evaluating these methods remains a major challenge.

In this thesis, we address key challenges in benchmarking GRN inference methods through two interconnected studies. *Chapter 3* introduces GRouNdGAN, a GRN-guided simulator that achieves state-of-the-art realism in both steady-state and transient-state scRNA-seq simulations, capturing regulatory properties while preserving gene identities without requiring user intervention. As a causal generative model, GRouNdGAN expresses genes under the control of their regulators and can sample from both observational and interventional distributions, allowing matched case/control in-silico perturbation experiments. *Chapter 4* formalizes the GRouNdGAN framework as a reproducible protocol, providing in-depth procedural guidance, and introduces GRouNdGAN-toolkit, an extension of the model that includes additional features and customizable modules aimed at facilitating its adaptation to various experimental setups, which are explored through a series of new empirical studies.

Together, these studies advance the development of GRN inference methods by providing synthetic gold-standard ground truths for any biological system of interest, along with procedures and tools for systematic evaluation, thereby closing the gap between in-silico and experimental data benchmarks of GRN inference methods.

Résumé

To my parents, this thesis is dedicated to you for being my support through it all.

Acknowledgments

My sincere thanks go to Prof. Amin Emad for welcoming me into computational biology and guiding me through every step of this research journey.

Endless conversations, thoughtful feedback, and well-timed jokes from the members of the COMBINE lab kept this thesis on course and my sanity intact; their presence is felt in each chapter. I have been fortunate to benefit from Abdulrahman Takiddeen's mentorship and Safyan Aman Memon's company during late lab sessions.

The work described in this thesis was made possible thanks to the support by the Natural Sciences and Engineering Research Council of Canada (NSERC), the Government of Canada's New Frontiers in Research Fund (NFRF), the Canada Foundation for Innovation (CFI), the McGill Initiative in Computational Medicine, Fonds de recherche du Québec, the Digital Research Alliance of Canada, and Calcul Québec.

Contents

Abstract ii

Résumé iii

Acknowledgments v

1 Introduction 5

Motivation 5

Thesis Organization & Contributions 5

Publications & Author Contributions 6

2 Background Material and Literature Review 7

The Biological Basis of Gene Expression and Regulation 8

Gene Expression and Transcriptional Control 8

Mechanisms of Gene Regulation and Transcription Factors 8

Gene Regulatory Networks as Biological Models 8

Computational & Statistical Models of Gene Regulatory Network Inference 8

Problem Formulation of GRN Inference 8

Benchmarking Gene Regulatory Network Inference Methods 8

Motivation and Challenges in Benchmarking 8

Experimental Validation-Based Benchmarks 8

Gene Regulatory-Guided Simulation 8

Generative Modeling and In-Silico Single-Cell Transcriptomics 10

Taxonomy of Generative Models 11

Generative Adversarial Networks (GANs) 15

On Convergence and Mode Collapse of GANs 15

Conditional and Controllable Generation with GANs 17

CausalGAN 18

In-Silico Simulation of scRNA-seq 20

3 GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks 23

Abstract 23

Introduction 23

Results 25

GRouNdGAN generates scRNA-seq data using causal generative adversarial networks 25

GRouNdGAN generates realistic scRNA-seq data 26

GRouNdGAN imposes a causal GRN in the simulated data 30

The imposed GRN can be reconstructed from the simulated data 32

GRouNdGAN-simulated data preserves trajectories and can be used for pseudo-time inference 34

GRouNdGAN achieves high performance on other datasets 37

Benchmarking GRN inference methods using GRouNdGAN confirms prior insights from curated and experimental datasets 38

In silico perturbation experiments using GRouNdGAN 40

Discussion 41

Methods 45

Datasets and preprocessing . . . 45
GRouNdGAN's model architecture . . . 46
Training procedure and hyperparameter tuning . . . 49
Causal GRN preparation . . . 50
Evaluation of the resemblance of real and simulated cells . . . 51
Baseline simulator models . . . 52
In silico perturbation experiments using GRoundGAN . . . 53
Trajectory inference and pseudo-time analysis . . . 53
GRN inference methods . . . 54
Statistics and reproducibility . . . 55
Reporting summary . . . 55
Data availability . . . 55
Code availability . . . 55
References . . . 56
Acknowledgements . . . 60
Author contributions . . . 60
Competing interests . . . 61
Additional information . . . 61

4 Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit 63

Abstract . . . 63
Introduction and Background . . . 64
Development of the protocol . . . 65
Applications of the method . . . 66
Comparison with other methods . . . 67

Experimental design . . . 70
Expertise needed to implement the protocol . . . 74
Discussion and Limitations . . . 74
Materials and Methods . . . 76
Equipment . . . 76
Equipment setup . . . 78
Procedure . . . 82
Step 1. Preprocessing . . . 84
Step 2. GRN Creation . . . 85
Step 3. Model Training . . . 88
Step 4. Simulation . . . 91
Step 5. Assessing simulated data quality (Optional) . . . 93
Step 6. GRN inference benchmarking (Optional) . . . 95
Step 7. Perturbation prediction (Optional) . . . 97
Timing . . . 99
General notes and troubleshooting . . . 99
Expected results . . . 100
Supplementary Information . . . 102
Authors' contributions . . . 102
Funding . . . 102
Data Availability . . . 102
Code Availability . . . 103
Competing interests . . . 103
References . . . 103

5 Discussion **107**

6 Conclusion **108**

List of Figures

Background Material and Literature Review **8**

2.1 Taxonomy of generative models 11

2.2 Example of a possible causal graph 18

2.3 The CausalGAN Architecture 19

GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks **23**

1 Architecture and training procedure of GRouNdGAN. 27

2 Performance of GRouNdGAN in generating realistic scRNA-seq data. . . . 29

3 Performance of GRNBoost2 in recovering the imposed edges versus unimposed edges (PBMC-CTL). 33

4 PAGA graphs and cell type marker gene activation patterns in experimental and simulated BoneMarrow dataset. 35

5 Consistency in trajectories and pseudo-temporal orderings between data generated by GRouNdGAN and the experimental BoneMarrow hematopoietic dataset. 36

6 TF knockout experiments and their effect on cell type generation. 39

Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit **63**

1 Schematic overview of the GRouNdGAN pipeline. 68

2 Cell type-specific simulation of cells using GRouNdGAN. 72

3 Robustness analysis of simulated data. 75

4 TensorBoard visualizations during GRouNdGAN training. 92

5 Example of expected plot outputs saved by the evaluation and benchmarking commands. 98

6 Example of expected plot outputs saved by the perturbation command. . . . 100

List of Tables

GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks **23**

1 Performance of different simulators in generating realistic scRNA-seq data using the PBMC-CTL dataset 28

2 Performance of different simulators in generating realistic scRNA-seq data using the PBMC-All dataset 31

3 Performance of GRouNdGAN in generating realistic scRNA-seq data using the Dahlin, Tumor-All, and Tumor-malignant datasets 37

Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit **63**

1 Performance of GRouNdGAN in generating realistic scRNA-seq data using cell-type specific GRNs using the PBMC dataset. 73

2 Arguments relevant to the preprocessing command in the configuration file. . 84

3 Arguments relevant to the GRN creation command in the configuration file. 86

4 Arguments relevant to the training command in the configuration file. 89

5 Arguments relevant to the generation command in the configuration file. . . 93

6 Arguments relevant to the evaluation command in the configuration file. . . . 94

7 Arguments relevant to the benchmarking command in the configuration file. 96

8 Arguments relevant to the perturbation command in the configuration file. . 97

9 Troubleshooting table. 101

List of Abbreviations

ATAC-seq	Assay for transposase-accessible chromatin using sequencing
AUPRC	Area under the precision-recall curve
AUROC	Area under the receiver operating characteristic curve
BEELINE	Benchmarking gene regulatory network inference from single-cell transcriptomic data algorithms
BoolODE	Boolean ordinary differential equation
CausalGAN	Causal generative adversarial network
CeSpGRN	Cell specific gene regulatory network inference
ChIP-seq	Chromatin immunoprecipitation followed by sequencing
CLI	Command line interface
cscGAN	Conditional single-cell generative adversarial network
CUDA	Compute unified device architecture
cwGAN	Conditional wasserstein generative adversarial network
DAG	Directed acyclic graph
DE	Differential expression
DREAM	Dialogue on Reverse Engineering Assessment and Methods
ESCAPE	Embryonic stem cell atlas from pluripotency evidence
GAN	Generative adversarial network
GENIE3	Gene network inference with ensemble of trees
GEO	Gene expression omnibus
GNU AGPL	GNU affero general public license
GNW	GeneNetWeaver

GPU	Graphics processing unit
GRN	Gene regulatory network
GRNBoost	Gene regulatory network inference with gradient boosting
GRouNdGAN	GRN-guided in silico simulation of single-cell RNA-seq data using causal generative adversarial networks
GTRD	Gene Transcription Regulation Database
HGNC	HUGO gene nomenclature committee
HPC	High-performance computing
HSPC	Hematopoietic stem and progenitor cell
I2D	Interologous interaction database
IID	Integrated interactions database
INI	Initialization
KEGG	Kyoto encyclopedia of genes and genomes
KO	Knockout
LEAP	Lag-based expression association for pseudotime-series
LLM4GRN	Large language model for gene regulatory network inference.
LSN	Library-size normalization.
MARS-seq	Massively parallel RNA single-cell sequencing framework
(m)iLISI	(Mean) integration local inverse Simpson's index
MMD	Maximum mean discrepancy
mRNA	Messenger ribonucleic acid
MTX	Matrix market format
NCBI	National center for biotechnology information
PAGA	Partition-based graph abstraction
PBMC	Peripheral blood mononuclear cell
PC	Principal component
PCA	Principal component analysis
PIDC	Partial information decomposition and context
PIP	Pip installs packages

PPCOR	Partial and semi-partial (part) correlation
PR	Precision-recall
PYPI	Python package index
ReLU	Rectified linear unit
RF	Random forest
RKHS	Reproducing kernel hilbert space
RNA-seq	Ribonucleic acid sequencing
ROC	Receiver operating characteristic
SCANPY	Single-cell analysis in python
scDesign	ScRNA-seq experimental design
scGAN	Single-cell generative adversarial network
scIGAN	Single-cell imputation generative adversarial network
SCODE	Describe regulatory networks and expression dynamics with linear ordinary differential equations
scRegulate	Single-cell regulatory-embedded variational inference of transcription factor activity from gene expression
scRNA-seq	Single-cell ribonucleic acid sequencing
SDE	Stochastic differential equation
SERGIO	Single-cell expression of genes in silico
SIF	Singularity image format
SINCERITIES	Single cell regularized inference using time-stamped expression profiles
SSH	Secure shell
TF	Transcription factor
t-SNE	T-distributed stochastic neighbor embedding
TSV	Tab-separated values
UMAP	Uniform manifold approximation and projection
UMI	Unique molecular identifier
URL	Uniform resource locator
VENV	Virtual environment

VRAM	Video random access memory
WGAN	Wasserstein generative adversarial network
WGAN-GP	Wasserstein generative adversarial network with gradient penalty

Chapter 1

Introduction

Motivation

Thesis Organization & Contributions

Publications & Author Contributions

This thesis is a manuscript-based thesis composed of two manuscripts that together form a cohesive body of work.

- **Chapter 3**

  Zinati, Yazdan, Abdulrahman Takiddeen, and Amin Emad. 2024. "GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks." *Nature Communications* 15(1): 4055. DOI:10.1038/s41467-024-48516-6.

  All authors contributed to the design of the study and the algorithm, the analyses, and the writing of the manuscript. A.E. supervised the study. Y.Z. implemented the model and performed the statistical analyses. Y.Z., A.T., and A.E. read and approved the final manuscript.

- **Chapter 4**

  The manuscript presented in this chapter is currently available through the bioRxiv online preprint repository.

  Zinati, Yazdan, and Amin Emad. 2025. "Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit." *bioRxiv*.
  DOI: 10.1101/2025.08.14.670294.

  All authors contributed to the design of the protocol, data analyses, and writing the manuscript. A.E. supervised the study. Y.Z. implemented the protocol, conducted computational analyses, developed the website, and managed releases across various formats.

Chapter 2

Background Material and Literature Review

The Biological Basis of Gene Regulation

Gene Expression and Transcriptional Control

Mechanisms of Gene Regulation and Transcription Factors

Gene Regulatory Networks as System-Level Biological Models

Computational & Statistical Models of Gene Regulatory Network Inference

Problem Formulation of Gene Regulatory Network Inference

Benchmarking Gene Regulatory Network Inference Methods

Motivation and Challenges in Benchmarking

Experimental Validation-Based Benchmarks

Gene Regulatory-Guided Simulation

GRN-guided simulators generate datasets given known user-provided ground truth regulatory interactions. They allow for quantitative and systematic evaluation of GRN inference under

a tightly controlled setup. GeneNetWeaver (GNW) is the earliest and possibly most influential example of such methods. It was originally developed for bulk RNA-seq and used for the community-driven benchmarks of the DREAM network inference challenges. GNW models both transcription and translation using a thermodynamic approach that allows for both additive and multiplicative regulatory interactions. To express the network in terms of Ordinary Differential Equations (ODEs), each node i in the network is assigned the rate of change of mRNA concentration and the rate of change of protein concentration as:

$$\frac{\mathrm{d}\left[x_i\right]}{\mathrm{d}t} = m\, f\left(R_i\right) - l_x\left[x_i\right]$$

$$\frac{\mathrm{d}\left[p_i\right]}{\mathrm{d}t} = r\left[x_i\right] - l_p\left[p_i\right]$$

where m is the rate of mRNA transcription, r is the translation rate, R_i is the set of regulators of i, and l_x and l_p are the mRNA and protein degradation rates, respectively. f models the relative activation (between 0 and 1) of node i given the TF or protein concentration of its regulators R_i. To account for stochasticity in gene expression, GNW converts these ODEs into Chemical Langevin Equations (CLE). It adds molecular noise proportional to the square root of the production and degradation rates terms to mimic thermal fluctuations:

$$\frac{dX_t}{dt} = V(X_t) - D(X_t) + c\left(\sqrt{V(X_t)}\,\eta_t + \sqrt{D(X_t)}\,\eta_d\right)$$

where $V(X_t)$ and $D(X_t)$ are the mRNA or protein production and degradation terms, respectively. c regulates the intensity of the molecular noise and η_v and η_d are independent Gaussian white-noises. GNW uses the Stratonovich scheme and the Milstein method to integrate the two equations in the form of the Stochastic ODE (SDE) above and create an internal cell state.

Although GNW has been used in some single-cell studies, at times with minor adaptation (such as inducing dropout events to simulate sparsity), it does not preserve trajectories or statistical properties of scRNA-seq. Inspired by GNW, BoolODE was developed to extend it to handle Boolean regulatory logic by requiring the user to provide truth tables. When used in the BEELINE study, the best-performing GRN inference methods on its synthetic benchmarks performed poorly on non-synthetic benchmarks. Similarly, SERGIO uses SDEs, but follows a two-step approach by first generating a dataset strictly following the GRN, then relying on the user to match it to a reference by iteratively adjusting three parameters to add technical noise. This process not only burdens the user but introduces subjectivity (affecting reproducibility) and can also disrupt regulatory interactions imposed in the first simulation step. Importantly, neither of these methods preserves gene identities by default. Furthermore, to model cooperative

regulation (multiple TFs regulating a gene), BoolODE relies on user-provided truth tables, and SERGIO assumes the combined effect of multiple TFs to be the sum of their individual effects.

More recently, scMultiSim, also following a mechanistic approach, uses a unified kinetic framework to generate multi-modal single-cell data (including gene expression, chromatin accessibility, RNA velocity, and spatial coordinates) while also modeling cell identity, GRNs, cell–cell interactions (CCI), and chromatin accessibility while incorporating technical noise. MOSim allows for multilayer GRNs to be imposed, but does so by "rearranging" a provided seed scRNA-seq and scATAC-seq to match defined regulatory patterns.

Generative Modeling and In-Silico Single-Cell Transcriptomics

Generative modeling is a branch of unsupervised learning that aims to learn an approximate probability distribution $p_{model}(x)$ from a finite set of observed data points $\mathcal{D} = \{x^{(1)}, x^{(2)}, \dots, x^{(n)}\}$ drawn from an unknown true distribution $p_{data}(x)$. The main challenge in generative modeling is the high dimensionality of the same space. In single-cell transcriptomics, where the dimensionality corresponds to the number of genes, approximately 20000, most combinations of gene expression do not represent biologically realistic states. The manifold hypothesis offers a widely recognized theoretical perspective that makes generative modeling in high-dimensional settings possible. It posits that many real-world datasets lie on intrinsically lower-dimensional geometric structures.

Once a generative model is learned, it can be used for three fundamental tasks: synthesis, density estimation, and representation learning. Synthesis is perhaps the most well-known capability of generative modeling with applications in image synthesis, text generation, and audio synthesis. It refers to the ability to draw "realistic" samples from the learned distribution samples that resemble observed data when $p_{model}(x) \approx p_{data}(x)$ without merely replicating them. Density estimation is the problem of explicitly or implicitly recovering the underlying probability density function of a dataset, allowing us to evaluate the likelihood of observations x under the learned model. This application is at the core of unsupervised learning tasks such as anomaly detection. Since many generative models first encode the input data into meaningful latent spaces z for reconstruction or generation back to a high-dimensional space ($p(x \mid z)$, following the manifold hypothesis), their learned representations can be used for feature extraction, manifold learning, and downstream tasks such as dimensionality reduction and visualization, classification, and clustering.

Taxonomy of Generative Models

The taxonomy tree of generative models (Fig. 2.1), famously established by Ian Goodfellow, can be broadly divided into two main branches, based on how they model the probability density function (PDF) $p_{model}(x)$ of the data they try to learn.

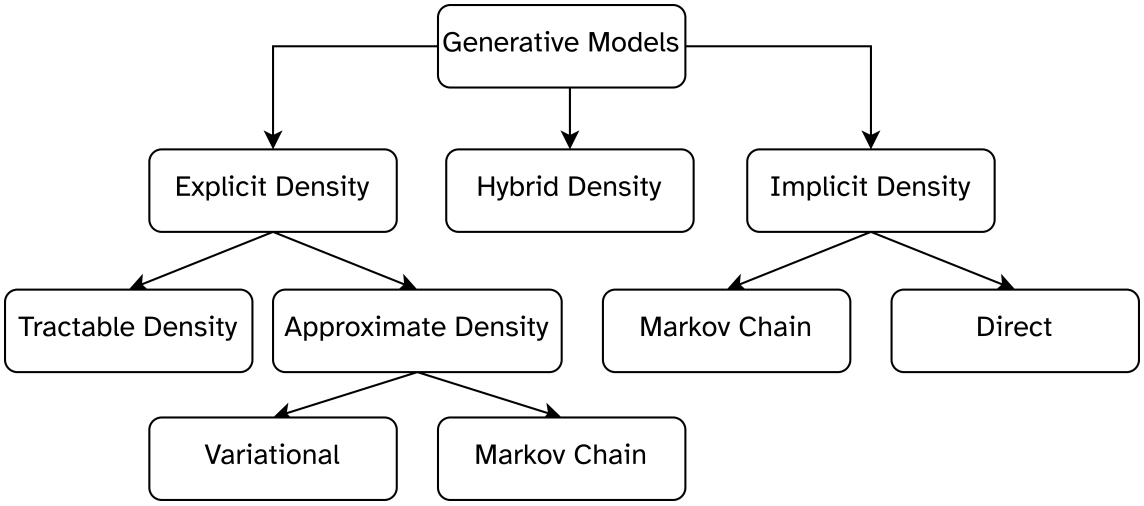


Figure 2.1: Taxonomy of generative models.

Explicit Generative Models

Explicit density models, as the name implies, directly represent and learn a PDF $p_{model}(x;\theta)$, controlled by parameter θ. Their objective is to maximize the likelihood of the training data by adjusting θ to make the observed data most probable under the model. Within this branch, models are further distinguished by the tractability of their density functions.

Tractable Explicit Density Models Models are classified as tractable when they allow for an exact evaluation and/or sampling of the likelihood at any given data point x without requiring computationally expensive numerical methods or approximations. Generative models in this category include normalizing flow-based models (e.g., NICE, RealNVP, Glow, MAF), autoregressive models (e.g., fully visible belief networks (NADE, MADE, PixelRNN, PixelCNN), WaveNet, and GPT), and change of variables models (e.g., nonlinear ICA).

Normalizing flow models, like many generative models, operate on the idea of transforming a simple base probability density $p_z(z)$, such as a multivariate Gaussian, into the complex target distribution $p_x(x)$ of the observed data through a series of invertible, differential mapping functions. To compute $p_x(x)$, these models apply the change of variables formula:

$$p_x(x) = p_z z) \left| \det \frac{\partial z}{\partial x} \right| = p_z(f^{-1}(x)) \left| \det \frac{\partial f^{-1}(x)}{\partial x} \right|,$$

where $\det \frac{\partial z}{\partial x}$ is the Jacobian determinant. Computing determinants of large matrices is generally very slow ($O(n^3)$). Through architectural designs such as block-diagonal, triangular, additive, or affine coupling layers, flow-based models simplify the calculation of the Jacobian determinant to $O(n)$.

Autoregressive models decompose a joint probability into a product of conditional probabilities, using the chain rule:

$$p(x) = p(x_1, \ldots, x_D) = \prod_{i=1}^{D} p(x_i \mid x_1, \ldots, x_{i-1}),$$

where x_i is the i-th variable, and D is the dimension of x (a sample). Instead of generating an entire sample (e.g., an image or text) at once, the model generates units (e.g., pixels, words, or tokens) one at a time, conditioned on all units that preceded it, in a fixed, predefined ordering. As such, sampling cannot be readily parallelized, long-range context is hard to maintain, small errors can accumulate or "snowball", making it difficult to maintain overall coherence over long sequences.

While these models provide a clear principled means of training via direct likelihood optimization, they are often computationally expensive. Autoregressive models suffer from slow inference, while normalizing flow models are constrained by the requirement for invertible transformations with tractable Jacobians, which can hinder their performance on some complex data distributions.

Approximate Explicit Density Models Oftentimes, the complexity of the target distribution makes the true density function mathematically intractable to compute or optimize directly, and models must rely on approximate density methods such as variational inference or Markov chain-based sampling . Variational inference introduces latent variables z and a parametric distribution $q_\phi(z|x)$ to approximate the true posterior $p(z|x)$. The objective becomes maximizing the Evidence Lower Bound (ELBO) on the log marginal likelihood:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z)}\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z) \,\|\, p_\theta(z)\big)$$

, where KL is the Kullback-Leibler divergence. In variational autoencoders (VAEs), the encoder and decoder neural networks parameterize $q_\phi(z|x)$ and $p_\theta(x|z)$, respectively. As VAE training involves stochastic gradient descent and directly sampling $z \sim q_\phi(z|x)$ would block backpropagation, the "reparameterization trick" is used to express z as a differentiable function of the

encoder outputs $\mu_\phi(x)$ and $\sigma_\phi(x)$ for each input x and random noise:

$$z = \mu_\phi(x) + \sigma_\phi(x) \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1),$$

Another class of models in this category is Markov-chain-based models, where the probability density is specified through an energy function. Sampling is done by modeling generation as a sequence of stochastic state transitions that converges to the desired distribution. Typical examples of energy-based methods (EMBs) include Boltzmann machines and their variations. They define a join distribution over visible units v (representing observed data) and hidden units h (that capture underlying features):

$$p(v, h; \theta) = \frac{\exp\left(-E(v, h; \theta)\right)}{Z(\theta)}$$

$$Z(\theta) = \sum_{v,h} \exp\left(-E(v, h; \theta)\right),$$

where $E(v, h; \theta)$ is the energy function and $Z(\theta)$ is the partition function. $p(v, h; \theta)$ specifies the likeliness of particular combinations of hidden and visible states, as determined by the overall energy associated with that configuration. Computing $Z(\theta)$ requires summing over all possible configurations, making its computation intractabe. We overcome this issue by relying on approximate Markov Chain Monte Carlo (MCMC) methods such as Contrastive Divergence and Gibbs sampling. Learning can be expressed as the difference between the expectation under the data distribution (positive phase) and the expectation under the model distribution (negative phase), approximated by running a short Markov chain initialized at the data.

$$\frac{\partial \mathcal{L}}{\partial w_{ij}} = \langle v_i h_j \rangle_{\text{data}} - \langle v_i h_j \rangle_{\text{model}}$$

Models like Deep Boltzmann Machines or Deep Belief Networks (composed of stacks of Restricted Boltzmann Machines (RBMs) use a similar concept for parameter learning and have explicit but approximate density.

Implicit Generative Models

Implicit generative models do not explicitly define a density function for the data. The goal is to sample realistic data points from a model distribution that is implicitly learned by the process used to generate the data itself, without relying on likelihood maximization. As a result, these models can only rely on indirect objectives such as adversarial and reconstruction losses or

learned stochastic transitions. Despite not being capable of density estimation, they synthesize very realistic data and empirically outperform explicit approximate density models such as VAEs.

Direct Implicit Density Models These models learn a direct mapping from a simple latent distribution (e.g., multivariate Gaussian noise) to the training data distribution (that is usually high-dimensional and complex) in a single pass without the need to sample iteratively. Generative Adversarial Networks, which we will discuss later on, are a classic example.

Markov Chain Implicit Density Models The sampling process in these models requires iteratively applying transitions until convergence by learning the stochastic transition operator of a Markov chain whose stationary distribution represents the target distribution. An emblematic example is Generative Stochastic Networks (GSNs), which generalize Denoising Auto-Encoders (DAE). GSNs learn the data manifold by being injected with noise and attempting to reconstruct the original data. They are trained at each step to predict a conditional distribution $P_\theta(X_{t+1} \mid H_t)$, where X_{t+1} is the next reconstructed sample and H_t is the hidden state at the current step of the Markov chain. Learning these local transitions is generally more simple than the entire data distribution. Because the Markov chain steps through small, local moves (as noise is injected at each step), its transition distributions usually include only a limited set of dominant modes near the current state and effectively becomes unimodal as moves become small. Consequently, GSNs' learning objectives resemble supervised function approximation rather than unsupervised global density estimation, and avoid computing intractable partition functions, unlike Boltzmann machines.

Hybrid Generative Models

This branch of the taxonomy includes models that incorporate elements from both explicit and implicit generative modeling. Diffusion models, for example, follow a fully fixed and tractable stochastic forward process that gradually adds noise to the data, forming an explicit likelihood-based model of data corruption. While diffusion models share the denoising mechanisms of GSNs, they are not implicit. GSNs use denoising to learn a transition operator for sampling. Meanwhile, in diffusion models, stepwise denoising serves as a proxy for score matching, where the model learns the gradient of the log-density of the data ($\nabla_x \log p(x)$). This allows diffusion models to be trained using likelihood-based objectives like ELBO, similar to VAEs. However, they differ from explicit models as their reverse process is intractable and learned implicitly through neural network approximations.

Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) are implicit generative models that learn a differentiable mapping function G, from a simple latent distribution (e.g., Gaussian or uniform noise) to the training distribution ($x_{gen} = G(z; \theta_g)$). To learn the function G, GANs follow a game-theoretic approach, using two players implemented as neural networks. A generator (G) with the goal of generating "fake" samples that closely mimic "real" samples (from the reference data) to deceive the second player. The discriminator (D) takes in samples from both the generator and the reference dataset, and tries to distinguish between them by assigning a probability that the input is real rather than fake. GANs are trained using stochastic gradient-based optimization methods, typically variants of the Adam optimizer, which update the parameters of both the generator and discriminator iteratively. During each update, one network's parameters are temporarily "frozen" while the other is optimized. GAN training involes the value function $V(D, G)$:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

The generator aims to minimize this objective while the discriminator seeks to maximize it. As the adversarial training approach is a zero-sum game (one player's gain is equivalent to the other player's loss), during training, the generator will update its parameters to maximize the probability of the discriminator misclassifying its outputs, while the discriminator, in turn, will update its parameters to better tell apart real and generated samples. By symbiotically providing feedback during training, the two adversaries can co-evolve together. The Nash equilibrium is reached when the generator outputs become indistinguishable from real data, and the discriminator outputs a probability of 0.5 for all inputs.

On the Convergence and Mode Collapse of GANs

GAN training, at least in the vanilla formulation introduced, is infamously fickle and unstable, and vulnerable to numerous failure modes. Mode collapse is one of the most common issues faced, where the generator learns to primarily produce from a subset of the reference data distribution, ignoring other modes. Intuitively, this issue arises when the generator has little incentive to generate from other modes of the data, when the discriminator consistently underperforms in its task. The adversarial training procedure must follow a delicate balance between the two players. Achieving this balance is a challenge, as GAN training is designed to follow the optimization procedure of a minimax game, which is not equivalent to that of the maximin game ($\min_G \max_D V(D, G) \neq \max_D \min_G V(D, G)$). The theoretical guarantees of GANs rely on solving the minimax game, but in practice, it is not possible to achieve the optimal

discriminator in a single step. Instead, training alternates between G and D, which can result in a drift toward a maximin-like regime, where the generator gets optimized against a suboptimal, fixed discriminator. A simple solution is to perform multiple updates of the discriminator for every update of the generator so that the generator is always provided with meaningful gradients. Unrolled GANs are another solution that mitigates this problem by "looking ahead" at future discriminator updates during the generator optimization.

GANs implicitly minimize the Jensen–Shannon (JS) divergence between $p_{model}(x)$ and $p_{data}(x)$, provided D is optimal. When the supports of these two distributions in low-dimensional manifolds are non-overlapping, the JS divergence saturates at a constant value of $\log(2)$. Consequently, training becomes unstable and the gradient of the loss function with respect to the generator's parameters vanishes, providing no feedback for the generator to evolve. To address this issue, Wasserstein GANs (WGANs) replace JS divergence with the Wasserstein-1 (Earth Mover's) distance:

$$W(p_{\text{data}}, p_g) = \inf_{\gamma \in \Pi(p_{\text{data}}, p_g)} \mathbb{E}_{(x,y)\sim\gamma} \left[\|x - y\| \right],$$

where $\Pi(p_{\text{data}}, p_g)$ are the set of all possible "transport plans" (joint distributions) that have marginals p_{data} and p_g. Intuitively, this distance measures the minimum cost of transporting mass from one distribution to the other and provides meaningful gradients even when the two distributions are disjoint in the low-dimensional manifold. The above equation is computationally intractable. For GAN training, it is rewritten using the Kantorovich-Rubinstein Duality as:

$$W(p_{\text{data}}, p_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x\sim p_{\text{data}}}[f(x)] - \mathbb{E}_{x\sim p_g}[f(x)]$$

Here, our critic (which replaces the discriminator as it outputs real-valued score instead of probabilities) approximates the function f and must be 1-Lipschitz continuous ($|f(x) - f(y)| \leq |x - y|$).

Early WGANs enforced this constraint by weight clipping, which limited model capacity. A more optimal approach involves penalizing the WGANs if the gradient norm of the Critic deviates from 1 by adding a gradient penalty term to the WGAN objective that is regulated by λ.

$$\mathcal{L}_D = \underbrace{\mathbb{E}_{\tilde{x}\sim p_g}[D(\tilde{x})] - \mathbb{E}_{x\sim p_{\text{real}}}[D(x)]}_{\text{Wasserstein distance}} + \underbrace{\lambda\, \mathbb{E}_{\hat{x}\sim p_{\hat{x}}} \left[\left(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1 \right)^2 \right]}_{\text{Gradient Penalty}},$$

where $\hat{x} = \epsilon x + (1 - \epsilon)\tilde{x}$ is a random sample interpolated between real and generated samples.

Wasserstein GANs with gradient penalty (WGAN-GPs) have shown to be successful in tackling model collapse and improving convergence and training stability across data modalities.

Conditional and Controllable Generation with GANs

Conditional GANs (cGANs) offer a method for directly generating samples from user-specified categorical labels y. The most direct form of conditioning involves concatenating one-hot-encoded class labels to the input of both the generator and the discriminator. In this setup, the generator produces $G(z|y)$, and the discriminator evaluates $D(x|y)$, generating samples that are not only realistic but also adhere to the provided conditions. Over the years, implementations of cGANs have been extended beyond discrete labels to continuous variables (InfoGAN), structured attributes (AC-GAN), textual embeddings (stackedGANs), feature maps (Pix2Pix, CycleGAN), and more. Furthermore, numerous new conditioning techniques have been proposed that improve class fidelity and training stability. For example, auxiliary classifier GANs (AC-GANs) add an additional classification head to the discriminator that predicts the conditioning label for both real and generated samples. In projection-based conditioning, the discriminator takes the inner product between learned class embeddings (V) and intermediate feature representations ($\phi(x)$) by projecting the one-hot label (y), and adds this term to its output score (resulting in $D(x, y) = y^T V\phi(x) + \psi(\phi(x))$). Conditioning can also be integrated in the generator using conditional batch normalization (CBN). In CBN, the scale ($\gamma(y)$) and shift ($\beta(y)$) parameters of normalization layers are made functions of the conditioning variable (y), modulating hidden activations (h): $\text{CBN}(h, y) = \gamma(y)\hat{h} + \beta(y)$.

A complementary line of work to cGANs is controllable generation, which offers more fine-grained control over synthesized outputs without the need for labeled samples by interpreting and deliberately sampling the latent space. These efforts were greatly influenced by the ability to interpolate the latent space and produce smooth semantic transitions on images of faces (e.g., from young to old) and perform arithmetic operations in the Z-space (e.g., $Z_{\text{smiling man}} = Z_{\text{smiling woman}} - Z_{\text{neutral woman}} + Z_{\text{neutral man}}$ produces a smiling man) in DCGAN. Yet, GAN latent spaces are entangled, and features are correlated, making the manipulation of an attribute affect other ones (we will discuss how to overcome this problem with CausalGANs). Substantial efforts have been made to achieve GAN latent space disentanglement. InfoGAN encourages disentanglement by introducing a mutual information regularization term to ensure that specific latent codes are strongly predictive of interpretable features in the generated data. Other methods penalize statistical dependence between latent dimensions. StyleGAN introduces a generator architecture with intermediate latent spaces to separate high-level semantics from stochastic variations at multiple levels of hierarchy.

CausalGAN

Conditional GAN architectures generate samples from class-conditional data distributions by conditioning on class labels that are modeled independently of eachother. A conditional GAN trained on images of faces, given labels *mustache* and *sex*, will simply sample from a subset of its learned training distribution (corresponding to $sex = male$) when conditioned on $mustache = true$. However, since the choice of one label (here, *mustache*) does not impact other labels (*sex*), we expect the model to output samples corresponding to both male and female labels. CausalGAN [1] captures interdependencies between labels themselves (e.g., *sex causes mustache* ($S \rightarrow M$)) through a causal graph (Figure 2.2) and the causal effect between labels and the output by conditioning on labels (i.e., *labels cause output* ($L \rightarrow O$)). By explicitly modeling the data-generating process as a causal mechanism, CausalGAN can not only sample from observational and conditional distributions but also interventional ones to synthesize samples that do not exist in the training dataset. Intervening on a label, for example, forcing $mustache = true$, only affects its descendants in the causal graph (e.g., *output*). This allows the model to generate both male and female faces with mustaches because the intervention breaks the causal link to the parent node (*sex*). In contrast, conditioning allows information to flow upstream to parent labels. By conditioning on mustache, we inadvertently update the probability of the *sex* label, shifting it heavily towards male.

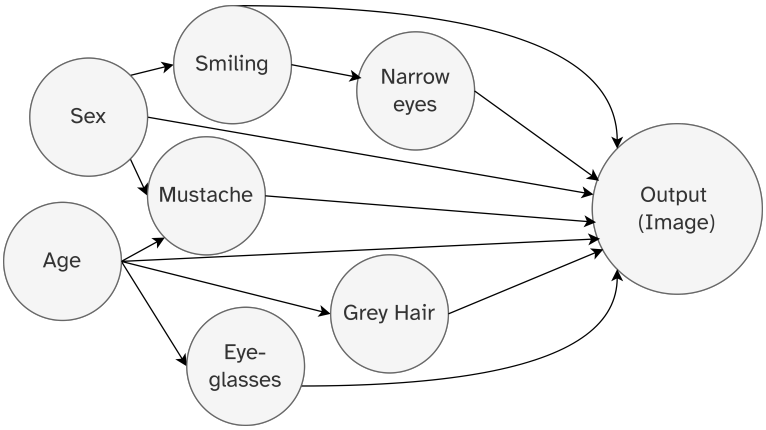


Figure 2.2: Example of a possible causal graph.

At the core of CausalGAN's architecture is a generator that strictly adheres to a user-provided causal graph (limited to a DAG), inheriting its neural connections directly from that causal structure. Thus, each variable is generated as a function of its parents in the DAG and independent noise. Returning to our example, under the causal sufficiency assumption, meaning there are no unobserved common causes (confounders) affecting the labels, the generation process for each node follows the path of the arrows:

- $N_S \rightarrow Sex$: represented as $Sex = f_S(N_S)$

- $\{Sex, N_M\} \rightarrow Mustache$: represented as $Mustache = f_M(Sex, N_M)$
- $\{Sex, Mustache, N_O\} \rightarrow Output$: represented as $Output = f_O(Sex, Mustache, N_O)$

where f_S, f_M, and f_O are functions that can represented by feedforward neural networks and the exogenous noise terms N_S, N_M, and N_O are jointly independent. In practice, simultaneously training such a generator to learn a joint distribution over discrete labels and high-dimensional images is challenging. CausalGAN employs a modular two-step approach.

1. Causal Controller (Label Generator): A causal implicit model is trained on all causal graph nodes (labels) preceding the output, using the DAG to arrange the generator's structure to generate labels in their topological order. Standard GAN training with Jensen-Shannon divergence has difficulties sampling from binary distributions; CausalGAN utilizes a WGAN-GP, which they empirically show can be used to output *essentially discrete* (sharply concentrated at 0 and 1) labels. Once trained, the causal controller is frozen and no longer updated.

2. Conditional Image Generator: This stage focuses on transitioning from all labels to the final output ($L \rightarrow O$). An image generator is trained conditioned on the outputs of the causal controller. Integrated with the pretrained causal controller, it forms a causal implicit generative model over both labels and images.

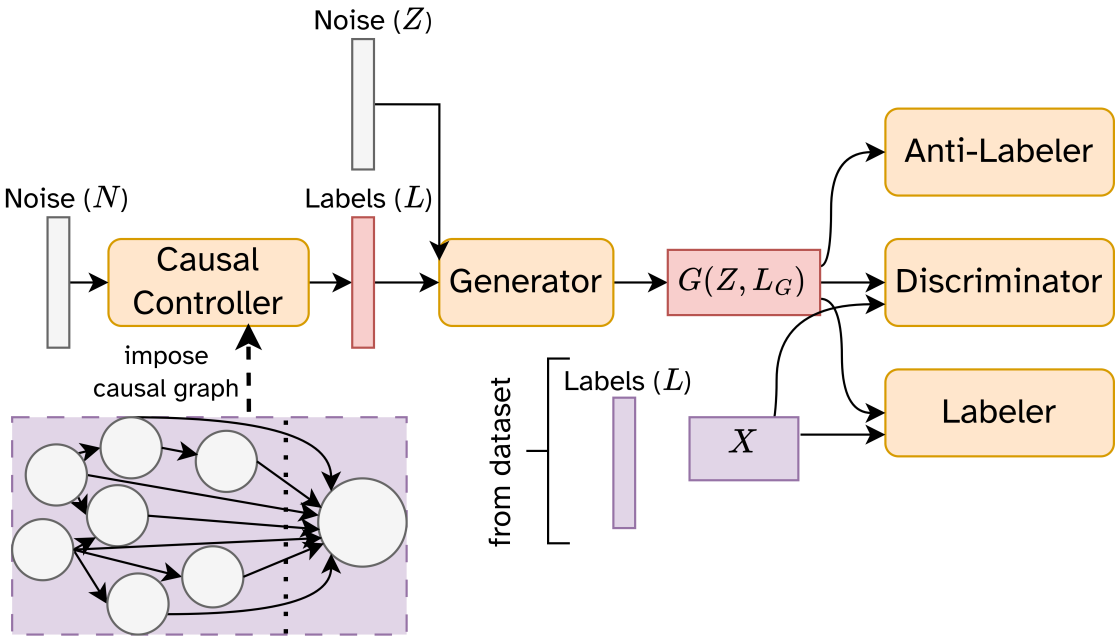


Figure 2.3: The CausalGAN Architecture (adapted from CausalGAN [1]).

In addition to the generator (G) and discriminator (D), CausalGAN extends the standard adversarial framework by incorporating two auxiliary networks: a Labeler and an Anti-Labeler

(Figure 2.3). The Labeler is trained on real and generated images, tasked with recovering labels used to produce them. By minimizing the Labeler loss, we encourage the conditional image generator to capture labels. Conversely, the Anti-Labeler is trained only on synthetic samples with the same task. Counterintuitively, the Anti-Labeler loss is maximized and serves as a critical regularizer in CausalGAN. If the generator were to generate repeated samples for specific label combinations, the Anti-Labeler would easily achieve high accuracy in the early stages of training. By maximizing this loss, the generator is effectively penalized for label-conditioned model collapse, encouraging it to produce more diverse samples. As training progresses, maximizing Anti-Labeler loss and minimizing Labeler loss become conflicting objectives. CausalGAN exponentially decays the weight of the Anti-Labeler loss as training progresses.

Under mild assumptions, notably that the joint distribution over labels is strictly positive, theoretical guarantees for CausalGAN establish that the two-stage procedure produces a valid causal implicit generative model, where the generator is proven to match the true class conditional output distribution ($p_{data}(x|l)$) at its global minimum. While this guarantee is for single binary labels, it can extend to multiple binary labels using cross-entropy losses for each label. This setup primarily captures individual label posteriors $p_r(l_i|x) = p_g(l_i|x)$, where $p_r(l_i|x)$ is the probability of the i-th label being present in a real image and $p_g(l_i|x)$ is the same for a fake image. However, it is sufficient to recover the full joint label posterior given the image in cases where the labels are completely determined by the image (as is the case for the CelebA dataset [2] used in the CausalGAN study).

In-Silico Simulation of scRNA-seq

GANs Applied to scRNA-seq

Numerous GAN-based methods were developed to tackle challenges in the scRNA-seq domain, including data augmentation, dropout imputation, batch effect correction, and cross-modality translation. Ghahramani et al. were one of the first to apply GANs to scRNA-seq. They found that WGAN-GPs lead to better training stability than vanilla GANs. This study also examined the use of learned features for dimensionality reduction and demonstrated a case of controllable generation by interpolating in the latent space (between z_{basal} and $z_{\text{differentiated}}$) to predict intermediate gene expression profiles of epidermal differentiation.

However, applied to 20000 genes, realism was subpar, and synthetic samples failed to capture the entire diversity of samples, even when qualitatively evaluated on a 2-dimensional t-SNE embedding. scRNA-seq data are highly heterogeneous with great cell-to-cell variability, even for the same cell types and populations. By library-size normalizing input cells, scGAN saw a significant improvement in simulation quality, likely due to gradient stabilization resulting from

gene expression being similarly ranged across cells. Furthermore, scGAN introduced a library-size normalization layer (LSN) to explicitly output the same read count as the training data for all generated cells, leaving the GAN with one less feature to learn. The same study also introduced cscGAN, a projection-based conditional extension of scGAN with CBN layers for conditioning on labels such as cell type, state, and cluster. Using cscGAN for data augmentation led to improvements in downstream classification and trajectory analysis tasks. Other augmentation methods include LSH-GAN, which guides the generator using locality-sensitive hashing to focus training on informative subsets of real samples, and GARAGE, which augments the generator's input with attention-weighted embeddings of real cells derived from a k-nearest-neighbor graph via a graph attention network.

In scRNA-seq, dropouts are zero-valued gene expression measurements due to technical limitations, such as low mRNA capture. Imputing these missing values is a challenge, as scRNA-seq data is sparse and standard methods that rely on borrowing information from similar cells risk oversmoothing and removing biological noise. scIGAN uses generated cells rather than observed cells to avoid these problems. scGGAN learns gene-gene relations using Graph Convolutional Networks (GCN)-based GAN, which uses a learned gene-gene network to simulate data used for imputation. scMultiGAN also does imputation through first training two generators, one to simulate authentic gene expression (without dropout) and another to model dropout. Then, a third generator uses pre-trained knowledge from the first training step to input raw expression and output imputed values. All the models discussed for imputation first reshape scRNA-seq data into an image-like format for training.

GANs are also used to remove batch effects (non-biological differences between datasets from different experiments). For example, AWGAN uses attention-guided WGANs, while ResPAN employs WGAN-GPs along with random walk mutual nearest neighbor pairing and fully skip-connected autoencoders to map cells from a query batch to a reference batch. Methods such as scMMGAN and scACT use cycle-consistency, inspired by CycleGAN, to integrate or map measurements across modalities, including scRNA-seq.

Background and Context for "*GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks*"

The article presented in Chapter 3 was first published in *Nature Communications* (Volume 15, Article 4055) on May 14, 2024. It is open access and made available under the Creative Commons Attribution 4.0 International (CC BY 4.0) license © Yazdan Zinati, Abdulrahman Takiddeen, and Amin Emad, 2024., which permits its inclusion in this thesis. DOI: 10.1038/s41467-024-48516-6.

Chapter 3

GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks

Abstract

We introduce GRouNdGAN, a gene regulatory network (GRN)-guided reference-based causal implicit generative model for simulating single-cell RNA-seq data, in silico perturbation experiments, and benchmarking GRN inference methods. Through the imposition of a user-defined GRN in its architecture, GRouNdGAN simulates steady-state and transient-state single-cell datasets where genes are causally expressed under the control of their regulating transcription factors (TFs). Training on six experimental reference datasets, we show that our model captures non-linear TF-gene dependencies and preserves gene identities, cell trajectories, pseudo-time ordering, and technical and biological noise, with no user manipulation and only implicit parameterization. GRouNdGAN can synthesize cells under new conditions to perform in silico TF knockout experiments. Benchmarking various GRN inference algorithms reveals that GRouNdGAN effectively bridges the existing gap between simulated and biological data benchmarks of GRN inference algorithms, providing gold standard ground truth GRNs and realistic cells corresponding to the biological system of interest.

Introduction

Unraveling gene regulatory interactions, often represented as a gene regulatory network (GRN), plays a crucial role in studying biological processes under different conditions[1,2,3], simulating knockdown and knockout experiments[4,5], and identifying therapeutic drug targets[6,7]. Many al-

gorithms have been proposed for GRN reconstruction using bulk or single-cell RNA sequencing data (scRNA-seq), alone[8,9,10,11,12,13,14] or with other modalities[15,16,17,18]. While these advances have provided great biological insights, evaluating the performance of GRN Inference algorithms remains challenging[19,20] due to the lack of reliable ground truth for the biological processes under study. Existing evaluation approaches often resort to curated databases[21,22,23]. However, the regulatory interactions in these databases are aggregated from a wide range of datasets and are not specific to a biological system, making them not ideal benchmarks due to context specificity of gene regulation. Another strategy is to verify regulatory interactions by conducting perturbation experiments on the system under study[20]. However, this approach is tedious, lengthy, and expensive. Another approach is to employ scRNA-seq simulators. Despite great progress in this domain, most simulators lack the essential properties for this task, such as the preservation of gene identities and simulation based on a user-provided ground truth GRN. For example, scGAN[24], cscGAN[24], scDESIGN2[25], and SPARSIM[26] capture inter-gene correlations in their datasets. However, since they do not explicitly impose a known GRN (that can act as the ground truth), they are not suitable for benchmarking GRN inference methods.

A small subset of simulators (e.g., BoolODE[19], SERGIO[4], and GeneNetWeaver (GNW)[27]) explicitly incorporate GRNs capturing transcription factor (TF)-gene dynamics. GNW was used to generate benchmarks for Dialogue for Reverse Engineering Assessment and Methods (DREAM) challenges[27,28,29]. However, being a bulk RNA-seq tool, its simulated datasets do not replicate the distribution of experimental scRNA-seq datasets nor exhibit their statistical properties, despite attempts to adapt it for this purpose by externally inducing dropout events[10,30]. SERGIO and BoolODE were designed to simulate scRNA-seq data using stochastic differential equations (SDEs)[4,19] and have been used to benchmark a variety of GRN inference methods. However, often a mismatch between the benchmarking results based on experimental and their simulated data have been reported, which may be attributed to differences between the simulated and experimental datasets. For example, in the BEELINE study (using BoolODE), some of the top ranking methods on simulated benchmarks reported near random performance on curated and experimental benchmarks[19]. Both simulators enable the user to simulate more realistic datasets by carefully selecting the values of the SDE parameters. Moreover, using a reference dataset, SERGIO allows fine tuning the added technical noise in an iterative procedure until the generated and reference datasets are matched[4]. While these steps can help improve the resemblance to real datasets, they are often suboptimal and pose an undue burden on the user: for example, in SERGIO, the user must evaluate the similarity based on five different statistics and change three parameters iteratively until desired resemblance is achieved (which itself can be subjective). In addition, since the GRN is imposed on the "clean" dataset, but technical noise is added afterwards, this step may change the encoded causal relationships in a non-trivial way (which may explain why the performance of GRN inference methods dropped close to random when applied to the noisy dataset[4]). Furthermore, SERGIO and BoolODE do not readily pre-

serve gene identities and make simplifying assumptions regarding cooperative regulation (see Discussion). These shortcomings demonstrate the need for simulators capable of generating realistic scRNA-seq data that retain the regulatory dynamics specified by a user-defined GRN. Importantly, with the growing interest in causal inference, simulators that impose causal GRNs are of great need.

GRouNdGAN (GRN-guided in silico simulation of single-cell RNA-seq data using Causal generative adversarial networks) is a causal implicit generative model for reference-based GRN-guided simulation of scRNA-seq data inspired by CausalGAN[31]. Given an input GRN and a reference scRNA-seq dataset, it can be trained to generate simulated data that is both indistinguishable from the reference data and faithful to the causal regulatory interactions of the input GRN. Unlike model-based simulators that rely on simplifying assumptions regarding co-regulatory patterns, in GRouNdGAN these patterns are *learned* through complex functions. This allows it not to compromise on the underlying complexity of the system and to model elaborate regulatory dynamics. GRouNdGAN provides state-of-the-art performance in realistic scRNA-seq data generation, while preserving gene identities, causal gene regulatory interactions, and cellular dynamics (e.g., lineage trajectory and pseudo-time ordering). This is achieved through implicit parameterization without the need for manual fine-tuning. Using GRouNdGAN, we benchmark eight GRN inference methods and find the results to be aligned with BEELINE's experimental results[19]. Furthermore, the causal structure of GRouNdGAN enables it to be used for sampling from both interventional and observational data distributions, enabling in silico knockout experiments.

Results

GRouNdGAN generates scRNA-seq data using causal generative adversarial networks

GRouNdGAN is a deep learning model that generates scRNA-seq data while imposing a user-defined causal GRN to describe the regulatory relationships of the genes and TFs. Its architecture builds on the causal generative adversarial network[31] and includes a causal controller, target generators, a critic, a labeler and an anti-labeler (Fig. 1). Training includes two steps. First, the causal controller that generates TF expression values is pre-trained (Fig. 1B) as the generator of a Wasserstein GAN (WGAN)[32] with gradient penalty (WGAN-GP) (Methods). In the second step (Fig. 1C), TF expression values (generated by the pre-trained causal controller) and randomly generated noise are provided as input to the target generators that produce target genes' expressions, while incorporating the TF-gene relationships of the input GRN. To achieve

this, as input, each target generator only accepts a noise value and the generated expression values of TFs that causally regulate it (Fig. 1C, Methods). The generated expression of genes and TFs are then fed to a library-size normalization (LSN) layer[24].

The two steps of training above are performed based on the same reference experimental scRNA-seq training dataset. The goal of the first step (pre-training, Fig. 1B) is to train the causal controller to learn scRNA-seq data distribution and generate realistic data, irrespective of any GRN. The GRN edges between a set of regulators (e.g., TFs) and their targets are imposed in the second step of training (Fig. 1C). To achieve this, in the second step of training, only the expression of regulators (TFs) outputted by the causal controller are used and its output of target genes' expressions are discarded and instead are re-generated using the target generator neural networks in Fig. 1C to enable imposition of causal relationships.

The critic's role (in both steps of training) is to quantify the Wasserstein distance (Supplementary Notes) between the reference and simulated data. The target generators are trained in an adversarial manner to generate realistic datapoints indistinguishable from the reference datapoints by the critic. The labeler/anti-labeler estimate the TF expression values *only* from the target genes' expressions to ensure that the generated causal TF-gene dependencies are encoded. The combination of the model architecture (in that each target generator only receives a noise value and the simulated expression of its regulating TFs in the GRN) and the labeler/anti-labeler ensure that the GRN edges are causally imposed in the simulated data. Details are provided in Methods, the architectural choices in Supplementary Tables 1 and 2, and an ablation study in Supplementary Notes and Supplementary Table 3.

GRouNdGAN generates realistic scRNA-seq data

We first trained GRouNdGAN on three datasets (results for three additional datasets will be discussed later). The first dataset contained scRNA-seq profiles of 68,579 human peripheral blood mononuclear cells (PBMCs) from 10x Genomics[33] corresponding to eleven cell types ("PBMC-All"). We formed a dataset of the most common cell type in the PBMC-All dataset containing 20773 CD8+ Cytotoxic T-cells ("PBMC-CTL"). Additionally, we obtained the scRNA-seq (MARS-seq) profile of 2730 cells corresponding to differentiation of hematopoietic stem cells to different lineages from mouse bone marrow[34] ("BoneMarrow"). We used GRNBoost[28] to identify fifteen TFs for each gene to form the input GRN. Note that these identified interactions may contain both spurious correlations and causal relationships in the reference dataset. However, when they are used as input to GRouNdGAN, they will correspond to causal interactions in the *simulated* dataset (see Kocaoglu et al. for theoretical details of causality in this architecture[31]). For each dataset, we trained GRouNdGAN on a randomly selected training set and evaluated on a held-out test set (Methods, Supplementary Data 1 –

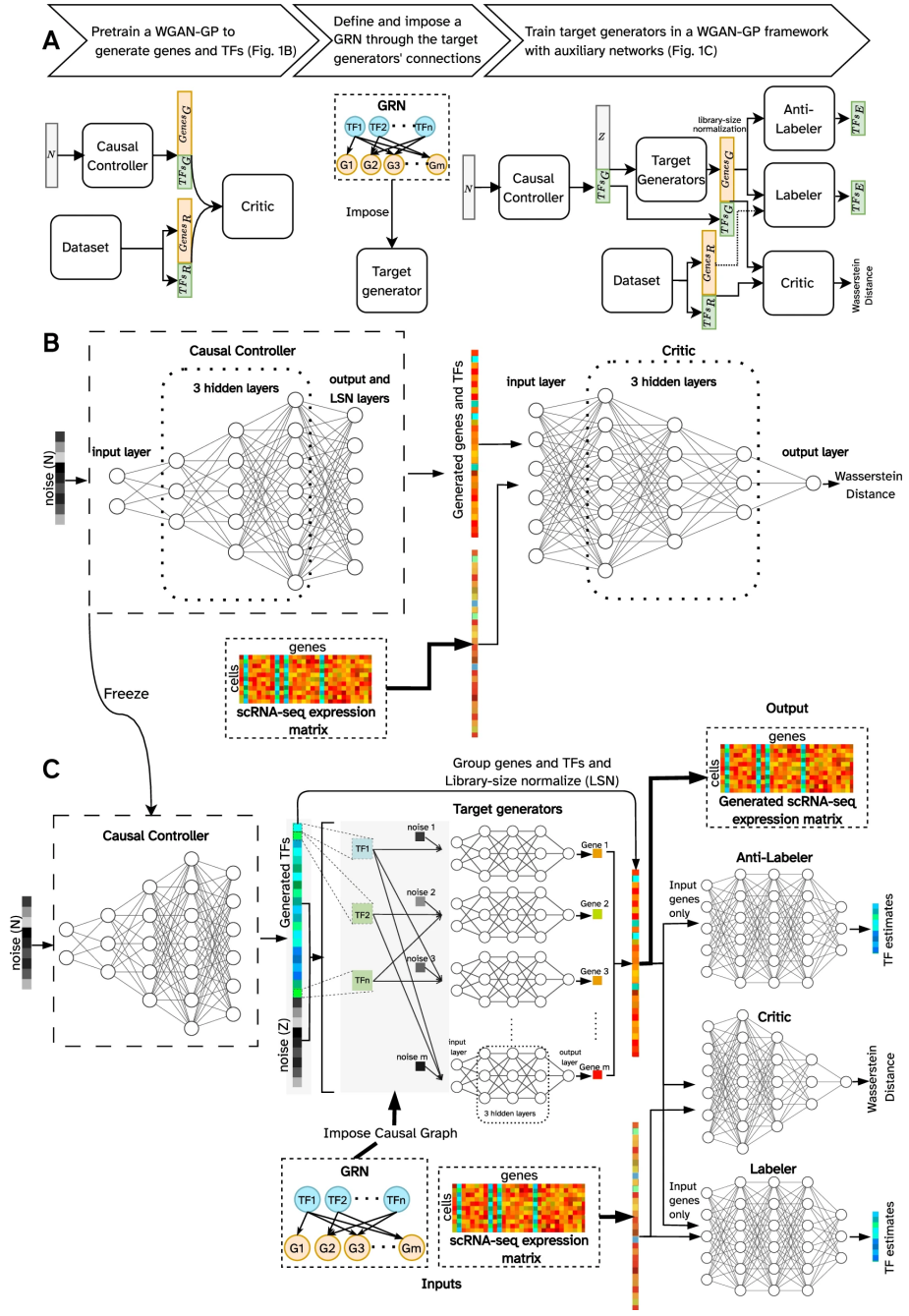


Figure 1: Architecture and training procedure of GRouNdGAN. **A** The flowchart representing the steps of the training procedure and the overall architecture of the model in each step. Subscripts G, R, and E represent generated, real, and estimated, respectively. **B** A WGAN-GP is pre-trained to generate realistic simulated cells using the reference (real) training set. **C** The LSN layer of the generator of the trained WGAN-GP (panel **B**) is removed, its weights are frozen, and is used as the causal controller to generate unnormalized TF expression values (expression of target genes generated by the causal controller are discarded). These TF expression values along with a noise vector are provided as input to the target generators, following the provided causal GRN. The generated gene and TF expression values are reorganized and passed through the LSN layer. The normalized simulated expression vectors and experimental reference data (the same training set as **B** are then passed to the critic to estimate Wasserstein distance between the reference and the generated data distributions. The anti-labeler estimates TF values based on generated target gene expressions. The labeler performs a similar task, but in addition to receiving generated values, it also utilizes target gene expression values from the reference data. Labeler and anti-labeler ensure that the causal GRN is incorporated by the target generators. Details of the model are provided in Methods.

Table 1: Performance of different simulators in generating realistic scRNA-seq data using the PBMC-CTL dataset

Simulator	Cosine distance	Euclidean distance	MMD	RF AUROC	miLISI
GRouNdGAN	**0.00057**	**182**	**0.026**	**0.54**	**1.891**
scGAN[24]	0.00095	222	0.031	0.59	1.888
scDESIGN2[25]	0.00100	229	0.065	0.76	1.736
SPARsim[26]	0.00104	235	0.309	0.95	1.625
Control	0.00019	99	0.012	0.50	1.909

The metrics are calculated between a simulated dataset of 1000 cells and the held-out test set of 1000 real cells (see Supplementary Data 1 – Sheet 2 for training set performance). Each gene in the imposed GRN of GRouNdGAN is regulated by 15 TFs (constructed using GRNBoost2 from the experimental training set). For the first three metrics, a value closer to zero is preferred, for RF AUROC a value closer to 0.5 is preferred, and for miLISI a value closer to 2 is preferred. For the first two metrics, the values correspond to the distance of the mean centroids of the real and simulated cells. The RF AUROC of control corresponds to perfect performance (of a random classifier). The other control metrics are calculated using the two halves of the real test dataset. Best performance values (excluding control) are in bold-face.

Sheets 2 and 3). Figure 2A, B and Supplementary Figs. 1–3 show the t-SNE plots of reference and simulated cells, qualitatively revealing their similarity.

We quantitatively assessed the resemblance using Euclidean distance, Cosine distance, maximum mean discrepancy (MMD)[35], mean integration local inverse Simpson's index (miLISI)[36], and the area under the receiver operating characteristic curve (AUROC) of a random forests (RF) classifier distinguishing simulated and experimental cells (Methods). As a "control" (and to enable calibration of these scores), we also calculated these metrics using two halves of the reference test set (corresponding to real cells). Table 1 shows the performance of GRouNdGAN, control, and three state-of-the-art simulators for the PBMC-CTL dataset (see Supplementary Data 1 – Sheet 2 for training set performance). We did not include BoolODE in these tables since it is a reference-free simulator and is not designed to match a particular reference dataset. We tried generating samples using SERGIO to match the PBMC-CTL dataset, but the simulated dataset did not show high resemblance to the PBMC-CTL dataset (see Supplementary Notes, Supplementary Tables 4 and 5 and Supplementary Fig. 4). However, we decided not to include these results in Table 1, since SERGIO relies on the user's judgement for distribution matching and the quality of the final simulated dataset is user dependent. Instead, we only included reference-based simulators that automatically generate scRNA-seq data resembling experimental data, in order to assess data quality systematically and fairly.

One of the challenges faced by GANs is the possibility of mode collapse: when applied to heterogenous datasets, they may learn to generate only a limited set of examples, instead of generating a variety of examples from the entire distribution of the training data. We trained GRouNdGAN using PBMC-All, which contains multiple cell types, to determine whether it

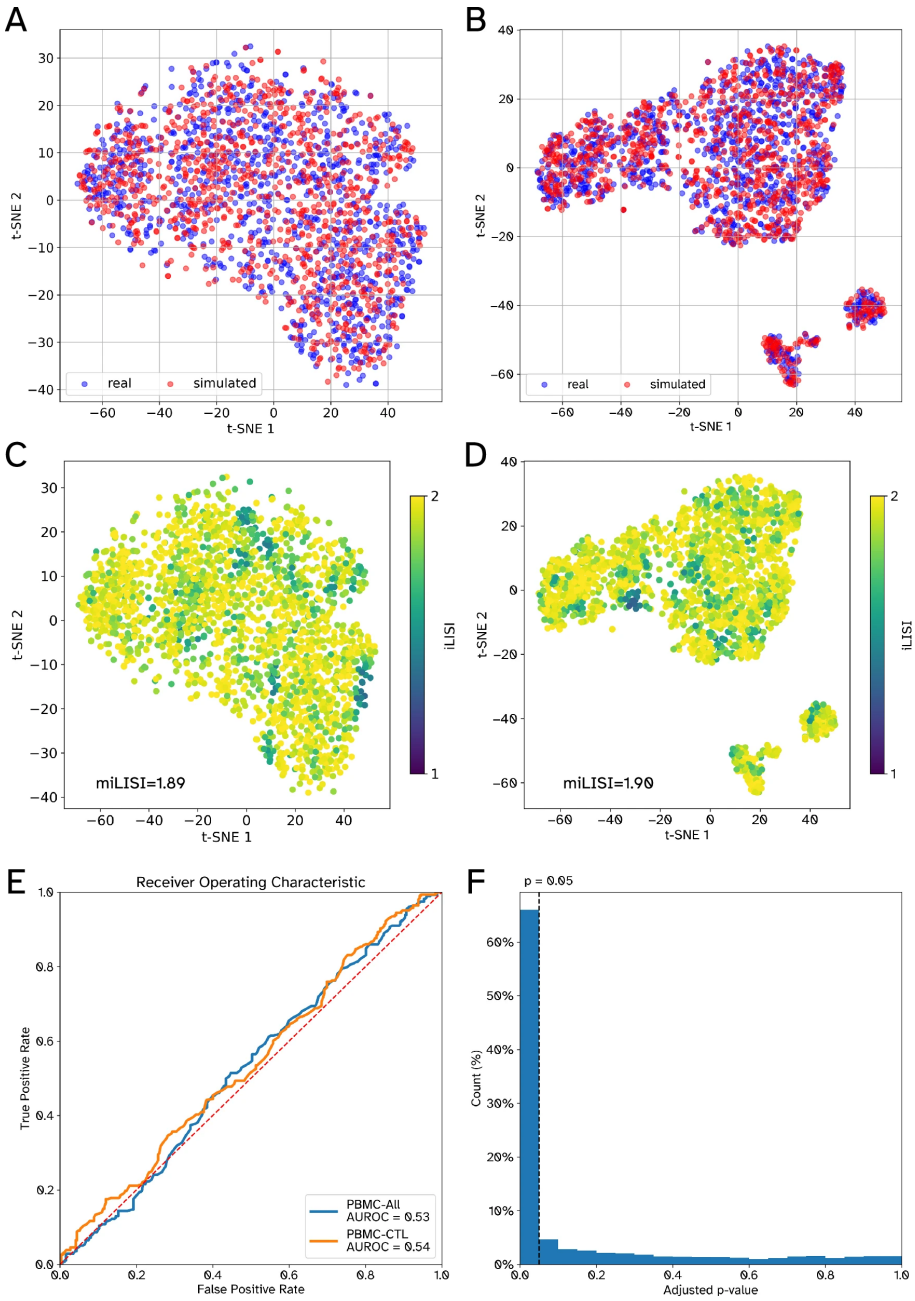


Figure 2: Performance of GRouNdGAN in generating realistic scRNA-seq data. All plots correspond to 1000 real (experimental) held-out test cells and 1000 simulated cells. Each gene in the imposed GRN of GRouNdGAN is regulated by 15 TFs (identified using GRNBoost2 from the experimental training set). Panels **A** and **B** depict the t-SNE plots of real (blue) and simulated (red) cells for PBMC-CTL and PBMC-All, respectively, while panels **C** and **D** also show their iLISI distributions. **E** The ROC curve of a Random Forest classifier in distinguishing between simulated and real cells. **F** The distribution of adjusted p-values corresponding to the TF perturbation study on the PBMC-CTL dataset. The p-values are obtained using two-sided Wilcoxon signed ranked tests and are adjusted for multiple hypotheses following the Benjamini–Hochberg procedure. Source data are provided as a Source Data file.

can generate samples from different cell types present in this dataset, or it will suffer from mode collapse and only generates samples corresponding to the most represented cell type. This is particularly important, since many scRNA-seq datasets are heterogenous and contain different cell types or cells in different states. If cell type information of the reference dataset is available a priori, one can easily generate realistic samples of each cell type separately (similar to the PBMC-CTL analysis) and use either cell type-specific GRNs, a shared GRN, or a combination of both. However, we were particularly interested to know whether our model could generate realistic samples *without* this knowledge, a more challenging task and useful for when such information is unavailable. Also, not requiring cell type information reduces the amount of user involvement (to annotate cell types) and improves model's usability. We repeated the analysis using the PBMC-All dataset, containing eleven cell types. Although GRouNdGAN did not receive cell type/cluster information, we did not observe mode collapse (Supplementary Fig. 2), and it was able to generate cells from distinct clusters (Fig. 2B, test set miLISI = 1.90). GRouNdGAN outperformed all simulators that did not use cell cluster information (Table 2) and compared to those that utilized this information, it was still the top performing based on MMD and RF AUROC and the second best according to the Euclidean distance (6% higher than the best) and miLISI (0.5% lower than the best) (Table 2). Moreover, we observed a high degree of concordance between the cell type marker expressions in the experimental data and GRouNdGAN-simulated data (Supplementary Notes and Supplementary Fig. 5). Finally, GRouNdGAN outperformed all other simulators based on all metrics when we repeated the analysis using the BoneMarrow dataset, containing continuous cell states and much fewer cells (Supplementary Data 1 – Sheet 3). These results show that GRouNdGAN is a stable simulator of scRNA-seq data and even without extra cell type information in heterogenous datasets, it automatically generates realistic samples reflecting different cell types (while imposing a GRN). See Supplementary Notes and Supplementary Data 2 for stability analysis and the effect of different GRN properties (including number of TFs) on the performance.

GRouNdGAN imposes a causal GRN in the simulated data

To assess the ability of GRouNdGAN in imposing the input causal GRN, we performed in silico TF knockout experiments (one of GRouNdGAN's capabilities) on the simulated cells ($n = 1000$) from the PBMC-CTL dataset. We performed a forward pass after setting the expression of each TF to zero (one at a time) in the causal controller output, while keeping all other parameters and TF expressions unchanged, ensuring that the perturbations were performed on the same batch of cells forming matched case/control experiments. There was no change in the expression of genes that were not regulated by the knocked-out TF (as expected), while the expression of the regulated genes changed. Figure 2F shows the distribution of the adjusted p-values (Benjamini–Hochberg) for each TF-gene edge in the GRN, comparing the expression of a gene across all cells before

Table 2: Performance of different simulators in generating realistic scRNA-seq data using the PBMC-All dataset

Simulator	Cell cluster labels/ratios provided as input	Cosine distance	Euclidean distance	MMD	RF AUROC	miLISI
GRouNdGAN	No	0.00028	**168**	**0.017**	**0.53**	1.90
cscGAN[24]	Yes	0.00025	242	0.030	0.56	**1.91**
cWGAN	Yes	0.00053	239	0.027	0.57	1.89
scGAN[24]	No	0.00081	300	0.035	0.58	1.90
scDESIGN2[25]	Yes	0.00032	181	0.046	0.86	1.81
scDESIGN2[25]	No	0.00057	256	0.124	0.91	1.53
SPARsim[26]	Yes	**0.00019**	**158**	0.287	0.96	1.66
SPARsim[26]	No	0.00035	245	0.307	0.96	1.39
Positive control	NA	0.00022	116	0.012	0.50	1.91

Baseline models that enable utilization of cell cluster labels or ratios are run with and without this information. The metrics are calculated between a simulated dataset of 1000 cells and the held-out test set of 1000 real cells. Each gene in the imposed GRN of GRouNdGAN is regulated by 15 TFs (constructed using GRNBoost2 from the experimental training set). For the first three metrics, a value closer to zero is preferred, for RF AUROC a value closer to 0.5 is preferred, and for miLISI a value closer to 2 is preferred. For the first two metrics, the values correspond to the distance of the mean centroids of the real and simulated cells. The RF AUROC of control corresponds to perfect performance (of a random classifier). The other control metrics are calculated using the two halves of the real test dataset. Best performance values (excluding control) are in bold-face.

and after the knockout of one of its regulating TFs (two-sided Wilcoxon signed-rank tests). In the majority (66.5%) of cases, the knockout of a gene's regulating TF significantly (adjusted p-value < 0.05) altered its expression, showcasing that the GRN is indeed imposed in the expression profiles of simulated cells. Note that since each gene is regulated by multiple TFs, we cannot expect that knockout of a single TF result in a significant change in the expression of its target genes in all cases. The analysis above shows that GRouNdGAN does not simply ignore the input TF values to instead rely solely on the noise input to generate realistic cells, and the effect of TF-gene edges are indeed imposed. Moreover, since each gene is regulated by multiple (15) TFs, but the knockout experiment is one TF at a time, this further shows that individual edges are imposed by the model.

GRouNdGAN does not require the input GRN to be the true causal GRN of the training data (we will discuss this further later). To further clarify this, we replaced GRNBoost2 with PPCOR[37] to form the GRN, which is a method based on (partial) correlation for inferring GRNs (see Supplementary Data 1 – Sheet 2 for resemblance metrics). Repeating the knockout experiments above ($n = 1000$ cells) resulted in similar results showing that in the majority (75.2%) of cases, the knockout of a gene's regulating TF significantly altered its expression (Supplementary Fig. 6). Taken together, these results show that GRouNdGAN can successfully impose different GRNs in the simulated data, while generating realistic cells.

The imposed GRN can be reconstructed from the simulated data

To test whether the imposed GRN can be reconstructed from the generated data, we first applied GRNBoost[28] to the reference PBMC-CTL dataset and recorded the top ten TFs for each gene. For each gene, we created two sets of five TFs based on their even/odd parity in the ranked list: TFs ranked 1st, 3rd, 5th, 7th, and 9th were connected to the gene and were imposed ("positive control GRN"), while the remaining five TFs were not ("negative control GRN"), resulting in two GRNs with identical densities and the same number of TFs and target genes. When considering the positive and negative control GRNs as ground truths (separately), the GRN reconstructed using GRNBoost2 from the reference (training) dataset ($n = 19{,}773$) resulted in comparable AUPRC (0.28 and 0.24, respectively, Supplementary Data 3 – Sheet 2), reflecting that they contain edges of comparable importance.

We simulated data ($n = 19{,}773$ cells) using GRouNdGAN by imposing only the positive control GRN, and used GRNBoost2, PIDC[10] GENIE3[9], and PPCOR[37] to reconstruct the underlying GRN (Fig. 3, Supplementary Fig. 7, Supplementary Data 3 – Sheet 2). All methods reconstructed the imposed edges with a much higher AUPRC compared to the reference dataset, while they did not assign high scores to the unimposed edges, resulting in close to random AUPRC. Figure 3 shows the performance of GRNBoost2 applied to the reference, GRouNdGAN-simulated, and scGAN-simulated data ($n = 19{,}773$ cells). The performance for scGAN (both GRNs) was lower than the reference dataset, showcasing that it had not preserved the TF-gene relationships of the experimental dataset well. The performance was relatively consistent between positive (Fig. 3A, C) and negative control (Fig. 3B, D) GRNs using the reference and scGAN-simulated data; however, it was much higher for the positive control GRN (compared to negative control GRN) using GRouNdGAN-simulated data. Similar patterns were observed when we repeated these analyses using the PBMC-All ($n = 67{,}579$ cells) and BoneMarrow ($n = 2230$ cells) datasets (Supplementary Data 3 – Sheets 3 and 4, Supplementary Figs. 8 and 9). These results show that the imposed edges were accentuated by GRouNdGAN, while the unimposed edges were disrupted and could not be found by GRN inference methods.

Next, we asked whether GRNBoost2 can reconstruct the imposed GRN, if GRNs of different densities are used to simulate data. Using data for which each gene was regulated by 15, 10, 5, and 3 TFs, we observed that in all cases the GRN imposed by GRouNdGAN could be inferred from the simulated data (Supplementary Data 3 – Sheets 5 and 6). Finally, we repeated the controlled analysis above, swapping the role of positive and negative control GRNs. Similar to the results of Fig. 3, the imposed edges were inferred by GRNBoost2, while the unimposed edges were not accurately discovered (Supplementary Data 3 – Sheet 7). These results show that 1) GRouNdGAN imposes the causal GRN, 2) the GRN inference methods can identify the imposed edges, and 3) the TF-gene relationships present in the reference dataset but unimposed

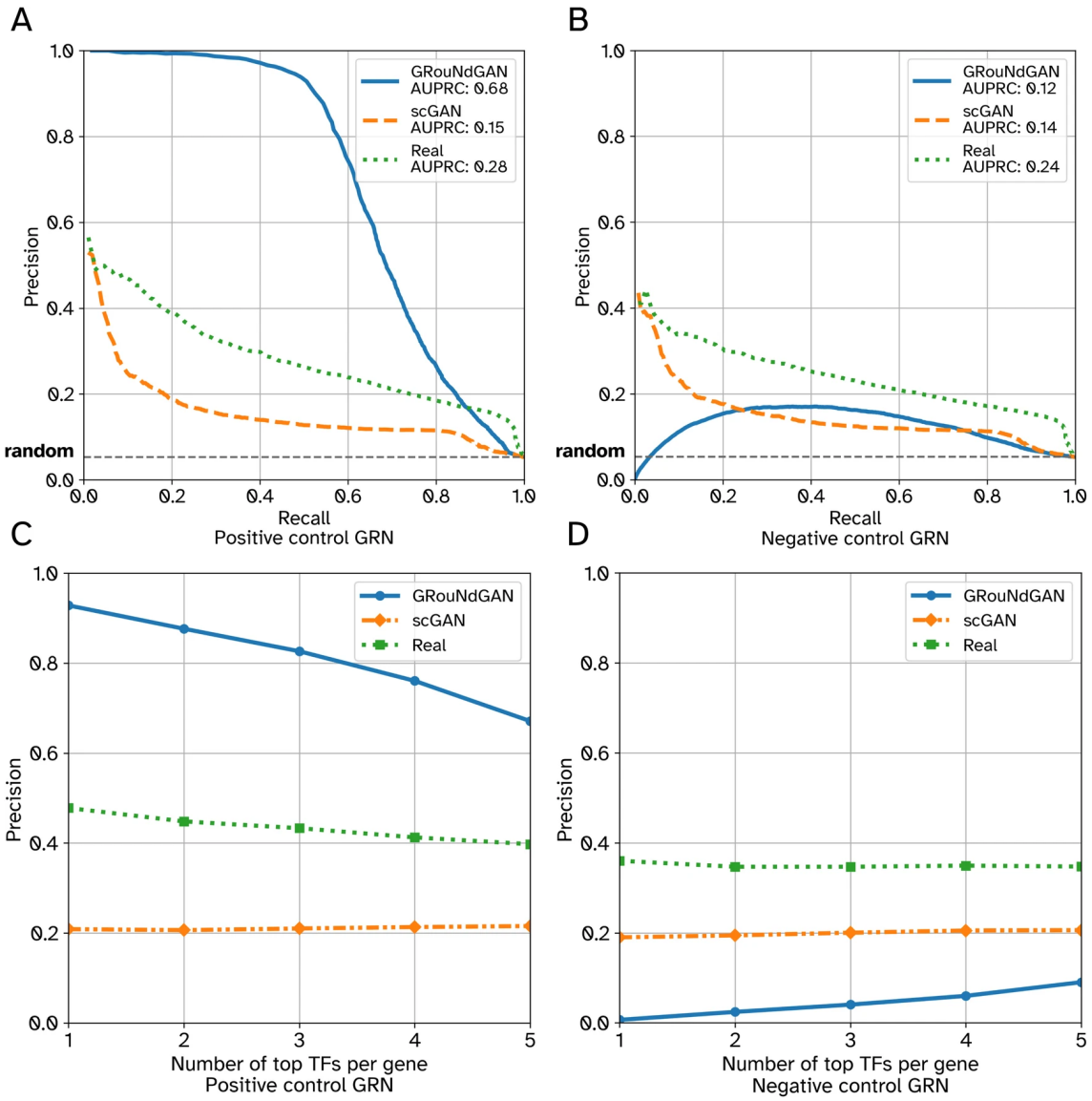


Figure 3: Performance of GRNBoost2 in recovering the imposed edges versus unimposed edges (PBMC-CTL). Left column shows the AUPRC (**A**) and Precision at k (per gene) (**C**) when the imposed edges (positive control GRN) were considered the ground truth. Right column shows the AUPRC (**B**) and Precision at k (per gene) (**D**) when the unimposed edges (negative control GRN) were considered the ground truth. Precision at k (per gene) refers to the precision when top k TFs for each gene is used to form the reconstructed GRN. All three datasets had the same number of cells ($n = 19{,}773$). Source data are provided as a Source Data file.

by GRouNdGAN are disrupted during simulation. The last property is particularly desirable for benchmarking GRN inference methods to ensure that the regulatory relationships present in the reference dataset (but unimposed) do not bias the simulated data, which could result in an inflated false positive rate.

Next, we repeated the analyses above on the PBMC-CTL dataset, but this time with input GRNs learned by PPCOR (a GRN inference based on partial correlation) from the training set. Similar to the results above, GRNBoost2 could reconstruct the imposed edges, and the importance

of the imposed edges were accentuated by GRouNdGAN (compared to the original training dataset) (Supplementary Data 3 – Sheet 8). We also imposed two randomly generated GRNs (with similar density to the GRNs above in which each gene was regulated by 5 TFs). Once again, the imposed edges were accentuated by GRouNdGAN, however imposing a random GRN (whose induced patterns may be inconsistent with real reference data) came at the cost of lower resemblance between simulated and reference data. (See Supplementary Notes and the Discussion for more details on why imposing a GRN that is inconsistent with the reference data (e.g., a random GRN) is a contradictory requirement to generating simulated data resembling the reference data).

GRouNdGAN-simulated data preserves trajectories and can be used for pseudo-time inference

In addition to deciphering discrete states, scRNA-seq data is often used to determine continuous cell transitions during biological processes such as differentiation, using trajectory and pseudo-time inference[38,39,40,41,42,43,44]. We set to determine whether GRouNdGAN-simulated data conforms to the transitional states and pseudo-time of the reference data. This is particularly important for benchmarking GRN inference methods that utilize pseudo-time information[12,13,14]. For this purpose, we used the BoneMarrow dataset corresponding to differentiation of hematopoietic stem cells to different lineages and simulated $n = 2230$ cells using GRouNdGAN (same as the number of real cells in the reference training dataset, to remain consistent).

We used Partition-based graph abstraction (PAGA)[38] for trajectory inference of the reference and GRouNdGAN-simulated data. PAGA computes graph-like maps of data manifolds faithful to the topology of data, retaining its continuous and discrete structures. Supplementary Fig. 10 shows the PAGA graphs, where nodes capture discrete states and the edges signify transitions between them. To compare the identified trajectories, we used markers of the cell types present in this dataset (Supplementary Table 6)[38]. Figure 4 shows the expression patterns of two marker genes for erythroid cells (*Gata1*, *Klf1*), neutrophils (*Mpo*, *Ctsg*), and basophils (*Mcpt8*, *Prss34*) for PAGA trajectory graphs of both datasets (Supplementary Figs. 11–17 show marker genes of all cell types). The comparison showed similar activation patterns between corresponding regions of PAGA graphs of the experimental and GRouNdGAN data, revealing several trajectories (e.g., erythroid and neutrophil branches). Strikingly, even the number of nodes activated in each figure scaled similarly between the two datasets: e.g., the markers of basophils were consistently expressed in only one node in each dataset (Fig. 4E, F).

We annotated different cell types present in both datasets using these marker genes (following PAGA's official tutorial) (Fig. 5A, B). The GRouNdGAN's annotated graph showed similar

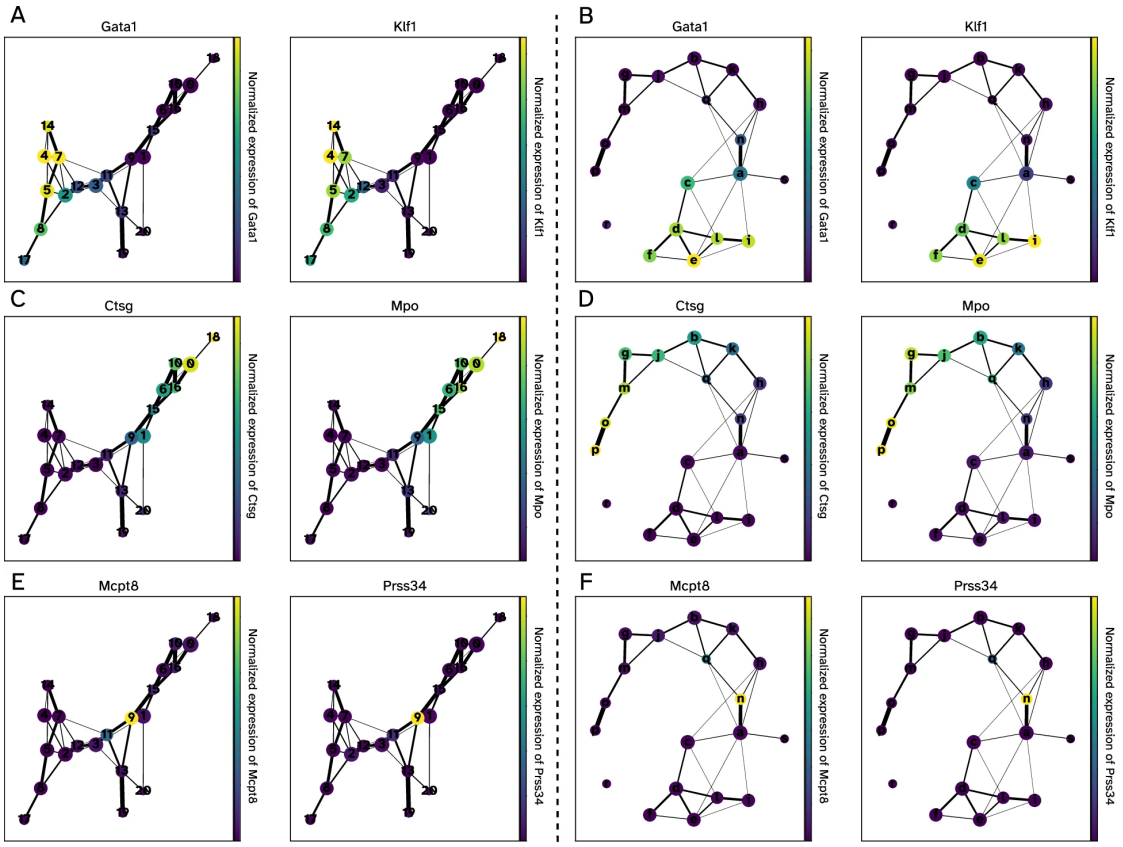


Figure 4: PAGA graphs and cell type marker gene activation patterns in experimental and simulated BoneMarrow dataset. In each figure, nodes represent Louvain clusters (capturing discrete states), edges represent transitions between states, and edge weights indicate confidence in the presence of connections. Low-connectivity edges below a threshold of 0.01 were discarded. Left panels (**A**, **C**, **E**) show the PAGA graph computed on GRouNdGAN-generated cells, and right panels (**B**, **D**, **F**) show the PAGA graph for the original BoneMarrow dataset. Node colors represent normalized expression of marker genes for erythroid cells (**A**, **B**), neutrophils (**C**, **D**), and basophils (**E**, **F**). Source data are provided as a Source Data file.

topological features to that of the reference data and captured known biological properties of hematopoiesis, which were also noted by Wolf et al.[38] using the reference dataset (e.g., proximity between monocytes and neutrophils or association between megakaryocyte and erythroid progenitors). These observations show that the key characteristics of the reference data are retained in GRouNdGAN-simulated data.

We used the diffusion pseudotime algorithm[39] (in scanpy[45]) to infer the progression of cells through geodesic distance (Fig. 5C, D). We noticed strong concordance when examining pseudotime values assigned to cell subgroups in the reference and GRouNdGAN-generated data. To track these patterns, we focused on marker expression changes along erythroid, neutrophil, and monocyte trajectories (Supplementary Figs. 11–13), similar to the analysis performed by Wolf et al.[38]. In both datasets, the activation of neutrophils' marker, *Elane*, and monocytes' marker, *Irf8*, were predominantly observed towards the later stages of their trajectories (Fig.

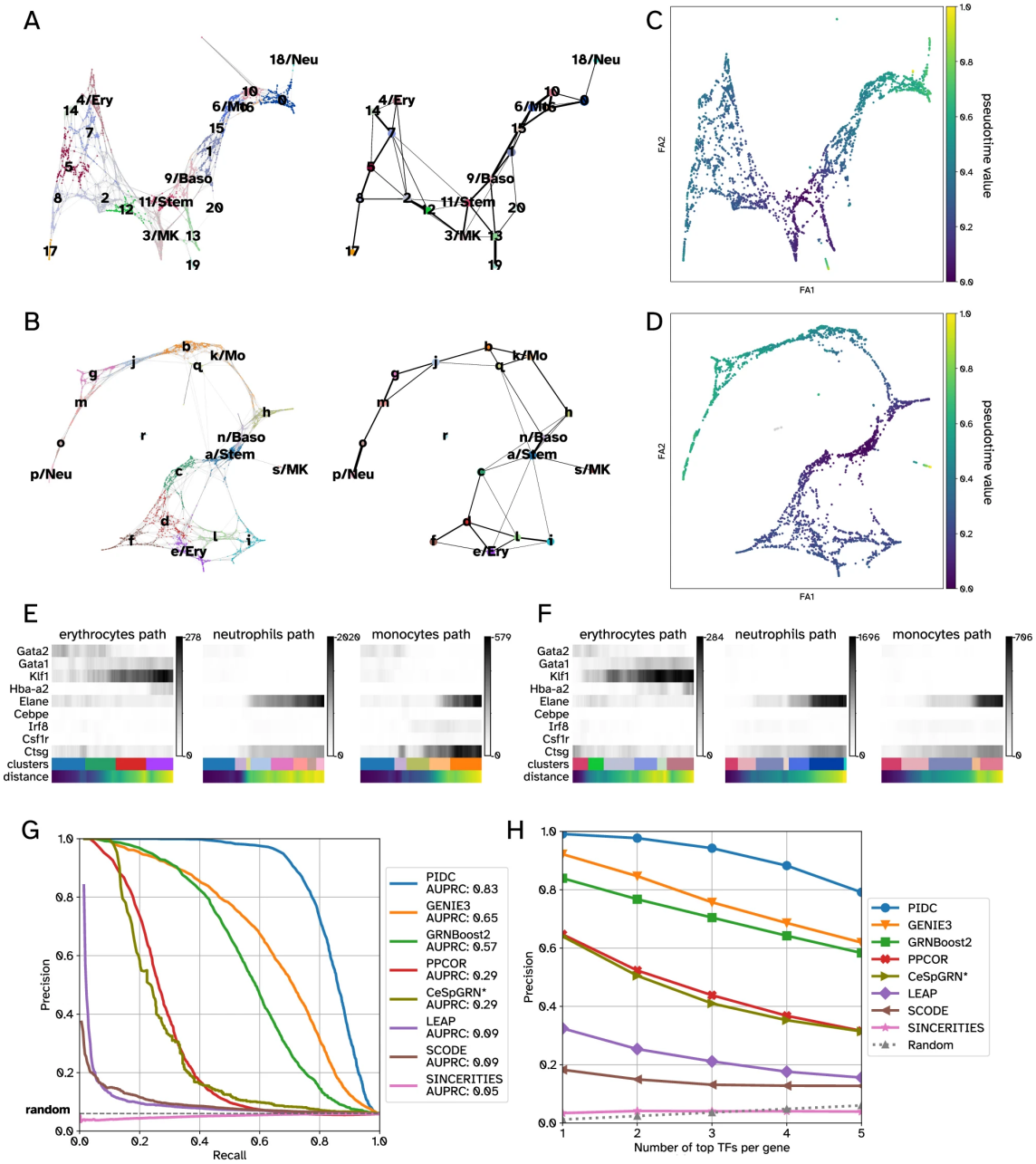


Figure 5: Consistency in trajectories and pseudo-temporal orderings between data generated by GRouNdGAN and the experimental BoneMarrow hematopoietic dataset. Panels **A** and **B** show the cell type annotations of simulated and experimental data, respectively. The figures on the left show the force-directed graphs (obtained using ForceAtlas2[66]) and the figures on the right show the PAGA graphs. Panels **C** and **D** show the PAGA-initialized force-directed graphs with cells colored by their inferred pseudo-time value. The following abbreviations are used: Stem for stem cells, Ery for erythroid cells, Neu for neutrophils, Mo for monocytes, MK for Megakaryocytes, and Baso for Basophils. Panels **E** and **F** display expression changes in the marker genes of erythrocyte (*Gata2*, *Gata1*, *Klf1*, and *Hba-a2*), neutrophil (*Elane* and *Cebpe*), and monocyte (*Irf8*, *Csf1r*, and *Ctsg*) branches for experimental (**E**) and simulated data (**F**). The distance refers to the geodesic distance from a root cell (here a stem cell). A darker shade of gray shows higher expression. Panels **G** and **H** show the performance of different GRN inference methods on data generated by GRouNdGAN using the BoneMarrow dataset. The benchmarking results are based on a dataset with $n = 2230$ simulated cells (except for CeSpGRN). Source data are provided as a Source Data file. * Due to the high memory requirement of CeSpGRN, we were only able to run it on a subset of data containing $n = 1000$ cells and $m = 100$ genes (the set of regulating TFs and the edges connecting them to these genes remained unchanged).

Table 3: Performance of GRouNdGAN in generating realistic scRNA-seq data using the Dahlin, Tumor-All, and Tumor-malignant datasets

	Dahlin		Tumor-All		Tumor-malignant	
	Training	**Testing**	**Training**	**Testing**	**Training**	**Testing**
Cosine distance	0.00011	0.00012	0.00141	0.00061	0.00059	0.00098
Euclidean distance	51	53	227	145	134	174
MMD	0.014	0.026	0.020	0.021	0.014	0.015
RF AUROC	0.52	0.54	0.53	0.52	0.53	0.57
miLISI	1.91	1.91	1.90	1.89	1.90	1.89

Each gene in the imposed GRN of GRouNdGAN is regulated by 15 TFs (constructed using GRNBoost2 from the experimental training set). For the first three metrics, a value closer to zero is preferred, for RF AUROC a value closer to 0.5 is preferred, and for miLISI a value closer to 2 is preferred. The metrics are calculated between a simulated dataset of 1000 cells and a set of 1000 real cells (all cells in the test set and 1000 randomly selected cells from the training set). For the first two metrics, the values correspond to the distance of the mean centroids of the real and simulated cells.

5E, F). Additionally, we observed the activation of erythroid maturity marker genes *Gata2*, *Gata1*, *Klf1*, and *Hba-a2* roughly in sequential order along the erythroid trajectory in both datasets (Fig. 5E, F), concordant with previous findings[38]. The striking similarity between the activation patterns of these markers in both datasets and the preservation of their ordering highlights GRouNdGAN's ability to capture dynamic transcriptional properties of scRNA-seq data, leading to correct trajectory inference and pseudo-time ordering.

GRouNdGAN achieves high performance on other datasets

In addition to the datasets discussed earlier, we applied GRouNdGAN to three other datasets. One corresponded to cells undergoing hematopoiesis (similar to the BoneMarrow dataset), but with much larger number of cells ($n = 44{,}802$)[46] (which we called Dahlin dataset). The second dataset corresponded to malignant cells and cells present in the tumor microenvironment of 20 fresh biopsies from follicular lymphoma tumors ($n = 136{,}147$), which we called the Tumor-All dataset[47]. We also formed a dataset containing only the malignant cells present in this dataset ($n = 89{,}203$ cells), which we called the Tumor-malignant dataset. Table 3 and Supplementary Data 1 - Sheet 2 show the performance of GRouNdGAN in generating realistic scRNA-seq data, while Supplementary Figs. 18–20 show the tSNE plots of simulated and real cells. For all three datasets, different metrics show that the generated data has a high degree of similarity to experimental data and there is a small difference between the training and test set performances.

We next repeated the trajectory inference, pseudo-time ordering, and analysis of cell type marker expression along the pseudo-time axis on the Dahlin dataset using both GRouNdGAN-simulated data ($n = 43{,}802$ cells) and real cells ($n = 43{,}802$), as we had done using the BoneMarrow dataset

(Supplementary Fig. 21). Similar to the results from the BoneMarrow dataset, we observed a high degree of concordance between the activation patterns of cell type markers in the simulated and real datasets and the preservation of their orderings. Taken together, these results suggest that GRouNdGAN can be used to generate realistic data from different biological contexts, while imposing a causal GRN and preserving various biological properties.

Benchmarking GRN inference methods using GRouNdGAN confirms prior insights from curated and experimental datasets

The BEELINE study[19] found conflicting results when benchmarking GRN inference methods on curated benchmarks and synthetic ones: top two performing models on synthetic data where among the bottom three on curated benchmarks. We re-investigated this using GRouNdGAN-simulated data with the BoneMarrow dataset and the positive and negative control GRNs (described earlier). We used seven GRN inference methods used by BEELINE, GRNBoost2[8], GENIE3[9], PIDC[10], PPCOR[37], LEAP[12], SCODE[13], and SINCERITIES[14], capturing a wide range of methods with a broad spread of performances reported on synthetic and curated benchmarks in the BEELINE study. We also included CeSpGRN[48], which is a cell-specific GRN inference method. We observed a striking resemblance between the performance pattern (order) of the methods on GRouNdGAN-simulated data and curated benchmarks from BEELINE (Fig. 5G, H and Supplementary Data 3 – Sheet 3 versus Fig. 4 of BEELINE[19]): the only difference was the swapping of LEAP and SCODE's order. We repeated this analysis using the PBMC-CTL dataset (Supplementary Data 3 – Sheet 2), obtaining similar patterns. We also tested these methods using the PBMC-All dataset (Supplementary Data 3 – Sheet 4). While the results were generally consistent, the order of GENIE3 and GRNBoost2 were swapped, which may be due to their ability in working with datasets containing multiple distinct cell types (also see Discussion).

Overall, PIDC[10] outperformed all methods using both datasets, followed by GENIE[39] and GRNBoost[28]. LEAP[12], SCODE[13] and SINCERITIES[14] (which use pseudo-time and could only be applied to the BoneMarrow dataset) performed worse than others, matching their behavior in the BEELINE study on curated benchmarks (but not on synthetic benchmarks). All methods performed close to random on unimposed negative control GRN edges. These results not only show that GRouNdGAN can be used for benchmarking GRN inference methods, but also shows that the insights obtained from it matches those obtained from curated and experimental benchmarks (such as those used in Figs. 4–6 of the BEELINE study), the formation of which requires extensive amount of resources and effort.

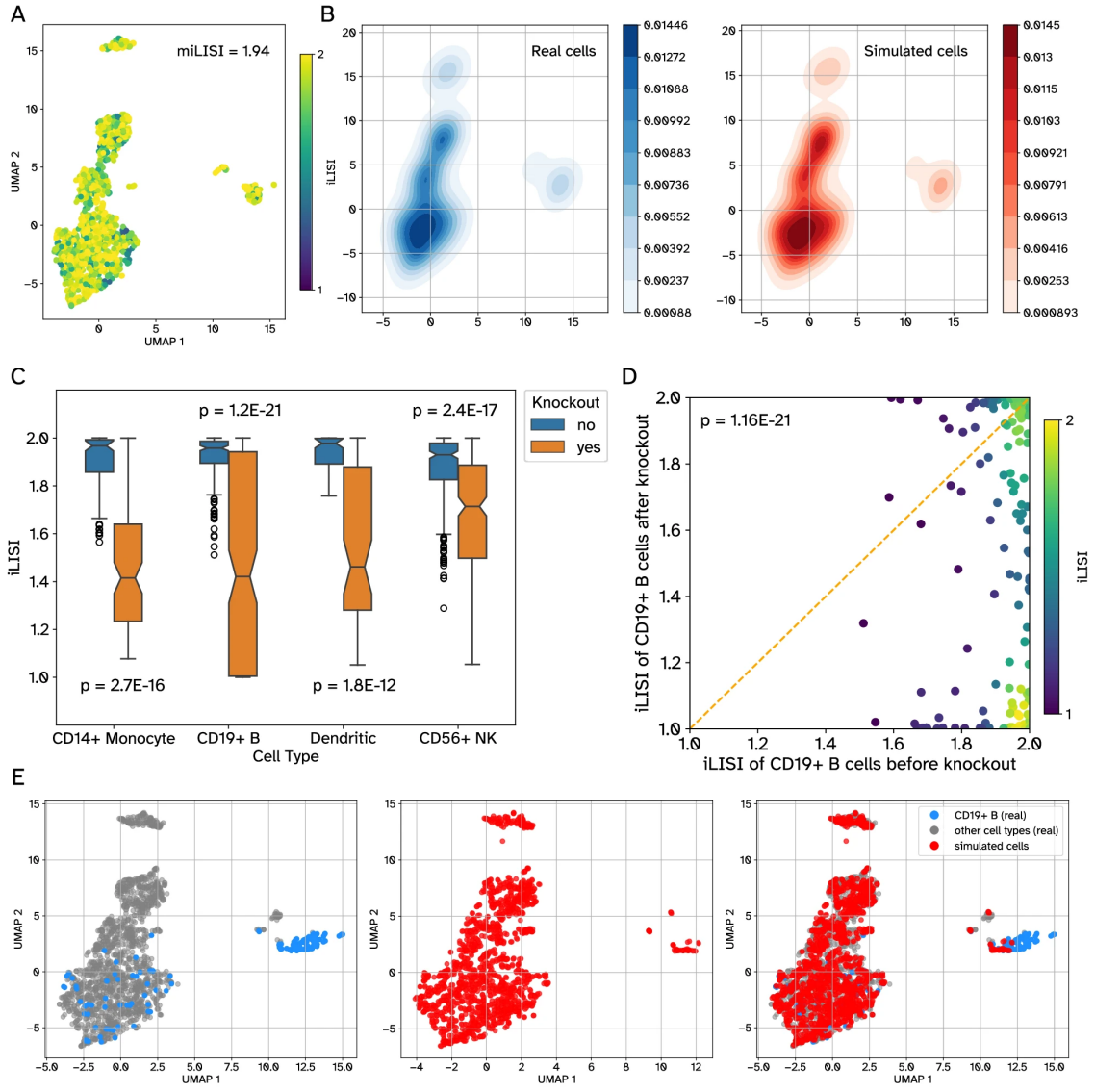


Figure 6: TF knockout experiments and their effect on cell type generation. **A** The UMAP shows the distribution of $n = 2000$ randomly selected cells from the experimental PBMC-All dataset. The color shows the iLISI value of each cell (miLISI = 1.94) calculated from a UMAP embedding, jointly obtained from the experimental cells and the same number of GRouNdGAN-generated cells. TFs were omitted as features when generating UMAP plots. In the simulated data, each gene was regulated by 15 TFs, identified using GRNBoost2. **B** Each plot shows the density of cells in the embedding space: the left plot (blue) corresponds to the real cells and the right plot (red) shows the simulated cells. **C** The boxplots show the distribution of iLISI values of $n = 2000$ real cells, calculated along with $n = 2000$ unperturbed simulated cells (blue) and with $n = 2000$ perturbed simulated cells (orange). For each cell type, top three most differentially expressed TFs were knocked out. The p-values reported in this figure were calculated using one-sided Wilcoxon signed rank tests. The center horizontal line corresponds to the median. Each box spans the interquartile range, and its limits show the upper (Q3) and lower quartiles (Q1). Whiskers extending from the boxes reach 1.5 times above and below the interquartile range. Outliers are shown individually if they fall outside the whiskers. **D** The scatter plot shows the iLISI values of CD19+B cells calculated along with unperturbed simulated cells (x-axis) and along with perturbed simulated cells (y-axis). The circles correspond to real cells and their colors reflect the density of datapoints in that region. The p-value reported in this figure was calculated using a one-sided Wilcoxon signed rank test. **E** The UMAP embedding of real cells (left), simulated cells after knockout of top TFs of CD19+B cells (middle), and all together (right). Source data are provided as a Source Data file.

In silico perturbation experiments using GRouNdGAN

In many studies, one is interested to characterize the relationship between TFs' expression and phenotypic labels (e.g., cell types). Differential expression (DE) analysis is a common approach to identify TFs (or other genes) most associated with a specific cell type. However, to directly test whether perturbing the expression of such candidate TFs change the expression of genes representative of a cell type, perturbation experiments (e.g., TF knockout) are desired. As a causal implicit generative model, GRouNdGAN provides the capability to sample from interventional distributions, thereby enabling in silico perturbation experiments. This functionality is provided at inference time by enabling manipulation of TFs' expressions as the outputs of the causal controller. This change is propagated along the GRN (through the model architecture), impacting the expression of genes regulated by the perturbed TF(s). By enabling a deterministic mode of operation, GRouNdGAN allows for exact comparison of gene expressions before and after perturbation while maintaining the invariance of other parameters (Methods).

To test this functionality, we asked whether the knockout of top 3 TFs most differentially expressed (Mann Whitney U test) between each cell type and the rest (identified from the reference dataset) would result in less cells being generated in the vicinity of the cells of that type. If positive, this implies that the perturbation in the TFs' expression has modified the gene expression profile of generated cells so that they no longer resemble that particular cell type (confirming our expectation). Figure 6A shows the UMAP embedding of the reference dataset (miLISI = 1.94 when compared with simulated data) and Fig. 6B shows the density plots of the experimental and simulated data, revealing a high degree of resemblance. We focused on the changes in the iLISI values of experimentally profiled cells of a specific type (annotated in the original study), when jointly mapped to the same embedding space with simulated data, before and after knockout. For the changes in the iLISI values to be observable, it was necessary to focus on cell types that occupy relatively distinct regions of the embedding space: CD14+ monocytes, CD19+ B cells, Dendritic cells, CD56+ natural killer cells (Supplementary Fig. 22); otherwise, the presence of other cell types in that region would confound the results.

Figure 6C shows the iLISI distributions for each cell type (only for real cells), calculated along with unperturbed simulated cells (blue) and perturbed simulated cells (orange). Figure 6D, E show the iLISI values and the distribution of generated cells after the knockout experiment for CD19+ B cells as an example (see Supplementary Figs. 23–26 for other cell types). In all cases, the iLISI values significantly reduced after the knockout experiment (Fig. 6C, Supplementary Table 7), showcasing that after the knockout of top TFs associated with a cell type, GRouNdGAN generated much fewer cells resembling that cell type (even though it did not have the knowledge of the cell type annotations). Instead, the simulated cells were dispersed into other cell types, yet they retained meaningful positions within the overall dataset embedding space. Also, the

miLISI of the other cell types remained relatively unchanged (Supplementary Table 7).

Next, we repeated the analyses above using Tumor-All dataset that contained different cell types including malignant cells, but also other cell types present in the tumor microenvironment. We considered cell types that made up of at least 0.25% of the dataset (Supplementary Table 8). When cells of two or more types populated the same regions of the embedding space, we kept the cell type containing more cells. Malignant cells, T cells, B cells, and plasma cells satisfied the conditions above, which we included in our analyses. Similar to the previous analyses, we observed that the KO of top 3 TFs of each cell type results in a significant reduction in the iLISI values (Supplementary Fig. 27, Supplementary Table 8).

These analyses suggest that in silico TF perturbation experiments using GRouNdGAN produces results concordant with results directly obtained from real biological data, a direction that we will explore further in the future studies.

Discussion

GRouNdGAN is a causal implicit generative model designed to simulate realistic scRNA-seq datasets based on a user-defined GRN and a reference dataset. By incorporating GRN connections in its architecture and including auxiliary tasks, it imposes causal TF-gene relationships in the gene expression profile of simulated cells. These causal relationships are verifiable by in silico knockout experiments and identifiable by GRN inference methods. (In addition to the experimental analyses performed in this study, an interested reader should refer to Kocaoglu et al.[31] for theoretical evidence of causality of data generated by this architecture.)

We demonstrated that GRouNdGAN achieves state-of-the-art performance on various tasks including realistic scRNA-seq data generation, GRN inference benchmarking, and in silico knockout experiments. Even when applied to heterogenous datasets, it generated realistic datasets without utilizing information regarding cell clusters or cell types, achieving best or second-best performance compared to models that used this information. We should note that it is trivially possible to use cell type/cluster information with GRouNdGAN: one can simply provide subsets of the experimental dataset corresponding to a cell type/cluster as a reference, one at a time, with a cell-type specific (or shared) GRN (similar to our analyses using PBMC-CTL and PBMC-NaiveT in Supplementary Data 2 – Sheet 2). However, the fact that this model does not require this information broadens its applicability. We showed that GRouNdGAN-generated data preserves complex cellular dynamics and patterns of the reference dataset such as lineage trajectories, pseudo-time orderings, and patterns of cell type markers' activation. One consideration in generating realistic and generalizable cells is the number of training examples. In the BoneMarrow dataset (which had only 2.7k cells), the performance of all simulators

dropped on the test set compared to the training set. When we generated cells using another similar hematopoiesis dataset (Dahlin), but with much larger number of cells (45k), the testing and training performance difference significantly reduced (Table 3). Similarly, the training and testing performance on all the other datasets that had more than ten thousand cells remained on par with each other, suggesting that a large number of cells improves the realism of the simulated data (which is expected). In spite of this, the cells generated by GRouNdGAN could capture trajectories and pseudo-time orderings not only in the larger Dahlin dataset, but also in the smaller BoneMarrow dataset.

GRouNdGAN imposes user-defined causal regulatory interactions, while excluding those present in the reference dataset, but not in the input GRN. This makes GRouNdGAN-simulated datasets ideal for benchmarking GRN inference algorithms with the input GRN as the ground truth. We benchmarked eight GRN inference algorithms on the simulated BoneMarrow dataset and showed that our results coincide with results reported by BEELINE[19] on curated and experimental benchmarks. This highlights GRouNdGAN's ability to bridge the gap between experimental and simulated datasets for GRN benchmarking without requiring excessive user manipulation. We assessed five of these methods (that did not require pseudo-time) on the PBMC-CTL dataset and observed that their ordering was consistent with the simulated BoneMarrow data. We then applied them to the PBMC-All dataset, containing multiple distinct cell types. While the general trend was consistent with the other benchmarks, the order of GENIE3 and GRNBoost2 switched. This could be related to the ability of these methods on handling distinct cell types. One should also note that the results of benchmarking on PBMC-All is not conclusive, and follow-up studies are required to fully characterize the effect of distinct cell types on the performance of GRN inference methods. In particular, GRNs are cell type- and context-specific, and such a study should impose different GRNs on different subpopulations of the dataset. Due to the flexibility provided by GRouNdGAN, one can evaluate a variety of GRN inference methods including supervised methods[49,50] or cell-specific GRN inference methods[48,51]. We included CeSpGRN[48] in our benchmarks, but a comprehensive study that evaluates this and other methods of this class from different perspectives is of great interest.

Since GRouNdGAN generates data based on a structural causal model, it can be used to sample synthetic cells from both observational and interventional distributions. Using this property, we conducted in silico perturbation experiments by knocking out key regulating TFs in various cell types and observed that the targeted cell-states were suppressed in the generated dataset. These results, while preliminary, show potential for developing a method based on GRouNdGAN to predict the results of TF knockout experiments. However, such method would inevitably need access to a high quality GRN underlying the real data, requiring additional data modalities such as (sc)ATAC-seq or TF ChIP-seq.

GRouNdGAN maintains key advantages over existing simulators that incorporate GRNs such

as SERGIO and BoolODE. BoolODE, unlike GRouNdGAN, is a reference-free (de novo) and rule-based simulator that relies on user-defined parameters instead of a reference dataset to artificially introduce properties of single-cell data (e.g., dropout, batch effect, existence of distinct populations, and variation among experimental samples and conditions). As a rule-based simulator, it allows for flexible selection of simulation parameters such as choice of noise strength, dropout rate, or kinetic parameters (for mRNA transcription, protein translation and mRNA and protein degradation rates). However, extensive user knowledge is required to leverage this flexibility to avoid generating unrealistic datasets and recent studies have argued that one should use discretion when interpreting results obtained from such de novo simulations[52]. This may explain why some of GRN inference methods that performed best on BoolODE's synthetic benchmarks, were among the worst on the experimental benchmarks of BEELINE. Moreover, as a reference-free simulator, BoolODE is not designed to match a specific reference dataset. As a result, it does not lend itself to benchmarking GRN inference methods in distinct contexts characterized by experimental reference datasets.

On the other hand, SERGIO is a hybrid method that contains elements of reference-free simulators, but also enables matching the simulated cells to a specific reference dataset by adding technical noise. SERGIO simulates data by first generating a "clean" dataset based on SDEs. Then, the user would iteratively fine-tune three noise parameters until the simulated and reference datasets are matched. However, this data-matching procedure puts extra burden on the user and might lead the user to overly rely on fine-tuning technical noise parameters, instead of SDE parameters. Our experiments with SERGIO (described in Supplementary Notes) showed that in spite of our best efforts for distribution matching, there were clear differences between the SERGIO-simulated data and the reference dataset. Another consideration is that over relying on added technical noise to achieve data-matching can potentially corrupt the data, rendering the underlying regulatory patterns unobservable in the final simulated dataset. This might explain the near-random GRN inference performance of various methods reported by SERGIO on their noisy datasets[4].

In contrast to BoolODE and SERIGO, during training, GRouNdGAN's parameters are optimized to minimize any mismatch between real and simulated data distributions automatically. This enables it to replicate scRNA-seq data characteristics (e.g., technical and biological noise) without any user intervention or post-processing. Moreover, GRouNdGAN is trained such that each of its generated genes or TFs mimics the expression patterns of a corresponding gene or TF in the reference dataset, preserving gene/TF identities. In contrast, BoolODE and SERGIO simulate pseudo-genes whose identities cannot be readily mapped to their counterparts in the reference dataset. Gene identity preservation can be attained in these simulators by fine-tuning the SDE parameters until the expression patterns of a given generated gene aligns with a corresponding gene in the reference dataset. This, however, places further burden on the user. Finally,

GRouNdGAN learns complex non-linear co-regulatory patterns from the data, instead of relying on simplifying assumptions or user input. In contrast, SERGIO assumes that the combined effect of multiple regulatory TFs is simply the sum of their individual effects. Similarly, BoolODE requires the user to provide a truth table specifying combinatory TF co-regulation rules for each target gene. In contrast, GRouNdGAN's neural network-based generator enables it to implicitly learn complex non-linear co-regulatory patterns that may represent co-expression, co-inhibition, or co-activation of individual genes. This is achieved during the training of GRouNdGAN without user intervention.

Several key points need to be considered when using GRouNdGAN. First, our analyses showed that the architectural and hyperparameter choices of the presented model result in good performance across different datasets. We performed hyperparameter tuning on one dataset and used the same set of values for all datasets and for different choices of GRNs, obtaining state-of-the-art performance. However, when trained on a completely new dataset, or datasets containing significantly different number of genes and TFs, it is advisable to try different choices of the hyperparameters using a validation set. Second, the architecture of the model includes LSN layers and, as such, the model generates expression values library-size normalized across the considered genes and TFs. Our ablation study (Supplementary Notes) and our experiments showed that the LSN layer improves the GRN imposition, the realism of the generated samples, and stabilizes the training (an observation also made by the scGAN study). In the future iterations of the model, we are planning to remove this restriction to enable generating unnormalized count data. Third, an important consideration for any causal inference problem is that given an observational dataset, its underlying causal graph is not necessarily unique. When benchmarking GRN inference methods, this problem exists regardless of how the ground truth GRN is obtained (even using knockout/knockdown experiments), and a GRN inference algorithm may find a plausible causal GRN, yet not the one that generated the data. Despite this, our analysis using the positive/negative control GRNs showed that GRN inference methods could detect the imposed edges while ignoring others that were unimposed. Based on this observation, we posit that the imposed GRN by GRouNdGAN, while not the only causal GRN capable of generating the simulated dataset, is the most probable one. However, verifying this requires theoretical analyses that are well beyond the scope of this study. Fourth, GRouNdGAN is a generative model and is not a predictive model. As such, its goal is to simulate cells similar to a reference training set (while imposing a user-defined GRN), and not to predict the scRNA-seq profile of cells in a new context or an independent dataset. Given a new dataset, the model needs to be retrained to generate realistic samples.

Finally, although GRouNdGAN can generate data with any GRN, the resemblance of the data to the reference dataset deteriorates if the imposed TF-gene relationships are significantly different from those consistent with the patterns in the reference dataset. The reason is that in such a

scenario, generating simulated datapoints resembling the reference training dataset and imposing the GRN act as contradictory requirements. This is a fundamental problem that is not unique to GRouNdGAN and is applicable to any simulator with these two requirements. However, simulators that do not try to impose a causal graph (e.g., scGAN) can focus on generating realistic scRNA-seq samples, even at the expense of disrupting some of the interactions present in the reference dataset (as we observed in Fig. 3 with scGAN). As a result, for GRN inference method benchmarking, we recommend first finding a set of regulatory edges from the reference dataset (e.g., using GRNBoost2 or even other data modalities) and imposing a selected subset of those in the simulated data. This ensures that the imposed edges are encoded in the simulated data, the unimposed (or undetected) edges are disrupted (as shown in our analyses), and the simulated dataset resembles the reference dataset. It is important to note that the regulatory edges inferred from the reference dataset do not need to be causal (and most likely, some are not); some of them could represent spurious correlations that are observable in the data. In fact, most GRN inference methods do not identify causal edges, yet can be used to form the input GRN. When a subset of the (potentially) non-causal edges of the reference dataset are imposed by GRouNdGAN, they are imposed in a *causal* manner, representing the causal graph underlying the *simulated* data (and not the reference data).

The current version of GRouNdGAN only allows imposing a bipartite GRN and does not support a multi-layer GRN, an assumption that we plan to relax with the future versions of the model. Moreover, we intend to augment the model's architecture in the future to enable imposing a causal graph with three layers of nodes capturing TF-gene-phenotype relationships. This is particularly useful for development and evaluation of phenotype-relevant GRNs, a concept that we have introduced in the past[11]. Additionally, a conditional version of GRouNdGAN can be developed, allowing the user to generate cells of specific cell type or on a specific stage along a lineage trajectory. Finally, by incorporating DAG structural learning[53], we intend to enable GRouNdGAN to infer a GRN while simultaneously learning to mimic the reference dataset.

Methods

Datasets and preprocessing

We downloaded the human peripheral blood mononuclear cell (PBMC Donor A) dataset containing the single-cell gene expression profiles of 68579 PBMCs represented by UMI counts from the 10x Genomics (`https://support.10xgenomics.com/single-cell-gene-expression/datasets/1.1.0/fresh_68k_pbmc_donor_a`). This dataset contains a large number of well-annotated cells and has been used by other models to generate synthetic scRNA-seq

samples[24]. We also downloaded the scRNA-seq profile of 2730 cells corresponding to differentiation of hematopoietic stem cells to different lineages from mouse bone marrow[34] ("BoneMarrow" dataset) from Gene Expression Omnibus (GEO) (accession number: GSE72857). We also obtained another haematopoietic dataset corresponding to the scRNA-seq (10x Genomics) profiles of 44,802 mouse bone marrow hematopoietic stem and progenitor cells (HSPCs) differentiating towards different lineages from GEO (accession number: GSE107727)[46] (called Dahlin dataset here). Finally, we obtained the batch-corrected scRNA-seq (10x Genomics) profile of 136,147 cells corresponding to malignant cells as well as cells in the tumor microenvironment (called Tumor-All dataset here) from 20 fresh core needle biopsies of follicular lymphoma patients from (https://cellxgene.cziscience.com/collections/968834a0-1895-40df-8720-666029b3bbac)[47].

We followed pre-processing steps similar to those of scGAN and cscGAN[24] using scanpy version 1.8.2[45]. In each dataset, cells with nonzero counts in less than ten genes were removed. Similarly, genes that only had nonzero counts in less than three cells were discarded. Top 1000 highly variable genes were selected using the dispersion-based method described by Satija et al.[54]. Finally, library-size normalization was performed on the counts per cell with a library size equal to 20,000 in order to be consistent with previous studies[24]. See Supplementary Data 1 - Sheet 3 for the number of cells and highly variable genes and TFs present in each final dataset.

GRouNdGAN's model architecture

GRouNdGAN's architecture consists of 5 components, each implemented using separately parameterized neural networks: a causal controller, target generators, a critic, a labeler, and an anti-labeler (Fig. 1C).

Causal controller

The role of causal controller is to generate the expression of TFs that causally control the expression of their target genes based on a user-defined gene regulatory network (Fig. 1C). To achieve this, it is first pre-trained (Fig. 1B) as the generator of a Wasserstein GAN with gradient penalty (WGAN-GP) (see Supplementary Notes for the formulation of the Wasserstein distance). GANs are a class of deep learning models that can learn to simulate non-parametric distributions[55]. They typically involve simultaneously training a generative model (called the "generator") that produces new samples from noise, and its adversary, a discriminative model that tries to distinguish between real and generated samples (called the "discriminator"). The generator's goal is to generate samples so realistic that the discriminator cannot determine whether it is real or simulated (an accuracy value close to 0.5). Through adversarial training, the generator and

discriminator receive feedback, allowing them to co-evolve in a symbiotic manner.

The main difference between a WGAN and a traditional GAN is that in the former, a Wasserstein distance is used to quantify the similarity between the probability distribution of the real data and the generator's produced data (instead of Kullback–Leibler or Jensen–Shannon divergences). Wasserstein distance has been shown to stabilize the training of WGAN without causing mode collapse[32]. The detailed formulation of the Wasserstein distance used as the loss function in this study is provided in Supplementary Notes. In addition, instead of a discriminator, WGAN uses a "critic" that estimates the Wasserstein distance between real and generated data distributions. In our model, we added a gradient penalty term for the critic (proposed by Gulrajani et al.[56] as an alternative to weight clipping used in the original WGAN) in order to overcome vanishing/exploding gradients and capacity underuse issues.

In the pretraining step, we trained a WGAN-GP with a generator (containing an input layer, three hidden layers, and an output layer), a library-size normalization (LSN) layer[24], and a critic (containing an input layer, three hidden layers, and an output node). A noise vector of length 128, with independent and identically distributed elements following a standard Gaussian distribution, was used as the input to the generator. The output of the generator was then fed into the LSN layer to generate the gene and TF expression values. The details of hyperparameters and architectural choices of this WGAN-GP are provided in Supplementary Table 1. Although we were only interested in generating expression of TFs using the generator of this WGAN-GP (in the second step of pipeline), the model was trained using all genes and TFs to properly enforce the library-size normalization. Once trained, we discarded the critic and the LSN layer, froze the weights of the generator and used it as the "causal controller"[31] to generate expression of TFs (Fig. 1C).

Target generators

The role of target generators is to generate the expression of genes causally regulated by TFs based on the topology of a GRN. Consider a target gene G_j regulated by a set of $TFs = \{TF_1, TF_2, ..., TF_n\}$. Under the causal sufficiency assumption and as a result of the manner by which TFs' expressions are generated from independent noise variables, we can write $E_{G_j} = f_{G_j}(E_{TF_1}, E_{TF_2}, ..., E_{TF_n}, N_{G_j})$ and $E_{TF_i} = f_{TF_i}(N_{TF_i})$ for $i = 1, 2, ..., n$, where E represents expression, N represents a noise variable, and f represents a function (to be approximated using neural networks). All noise variables are jointly independent. Following the theoretical and empirical results of CausalGAN[31], we can use feedforward neural networks to represent functions f by making the generator inherit its neural network connections from the causal GRN. To achieve this, we generate each gene in the GRN by a separate generator such that target gene generators do not share any neural connections (Fig. 1C).

As input, the generator of each gene accepts a vector formed by concatenating a noise variable and a vector of non-library size normalized TF expressions from the causal controller, corresponding to the TFs that regulate the gene in the imposed GRN (i.e., its parents in the graph). The expression values of TFs and the generated values of target genes are arranged into a vector, which is passed to an LSN layer for normalization. We used target generators with three hidden layers of equal width. The width of the hidden layers of a generator is dynamically set as twice the number of its regulators (the noise variable and the set of regulating TFs). If the imposed GRN is relatively dense and contains more than 5000 edges, we set the depth to 2 and the width multiplier to 1 to be able to train on a single GPU. The details of hyperparameters and architectural choices are provided in Supplementary Table 2.

In practice, generating each target gene's expression using separate neural networks introduces excess overhead. This is because in every forward pass, all target genes' expressions must be first generated before collectively being sent into the LSN layer. As a result, instead of parallelizing the generation of each target gene's expression (which due to the bottleneck above does not provide a significant computational benefit), we implemented target generators using a single large sparse network. This allows us to reduce the overhead and the training time to train the model on a single GPU, and to benefit from GPU's large matrix multiplication. We mask weights and gradients to follow the causal graph, while keeping the generation of genes independent from each other. From a logical standpoint, our implementation has the same architecture described earlier, but is significantly more computationally efficient. See Supplementary Notes for details.

Critic

Similar to a traditional WGAN, the objective of GRouNdGAN's critic (Fig. 1C) is to estimate the Wasserstein distance between real and generated data distributions. We used the same critic architecture as the WGAN-GP trained in the first stage.

Labeler and anti-labeler

Although the main role of the target generators is to produce realistic cells to confuse the critic, it is crucial that they rely on the TFs' expression (in addition to noise) in doing so. One potential risk is that the target generators disregard the expression of TFs and solely rely on the noise variables. This is particularly probable when the imposed GRN does not conform to the underlying gene expression programs of real cells in the reference dataset; in such a scenario, and to make realistically looking simulated cells, it is more convenient for a WGAN to simply ignore the strict constraints of the GRN and solely rely on noise.

To overcome this issue, we used auxiliary tasks and neural networks known as "labeler" and "anti-

labeler"[31]. The task of these two networks is to estimate the causal controller's TF expressions (here called labels) from the target genes' expressions alone, by minimizing the squared L2 norm between each element's TF estimates and their true value. More specifically, the corresponding loss for a batch of size N_{batch} is of the form $\frac{1}{N_{batch}} \sum_{i=1}^{N_{batch}} ||\hat{y}_i - y_i||_2^2$, where $\hat{y}_i$ is the estimated vector of TF expression values generated by the labeler or anti-labeler. For anti-labeler, y_i corresponds to the TF expression values outputted by the causal controller; for labeler, this vector can also correspond to TF expression values from the real training data. This resembles the idea behind an autoencoder and ensures that the model will not disregard the expression of TFs in generating the expression of their target genes. The anti-labeler is trained solely based on the outputs of the target generators, while the labeler utilizes both the outputs of target generators and the expression of real cells (Fig. 1C). They are both implemented as fully connected networks with a width of 2000 and a depth of 3 and optimized using the AMSGrad[57] algorithm. Each layer, except the last one, utilizes a ReLU activation function and batch normalization. In addition to WGAN-GP losses, we add labeler and anti-labeler losses to the generator to minimize both. This is different from the approach used in CausalGAN, where the anti-labeler's loss is maximized in the early training stages to ensure that the generator doesn't fall into label-conditioned mode collapse. In GRouNdGAN, the causal controller is pretrained and generates continuous labels (TFs expression) and does not face a similar issue. As a result, we instead minimized the loss of the anti-labeler from the beginning and as such the labeler and anti-labeler both act as auxiliary tasks to ensure the generated gene expression values take advantage of TFs expression.

Training procedure and hyperparameter tuning

We follow a two-step training procedure comprising of training two separate WGAN-GPs (Fig. 1) to train GRouNdGAN. The generators and critics in both GANs are implemented as fully connected neural networks with rectified linear unit (ReLU) activation functions in each layer, except for the last layer of the critic. The weights were initialized using He initialization[58] for layers containing ReLU activation and Xavier initialization[59] for other layers (containing linear activations). We used Batch Normalization[60] to normalize layer inputs for each training minibatch, except for the critic. This is since using it in the critic invalidates the gradient penalty's objective, as it penalizes the norm of the critic's gradient with respect to the entire batch rather than to inputs independently. An LSN layer[24] was used in both WGAN-GPs (Fig. 1B, C) to scale counts in each simulated cell to make it consistent with the library size of input reference dataset. This normalization results in a dramatic decrease in convergence time and smooths training by mitigating the inherent heterogeneity of scRNA-seq data.

We would like to point out that both steps of training (i.e., the pretraining of Fig. 1B and training of Fig. 1C) are performed on the exact same training set, and when data corresponding to a new

dataset is to be generated, these steps need to be repeated. The goal of the first step is to train the causal controller to learn the distribution of training set and generate realistic TF expression values (without imposing TF-gene relationships). The goal of the second step is to use the TF expression values generated by the trained causal controller to generate expression of target genes while imposing the TF-gene causal relationships.

GRouNdGAN solves a min-max game between the generator (f_g) and the critic (f_c), with the following objective function:

$$\min_{f_g} \max_{\|f_c\|_L \leq 1} \mathbb{E}_{x \sim \mathbb{P}_r} f_c(x) - \mathbb{E}_{x \sim f_g(\mathbb{P}_{\text{noise}})} f_c(x) \tag{1}$$

The training objective of the critic involves maximizing the difference between the average score assigned to real ($\mathbb{E}_{x \sim \mathbb{P}_r} f_c(x)$) and generated samples ($\mathbb{E}_{x \sim f_g(\mathbb{P}_{noise})} f_c(x)$) with respect to its parameters following Eq. (1). On the contrary, the generator attempts to minimize the average score that the critic assigns to real and generated samples. Through this adversarial game, both the critic and generator co-evolve and the generator learns a mapping f_g from a simple standard Gaussian noise distribution ($\mathbb{P}_{noise}$) to a distribution $f_g\left(\mathbb{P}_{noise}\right)$ that approximates the real data distribution $\mathbb{P}_r$. An important point to consider is that the critic does not directly compute the Wasserstein distance (mathematically defined in Supplementary Notes - Details regarding the Wasserstein distance). Instead, it is encouraged to provide a meaningful estimate of the Wasserstein distance between the distribution of real data ($\mathbb{P}_r$) and the distribution of generated data ($f_g\left(\mathbb{P}_{noise}\right)$) through its training process.

We alternated between minimizing the generator loss for one iteration and maximizing the critic loss for five iterations. We employed the AMSGrad[57] optimizer with the weight decay parameters $\beta_1 = 0.5, \beta_2 = 0.9$ and employed an exponentially decaying learning rate for the optimizer of both the critic and generator.

The hyperparameters were tuned using a validation set consisting of 1000 cells from the PBMC-CTL dataset (Supplementary Tables 1 and 2) based on the Euclidean distance and the RF AUROC score, which were consistently in accord. The same hyperparameters were used for all other analyses and datasets.

Causal GRN preparation

This section describes the creation of the causal graph inputted to GRouNdGAN used to impose a causal structure on the model. GRouNdGAN accepts a GRN in the form of a bipartite directed acyclic graph (DAG) as input, representing the relationship between TFs and their target genes. In this study, we created the causal graph using the 1000 most highly variable genes

and TFs identified in the preprocessing step (Supplementary Data 1 - Sheet3). First, the set of TFs among the highly variable genes were identified based on the AnimalTFDB3.0 database[61] and a GRN was inferred using GRNBoost2[8] (with the list of TFs provided) from the training reference dataset. It is important to note that the regulatory edges identified from the reference dataset using GRNBoost2 are not necessarily causal edges (and they do not need to be for the purpose of forming the input GRN), but they are consistent with the patterns of the data. However, when this (potentially non-causal) GRN is imposed by GRouNdGAN, it is imposed in a causal manner and represents the causal data generating graph of the simulated data (and not the reference data).

Evaluation of the resemblance of real and simulated cells

We evaluated all models using held-out test sets containing randomly selected cells from each reference dataset (500 cells from BoneMarrow and 1000 cells for all other datasets) (see Supplementary Data 1 - Sheet 3 for other statistics about the datasets). To quantify the similarity between real and generated cells, we employed various metrics. For each cell represented as a datapoint in a low dimensional embedding (e.g., t-SNE or UMAP), the integration local inverse Simpson's Index (iLISI)[36] captures the effective number of datatypes (real or simulated) to which datapoints of its local neighborhood belong based on weights from a Gaussian kernel-based distributions of neighborhoods. The miLISI is the mean of all these scores and in our study ranges between 1 (poor mixing of real and simulated cells) and 2 (perfect mixing of real and simulated cells). Additionally, we calculated the cosine and Euclidean distances of the centroids of real cells and simulated cells, where the centroid was obtained by calculating the mean along the gene axis (across all simulated or real cells).

To estimate the proximity of high-dimensional distributions of real and simulated cells without creating centroids, we used the maximum mean discrepancy (MMD)[35]. Given two probability distributions p and q and a set of independently and identically distributed (i.i.d.) samples from them, denoted by X and Y, MMD with respect to a function class $\mathcal{F}$ is defined as:

$$\mathrm{MMD}\left[\mathcal{F}, X, Y\right] := \sup_{f \in \mathcal{F}} \left(\mathbb{E}_x \left[f(x) \right] - \mathbb{E}_y \left[f(y) \right] \right), \tag{2}$$

where *sup* refers to supremum and $\mathbb{E}$ denotes expectation. When the MMD function class $\mathcal{F}$ is a unit ball in a reproducing kernel Hilbert space (RKHS) $\mathcal{H}$ with kernel k, the population MMD takes a zero value if and only if $p = q$ and a positive unique value if $p \neq q$. The squared MMD can be written as the distance of mean embeddings μ_p, μ_q of distributions p and q, which

can be expressed in terms of kernel functions:

$$\begin{aligned} \mathrm{MMD}^2[F,\, p,\, q] &= \|\mu_p - \mu_q\|_H^2 \\ &= \mathbb{E}_{x,x'}\big[k(x, x')\big] - 2\mathbb{E}_{x,y}\big[k(x, y)\big] + \mathbb{E}_{y,y'}\big[k(y, y')\big] \end{aligned} \tag{3}$$

Following existing implementations of MMD in the single-cell domain[24,62], we chose a kernel that is the sum of three Gaussian kernels to increase sensitivity of the kernel to a wider range:

$$k\left(x,\, y\right) = \sum\nolimits_i \left(\frac{||x - y||^2}{\sigma_i^2}\right),\; i = 1, 2, 3 \tag{4}$$

where $_i$ denote standard deviations and were chosen to be the median of the average distance between a point to its 25 nearest neighbors divided by factors of 0.5, 1, and 2 in the three kernels, respectively.

We also used a random forests (RF) classifier and used its area under the receiver operating characteristic (AUROC) curve to determine whether the real and simulated cells can be distinguished from each other. Consistent with previous studies[24,63], we first performed a dimensionality reduction using principal component analysis (PCA) and used the top 50 PCs of each cell as the input features to the RF model, which improves the computational efficiency of this analysis. The RF model was composed of 1000 trees and the Gini impurity was used to measure the quality of a split.

Baseline simulator models

We compared the performance of GRouNdGAN to scDESIGN2[25], SPARsim[26], and three GAN-based methods: scGAN[24], cscGAN with projection-based conditioning[24], and a conditional WGAN (cWGAN). The cWGAN method conditions by concatenation following the cGAN framework[64]. More specifically, it concatenates a one-hot encoded vector (representing the cluster number or cell type) to the noise vector input to the generator and cells forwarded to the discriminator. We did not train the cWGAN or cscGAN on the PBMC-CTL dataset, since it contains only one cell type. For the PBMC-All and the BoneMarrow dataset, we trained all models above. Additionally, we simulated data using scDESIGN2 and SPARsim with and without cell cluster information, as they allow providing such side information in their training.

To train models that utilized cell cluster information, we performed Louvain clustering and provided the cluster information and ratio of cells per cluster during training. Clustering was done by following the cell ranger pipeline[33], based on the raw unprocessed dataset (and independent of the pre-processing steps described earlier for training simulators). First, genes with no UMI count in any of the cells were removed. Then the gene expression profile of each

cell was normalized by the total UMI of all (remaining) genes, and highly variable genes were identified. The gene expression profile of each cell was then re-normalized by the total UMI of retained highly variable genes, and each gene vector (representing its expression across different cells) was z-score normalized. Given the normalized gene expression matrix above, we found top 50 principal components (PCs) using PCA analysis. These PCs were then used to compute a neighborhood graph with a local neighborhood size of 15, which was used in Louvain clustering. We ran the Louvain algorithm with a resolution of 0.15.

For SPARSim, we set all sample library sizes to 20000 and estimated gene expression level intensities and simulation parameters by providing it with both raw and normalized count matrices. When cell cluster information was provided, distinct SPARSim simulation parameters were estimated per cell for each cluster. scDESIGN2 accepts input matrices where entries are integer count values; we thus performed rounding on the expression matrix before fitting scDESIGN2. With cluster information provided, a scDESIGN2 model was fit separately for cells of each cluster, and similar to conditional GANs, the ratio of cells per cluster was provided to the method.

In silico perturbation experiments using GRoundGAN

To perform perturbation experiments using GRoundGAN, we put the trained model in a deterministic mode of operation. This is necessary to ensure that the perturbation experiments are performed on the same batch (i.e., replicate) of generated cells to form matched case/control experiments. To do this, we performed a forward pass through the generator and then saved the input noise to the causal controller, the input noise to the target generators, the TF expression values generated by the causal controller, and the per-cell scaling factor of the LSN layer. Subsequent passes through the generators used the saved parameters so that ensuing runs always output the same batch of cells (instead of generating new unmatched cells).

Trajectory inference and pseudo-time analysis

Following official PAGA tutorial for the BoneMarrow dataset (`https://github.com/scverse/scanpy-tutorials/blob/master/paga-paul15.ipynb`), we used (Partition-based graph abstraction) PAGA[38] for trajectory inference and analysis. We built force-directed graphs[65] (with ForceAtlas2[66]) using the top 20 principal components of the data (using principal component analysis or PCA) and a neighborhood graph of observations computed using UMAP (to estimate connectivities). We next denoised the graph by representing it in the diffusion map space and computed distances and neighbors as before using this new representation. After denoising, we then ran the Louvain clustering algorithm with a resolution of 0.6. Finally, we

ran the PAGA algorithm on the identified clusters and used the obtained graph to initialize and rebuild the force-directed graph.

GRN inference methods

In our GRN benchmarking analysis, we focused on eight GRN inference algorithms: GENIE3[9], GRNBoost2[8], PPCOR[37], PIDC[10], LEAP[12], SCODE[13] and SINCERITIES[14], which were used in the BEELINE study[19], as well as CeSpGRN[48]. Of these methods, LEAP requires pseudo-time ordering of cells, while SCODE and SINCERITIES require both pseudo-time ordering and pseudo-time values. Since not all algorithms inferred the edge directionality or its sign (activatory or inhibitory nature), we did not consider these factors in our analysis to be consistent among different models.

For the methods available in the BEELINE study, we ran them as docker containers using the docker images provided by BEELINE's GitHub (`https://github.com/Murali-group/Beeline`)[19] with the default parameters used in BEELINE. These methods were applied to nine datasets simulated by GRouNdGAN and scGAN, and the original training real dataset corresponding to PBMC-CTL, PBMC-All, and BoneMarrow. To ensure consistency, the same number of cells as the real training set were simulated using GRouNdGAN and scGAN for each dataset: $n = 19773$ for PBMC-CTL, $n = 67579$ for PBMC-All, and $n = 2230$ for BoneMarrow. Number of genes and TFs present in the GRN for each dataset is provided in Supplementary Data 1 - Sheet 3. To benchmark algorithms requiring pseudo-time ordering of cells, we computed the pseudo-times of GRouNdGAN-simulated data (based on the BoneMarrow dataset) using PAGA[38] and diffusion pseudotime[39], following the methodology described earlier. In the GRN inference benchmark analysis, we did not provide the list of TFs to GRNBoost2 to make it consistent with other GRN inference methods.

We also included CeSpGRN, which is a cell-specific GRN inference method. Since this method first generates one GRN for each cell, it requires a high amount of memory to run. As a result, we were only able to benchmark it using a subset of data consisting of only $n = 1000$ cells (for any of the three datasets) and 100 genes (we did not change the number of TFs, or the GRN edges connecting them to the considered genes). Following the method described in the original study, we then averaged the total absolute edge weights across all cells to form a consensus GRN using CeSpGRN.

Statistics and reproducibility

The sample sizes were selected by original studies producing datasets used for training GRouNdGAN. Preprocessing steps are described in the datasets and preprocessing section. Other than standard cell-level and gene-level filtering, there were no data exclusions. Statistical tests used for each analysis are described in the corresponding sections and include Wilcoxon signed rank and Mann Whitney U tests.

Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this article.

Data availability

The PMBC reference dataset is available from the 10x Genomics repository (corresponding to healthy donor A) from the following link: `https://support.10xgenomics.com/single-cell-gene-expression/datasets/1.1.0/fresh_68k_pbmc_donor_a?` (related to PBMC-All, PBMC-CTL, and PBMC-NaiveT). The BoneMarrow reference dataset is available in the Gene Expression Omnibus (GEO) repository under accession number GSE72857. The Dahlin reference dataset is available in GEO (accession number: GSE107727). The Tumor-All (and Tumor-malignant) dataset is available from cellxgene (`https://cellxgene.cziscience.com/collections/968834a0-1895-40df-8720-666029b3bbac`). Simulated data corresponding to PBMC-All, PBMC-CTL, BoneMarrow, Dahlin, Tumor-All, and Tumor-malignant and their corresponding imposed GRNs are provided on GRouNdGAN's website (`https://emad-combine-lab.github.io/GRouNdGAN/benchmarking`) and can be used for benchmarking different GRN Inference methods. Source data are provided with this paper.

Code availability

Our implementation and evaluation of GRouNdGAN in Python 3.9.6 using the PyTorch framework[67] along with a tutorial is freely available under the GNU Affero General Public License v3.0 on GitHub (`https://github.com/Emad-COMBINE-lab/GRouNdGAN`) which is archived in Zenodo under record number 11068246[68] (`https://doi.org/10.5281/zenodo.11068246`). The repository also contains a Docker image for reproducibility and our PyTorch implementation of scGAN, projection-based conditioning cscGAN[24] and a variation

of cscGAN that uses conditioning by concatenation (cWGAN). More information about the installation, a detailed tutorial, and simulated datasets that can be readily used are provided in (https://emad-combine-lab.github.io/GRouNdGAN).

References

1. Lee, T. I. et al. Transcriptional regulatory networks in Saccharomyces cerevisiae. *Science* **298**, 799–804 (2002).

2. Che, D. et al. Dynamic and modular gene regulatory networks drive the development of gametogenesis. *Brief. Bioinform.* **18**, 712–721 (2017).

3. Olson, E. N. Gene regulatory networks in the evolution and development of the heart. *Science* **313**, 1922–1927 (2006).

4. Dibaeinia, P. & Sinha, S. SERGIO: a single-cell expression simulator guided by gene regulatory networks. *Cell Syst.* **11**, 252–271. e211 (2020).

5. Yang, Y. et al. Gene knockout inference with variational graph autoencoder learning single-cell gene regulatory networks. *Nucleic Acids Res.* **51**, 6578–6592 (2023).

6. Madhamshettiwar, P. B., Maetschke, S. R., Davis, M. J., Reverter, A. & Ragan, M. A. Gene regulatory network inference: evaluation and application to ovarian cancer allows the prioritization of drug targets. *Genome Med.* **4**, 1–16 (2012).

7. Manuel, A. M., Dai, Y., Jia, P., Freeman, L. A. & Zhao, Z. A gene regulatory network approach harmonizes genetic and epigenetic signals and reveals repurposable drug candidates for multiple sclerosis. *Hum. Mol. Genet.* **32**, 998–1009 (2023).

8. Moerman, T. et al. GRNBoost2 and Arboreto: efficient and scalable inference of gene regulatory networks. *Bioinformatics* **35**, 2159–2161 (2019).

9. Huynh-Thu, V. A., Irrthum, A., Wehenkel, L. & Geurts, P. Inferring regulatory networks from expression data using tree-based methods. *PloS one* **5**, e12776 (2010).

10. Chan, T. E., Stumpf, M. P. & Babtie, A. C. Gene regulatory network inference from single-cell data using multivariate information measures. *Cell Syst.* **5**, 251–267. e253 (2017).

11. Emad, A. & Sinha, S. Inference of phenotype-relevant transcriptional regulatory networks elucidates cancer type-specific regulatory mechanisms in a pan-cancer study. *NPJ Syst. Biol. Appl.* **7**, 9 (2021).

12. Specht, A. T. & Li, J. LEAP: constructing gene co-expression networks for single-cell RNA-sequencing data using pseudotime ordering. *Bioinformatics* **33**, 764–766 (2017).

13. Matsumoto, H. et al. SCODE: an efficient regulatory network inference algorithm from single-cell RNA-Seq during differentiation. *Bioinformatics* **33**, 2314–2321 (2017).

14. Papili Gao, N., Ud-Dean, S. M., Gandrillon, O. & Gunawan, R. SINCERITIES: inferring gene regulatory networks from time-stamped single cell transcriptional expression profiles. *Bioinformatics* **34**, 258–266 (2018).

15. Aibar, S. et al. SCENIC: single-cell regulatory network inference and clustering. *Nat. Methods* **14**, 1083–1086 (2017).

16. Bravo González-Blas, C. et al. SCENIC+: single-cell multiomic inference of enhancers and gene regulatory networks. *Nat. Methods* **20**, 1355–1367 (2023).

17. Kartha, V. K. et al. Functional inference of gene regulation using single-cell multi-omics. *Cell Genomics* **2**, 100166 (2022).

18. Badia-i-Mompel, P. et al. Gene regulatory network inference in the era of single-cell multi-omics. *Nat. Rev. Genet.* **24**, 739–754 (2023).

19. Pratapa, A., Jalihal, A. P., Law, J. N., Bharadwaj, A. & Murali, T. Benchmarking algorithms for gene regulatory network inference from single-cell transcriptomic data. *Nat. Methods* **17**, 147–154 (2020).

20. Emmert-Streib, F., Dehmer, M. & Haibe-Kains, B. Gene regulatory networks and their applications: understanding biological and medical problems in terms of networks. *Front. Cell Dev. Biol.* **2**, 38 (2014).

21. Kolmykov, S. et al. GTRD: an integrated view of transcription regulation. *Nucleic Acids Res.* **49**, D104–D111 (2021).

22. Xu, H. et al. ESCAPE: database for integrating high-content published data collected from human and mouse embryonic stem cells. *Database* **2013**, bat045 (2013).

23. Kanehisa, M. & Goto, S. KEGG: kyoto encyclopedia of genes and genomes. *Nucleic Acids Res.* **28**, 27–30 (2000).

24. Marouf, M. et al. Realistic in silico generation and augmentation of single-cell RNA-seq data using generative adversarial networks. *Nat. Commun.* **11**, 1–12 (2020).

25. Sun, T., Song, D., Li, W. V. & Li, J. J. scDesign2: a transparent simulator that generates high-fidelity single-cell gene expression count data with gene correlations captured. *Genome Biol.* **22**, 1–37 (2021).

26. Baruzzo, G., Patuzzi, I. & Di Camillo, B. SPARSim single cell: a count data simulator for scRNA-seq data. *Bioinformatics* **36**, 1468–1475 (2020).

27. Schaffter, T., Marbach, D. & Floreano, D. GeneNetWeaver: in silico benchmark generation and performance profiling of network inference methods. *Bioinformatics* **27**, 2263–2270 (2011).

28. Marbach, D. et al. Revealing strengths and weaknesses of methods for gene network inference. *Proc. Natl Acad. Sci.* **107**, 6286–6291 (2010).

29. Marbach, D. et al. Wisdom of crowds for robust gene network inference. *Nat. Methods* **9**, 796–804 (2012).

30. Chen, S. & Mar, J. C. Evaluating methods of inferring gene regulatory networks highlights their lack of performance for single cell gene expression data. *BMC Bioinforma.* **19**, 1–21 (2018).

31. Kocaoglu, M., Snyder, C., Dimakis, A. G. & Vishwanath, S. CausalGAN: Learning causal implicit generative models with adversarial training. Preprint at https://arxiv.org/abs/1709.02023 (2017).

32. Arjovsky, M., Chintala, S. & Bottou, L. Wasserstein generative adversarial networks. In *International conference on machine learning*. 214–223 (PMLR, 2017).

33. Zheng, G. X. et al. Massively parallel digital transcriptional profiling of single cells. *Nat. Commun.* **8**, 1–12 (2017).

34. Paul, F. et al. Transcriptional heterogeneity and lineage commitment in myeloid progenitors. *Cell* **163**, 1663–1677 (2015).

35. Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B. & Smola, A. A kernel two-sample test. *J. Mach. Learn. Res.* **13**, 723–773 (2012).

36. Korsunsky, I. et al. Fast, sensitive and accurate integration of single-cell data with Harmony. *Nat. Methods* **16**, 1289–1296 (2019).

37. Kim, S. ppcor: an R package for a fast calculation to semi-partial correlation coefficients. *Commun. Stat. Appl. Methods* **22**, 665 (2015).

38. Wolf, F. A. et al. PAGA: graph abstraction reconciles clustering with trajectory inference through a topology preserving map of single cells. *Genome Biol.* **20**, 1–9 (2019).

39. Haghverdi, L., Büttner, M., Wolf, F. A., Buettner, F. & Theis, F. J. Diffusion pseudotime robustly reconstructs lineage branching. *Nat. Methods* **13**, 845–848 (2016).

40. Trapnell, C. et al. Pseudo-temporal ordering of individual cells reveals dynamics and regulators of cell fate decisions. *Nat. Biotechnol.* **32**, 381 (2014).

41. Kumar, P., Tan, Y. & Cahan, P. Understanding development and stem cells using single cell-based analyses of gene expression. *Development* **144**, 17–32 (2017).

42. Saelens, W., Cannoodt, R., Todorov, H. & Saeys, Y. A comparison of single-cell trajectory inference methods. *Nat. Biotechnol.* **37**, 547–554 (2019).

43. Street, K. et al. Slingshot: cell lineage and pseudotime inference for single-cell transcriptomics. *BMC Genom.* **19**, 1–16 (2018).

44. Hwang, B., Lee, J. H. & Bang, D. Single-cell RNA sequencing technologies and bioinformatics pipelines. *Exp. Mol. Med.* **50**, 1–14 (2018).

45. Wolf, F. A., Angerer, P. & Theis, F. J. SCANPY: large-scale single-cell gene expression data analysis. *Genome Biol.* **19**, 1–5 (2018).

46. Dahlin, J. S. et al. A single-cell hematopoietic landscape resolves 8 lineage trajectories and defects in Kit mutant mice. *Blood J. Am. Soc. Hematol.* **131**, e1–e11 (2018).

47. Han, G. et al. Follicular lymphoma microenvironment characteristics associated with tumor cell mutations and MHC class II expression. *Blood Cancer Discov.* **3**, 428–443 (2022).

48. Zhang, Z., Han, J., Song, L. & Zhang, X. Inferring cell-specific gene regulatory networks from single cell gene expression data. Preprint at https://www.biorxiv.org/content/10.1101/2022.03.03.482887v3.full (2022).

49. Yuan, Y. & Bar-Joseph, Z. Deep learning for inferring gene relationships from single-cell expression data. *Proc. Natl Acad. Sci.* **116**, 27151–27158 (2019).

50. Chen, J. et al. DeepDRIM: a deep neural network to reconstruct cell-type-specific gene regulatory network using single-cell RNA-seq data. *Brief. Bioinforma.* **22**, bbab325 (2021).

51. Zhang, S. Y. & Stumpf, M. P. Learning cell-specific networks from dynamical single cell data. Preprint at https://www.biorxiv.org/content/10.1101/2023.01.08.523176v2 (2023).

52. Crowell, H. L., Morillo Leonardo, S. X., Soneson, C. & Robinson, M. D. The shaky foundations of simulating single-cell RNA sequencing data. *Genome Biol.* **24**, 1–19 (2023).

53. Gao, Y., Shen, L. & Xia, S.-T. DAG-GAN: Causal structure learning with generative adversarial nets. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 3320-3324 (IEEE, 2021).

54. Satija, R., Farrell, J. A., Gennert, D., Schier, A. F. & Regev, A. Spatial reconstruction of single-cell gene expression data. *Nat. Biotechnol.* **33**, 495–502 (2015).

55. Goodfellow, I. et al. Generative adversarial networks. *Commun. ACM* **63**, 139–144 (2020).

56. Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V. & Courville, A. C. Improved training of wasserstein gans. In *Advances in neural information processing systems*, 30 (NeurIPS, 2017).

57. Reddi, S. J., Kale, S. & Kumar, S. On the convergence of adam and beyond. Preprint at https://arxiv.org/abs/1904.09237 (2019).

58. He, K., Zhang, X., Ren, S. & Sun, J. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*. 1026–1034 (IEEE, 2015).

59. Glorot, X. & Bengio, Y. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. 249-256 (JMLR Workshop and Conference Proceedings, 2010).

60. Ioffe, S. & Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*. 448-456 (PMLR, 2015).

61. Hu, H. et al. AnimalTFDB 3.0: a comprehensive resource for annotation and prediction of animal transcription factors. *Nucleic Acids Res.* **47**, D33–D38 (2019).

62. Shaham, U. et al. Removal of batch effects using distribution-matching residual networks. *Bioinformatics* **33**, 2539–2546 (2017).

63. Shu, H. et al. Modeling gene regulatory networks using neural network architectures. *Nat. Comput. Sci.* **1**, 491–501 (2021).

64. Mirza, M. & Osindero, S. Conditional generative adversarial nets. Preprint at https://arxiv.org/abs/1411.1784 (2014).

65. Islam, S. et al. Characterization of the single-cell transcriptional landscape by highly multiplex RNA-seq. *Genome Res.* **21**, 1160–1167 (2011).

66. Jacomy, M., Venturini, T., Heymann, S. & Bastian, M. ForceAtlas2, a continuous graph layout algorithm for handy network visualization designed for the Gephi software. *PloS One* **9**, e98679 (2014).

67. Paszke, A. et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems* 32 (NeurIPS, 2019).

68. Zinati, Y., Takiddeen, A. & Emad, A. GRouNdGAN. Zenodo https://doi.org/10.5281/zenodo.11068246 (2024). article-v1.0.0.

Acknowledgements

This work was supported by grants from Natural Sciences and Engineering Research Council of Canada (NSERC) [RGPIN-2019-04460] (A.E.), Government of Canada's New Frontiers in Research Fund (NFRF) [NFRFE-2019-01290] (A.E.), Canada Foundation for Innovation (CFI) JELF [project 40781], and McGill Initiative in Computational Medicine (A.E.). This research was enabled in part by support provided by the Digital Research Alliance of Canada (`https://alliancecan.ca`) and Calcul Québec (`www.calculquebec.ca`). *This publication is part of the Human Cell Atlas* –`www.humancellatlas.org/publications`.

Author contributions

All authors contributed to the design of the study and the algorithm, the analyses, and the writing of the manuscript. A.E. supervised the study. Y.Z. implemented the model and performed the statistical analyses. Y.Z., A.T., and A.E. read and approved the final manuscript.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at https://doi.org/10.1038/s41467-024-48516-6.

Correspondence and requests for materials should be addressed to Amin Emad.

Peer review information Nature Communications thanks Eric Lu Zhang, who co-reviewed with Xu Yuand Hao Li for their contribution to the peer review of this work. A peer review file is available.

Reprints and permissions information is available at http://www.nature.com/reprints

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/ licenses/by/4.0/.

© The Author(s) 2024

Background and Context for "*Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit*"

Chapter 4

Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit

Abstract

Background Rapid advances in high-throughput single-cell sequencing technologies, coupled with the development of computational methods capable of leveraging large datasets, have led to the emergence of numerous approaches for deciphering regulatory interactions in the form of Gene Regulatory Networks (GRNs). However, in the absence of context-specific gold-standard ground truths, particularly those containing causal interactions, systematically benchmarking GRN inference methods remains a challenge. Thus, we previously developed GRouNdGAN, a causal implicit generative model capable of simulating realistic observational and interventional scRNA-seq data following a user-defined GRN on any biological system of interest. Importantly, we demonstrated that GRouNdGAN generates datasets that bridge the gap between experimentation and simulation for GRN inference benchmarking.

Method Building upon the GRouNdGAN framework, we developed an extended toolkit that offers additional features, including interactive model visualization, training monitoring, more customizable GRN creation options, synthetic data similarity and GRN inference benchmarking metrics, and an intuitive TF knockout prediction module. Here, we provide a step-by-step procedure for implementing the protocol from start to finish and introduce alternative variations to adapt GRouNdGAN to studies with different experimental setups. GRouNdGAN-Toolkit is publicly available as a python code repository and containerized application and is accompanied by a tutorial website featuring a collection of simulated datasets. Model training largely depends on graphic hardware and the size and density of the input GRN, and typically takes around 75h to complete on a single GPU. Excluding model training, this protocol typically takes less than

25min to complete.

Discussion GRouNdGAN-Toolkit is a versatile simulator with user friendly interface that does not assume advanced computational genomics expertise, enhancing its usability and accessibility across a wide range of users.

Keywords: GRouNdGAN, Gene Regulatory Networks, GRN Inference, GRN Inference Benchmarking, scRNA-seq, Data Augmentation, In-Silico Perturbation.

Introduction and Background

Understanding how the gene expression is regulated within cells, including when, where, and to what extent a gene is transcribed and translated into a functional product like a protein, is fundamental to molecular biology[1, 2]. These regulatory processes, collectively known as gene regulation, are often conceptualized as transcriptional or gene regulatory networks (GRNs), which capture the dynamic interactions among genes and their regulators. To allow for computational analysis, GRNs are often modeled as directed acyclic bipartite graphs, linking transcription factors (TFs) to their target genes[3]. Inferring GRNs from bulk or single-cell RNA sequencing data[4-7] alone or in combination with other data modalities[8-10], has long been a topic of significant interest. This is due to the fact that GRNs play a critical role in our understanding of biological processes involved in cellular development, function, and disease, with applications in identifying therapeutic targets and predicting the outcomes of perturbation experiments[11]. However, systematically benchmarking GRN inference tools remains a challenge due to the absence of context-specific gold-standard ground truth networks and their associated datasets, particularly those that contain causal interactions[12]. While a ground truth GRN could be constructed from literature and curated biological databases of interactions, it may not accurately reflect the context- and cell type-specific regulatory programs of the biological system under study as they are aggregated across various datasets. Experimental validation through perturbation experiments could provide definitive evidence for the presence or absence of individual interactions, but such approaches are costly, time-consuming, and laborintensive[11, 13, 14]. To address this, multiple GRN-guided simulators have been developed, offering an alternative means of generating benchmarking datasets[12, 15-17]. However, existing simulators often fail to impose regulatory relationships causally within their generated data and have key limitations (see our previous study for a detailed discussion[18]). Moreover, reported mismatches between benchmarking results obtained from experimental and simulated datasets raise concerns about their reliability[12].

Development of the protocol

We developed GRouNdGAN[18], a generative deep learning model capable of simulating realistic single cell data, performing in-silico perturbation experiments, and systematically benchmarking GRN inference methods. GRouNdGAN combines and builds upon elements from previous studies on generative adversarial networks (GANs) namely scGAN[19] and causalGAN[20], to generate synthetic single cells mimicking those within a provided reference dataset while imposing causal relationships between TFs and genes as defined by a user-provided GRN.

Our approach follows a two-step training process. First, we train a Wasserstein GAN[21] with gradient penalty (WGAN-GP)[22] to generate realistic single cells without the regulatory constraints of the GRN. Once trained, we freeze the parameters of the generator to use as a "causal controller" in training a second WGAN-GP. In the second step, random Gaussian noise, along with TF expression values coming from the causal controller, are provided to a generator (called the "target generator") which derives its neural network connections from the provided GRN. The masking of neural connections in the target generator ensures that target genes are causally expressed under the influence of their regulating TFs and some unobserved/unknown confounders (modelled by the inputted random noise unit). Both WGAN-GPs include a library size normalization layer as their last layer. Since only unnormalized TF expression values are needed from the causal controller, the LSN layer of the first WGAN-GP is discarded after it is trained in the first step. These TF expression values, along with the target gene expression values regenerated by the target generator, are then fed into the LSN layer of the target generator for final normalization.

Both WGAN-GPs are trained in an adversarial manner, with a "Critic" that evaluates generated samples by estimating the Wasserstein distance between the reference and simulated data distributions, and a generator that continuously refines its output to minimize this distance. Through this minimax game, both components co-evolve in a dynamic feedback loop, each improving in response to the other.

Unlike traditional GANs, GRouNdGAN employs two auxiliary networks, the labeler and antilabeler, to ensure the target generator incorporates TF expression when generating target gene expression values, rather than relying solely on the input noise. These networks predict TF expression from target gene expression by minimizing the L2 loss between estimated and true TF values. The anti-labeler is trained exclusively on generated data, while the labeler is trained on both real and generated data to make further connection to the biology of the reference dataset. This approach reinforces causal TF-gene relationships within the GAN framework ensuring that regulatory consistency and realistic single-cell generation are optimized in tandem.

Applications of the method

In our original study[18], we demonstrated that GRouNdGAN possesses essential properties for systematically benchmarking of GRN inference algorithms. This was achieved by generating benchmarking datasets based on six experimental datasets, and various causal GRN configurations. In summary, GRouNdGAN generates realistic scRNA-seq data while simultaneously imposing the causal relationships defined by the user (provided as an input GRN). It ensures that imposed edges are emphasized, making them recoverable by a high-performing GRN inference algorithm. Conversely, regulatory relationships that were intrinsic to the reference biological data, yet were not part of the input GRN are disrupted in the simulated dataset, limiting the potential of bias and ensuring a fair evaluation of GRN inference methods. We observed that benchmarking results of seven GRN inference methods using GRouNdGAN were consistent with insights previously found by BEELINE[12] from curated and experimental datasets, suggesting that GRouNdGAN effectively bridges the gap between experimental and simulated data benchmarks. Since then, GRouNdGAN has facilitated the development and evaluation of new GRN inference methods, such as LLM4GRN[3] and scRegulate[23], enabling researchers to carry out a systematic comparative evaluation of their method compared to the state-of-the-art. Figure 1 demonstrates an overview of the GRouNdGAN pipeline.

In addition to sampling from observational distributions to generate realistic benchmarking datasets, GRouNdGAN can also sample from interventional distributions and generate cells that do not naturally occur in the dataset, since it is a causal generative model. We previously showcased this application by performing in silico knock out experiments on top differentially expressed TFs of specific cell types, and observing their effect on the generated cells. The in silico perturbations can be performed during inference by manipulating the TF expressions generated by the causal controller, which are then fed to the target generator to produce the regulated target genes. In GRouNdGAN-Toolkit, we provide a user-friendly interface that allows users to generate cells under various perturbation conditions such as knockout. This application is not only suitable for predicting the outcome of specific perturbations but also for creating case/control experiments for studies that require such setups. Since our in silico perturbation is applied to the same batch of cells, keeping all generation parameters such as noise constant, it is deterministic, allowing to observe the effect of perturbations on the exact set of cells, providing matched case/control simulated datasets. This is not easily possible on experimental scRNA-seq data since most sequencing technologies are destructive, preventing the same cell from being sequenced twice (before and after perturbation).

As a specialized extension of general-purpose simulators, GRouNdGAN can fulfill all the same functions of non-causal simulators, when a GRN that accurately represents the reference dataset is imposed. Interestingly, we previously observed that general-purpose simulators that do not

explicitly impose a causal graph, focus on generating realistic cells at the expense of compromising TF-gene relationships of the reference experimental dataset[18]. In this regard, GRouNdGAN may be a better choice even for tackling use cases typically handled by generalpurpose simulators, as we have previously demonstrated that it outperforms state-of-the-art simulators in generating realistic data[18]. Here, we will briefly describe some use-cases of simulated data that GRouNdGAN can also support. Simulated data can be used for data augmentation to supplement limited sample availability caused by biological constraints, such as rare cell types, specific contexts, or populations. It can also help overcome technical limitations, like low-quality or low-yield sequencing data, and address ethical concerns related to human or animal experiments. The ability to generate synthetic data can significantly reduce the need for wet-lab experiments, which leads to considerable time and cost savings, particularly when testing biological hypotheses. Moreover, since GRouNdGAN can generate an unlimited number of cells without duplicates, it helps overcome challenges related to small sample sizes, which might not accurately reflect the broader population and can lead to poor robustness and reduced reproducibility of experimental results, though further validation is needed to confirm its effectiveness in this context.

In our original study, we showed that GRoundGAN-generated datasets effectively preserve key biological features such as cellular trajectories, pseudo-time ordering of cells, and marker gene expression. As a result, GRouNdGAN-generated data has the potential to improve downstream analyses, such as marker gene detection, classification of sparse cells, and trajectory inference, as previously demonstrated by scGAN[24] on their own data. Moreover, augmented data can enhance denoising and dropout imputation algorithms. Traditional imputation methods typically rely on borrowing information from similar cells within the dataset, but this approach can lead to oversmoothing, particularly for rarer cell populations, ultimately suppressing natural cell-tocell variability. sciGAN[25] for instance, has demonstrated that synthetic cells can be used for information borrowing, rather than relying solely on observed cells, thus preserving the inherent stochasticity of the data. Finally, simulated data has the potential to be studied as means of sharing transcriptomic datasets at a reduced risk of exposing sensitive information, all while preserving the statistical and biological integrity of the original experimental reference data. However, demonstrating the effectiveness of this approach in addressing privacy concerns remains an ongoing challenge and is beyond the scope of our current work.

Comparison with other methods

Several tools have been developed for simulation of scRNA-seq data over the years[24, 26-29]. However, most are general-purpose simulators that mainly focus on generating high-fidelity data that adheres to the statistical properties of experimental datasets. Using such simulators, one can augment data, improve downstream analysis, and benchmark some computational tools

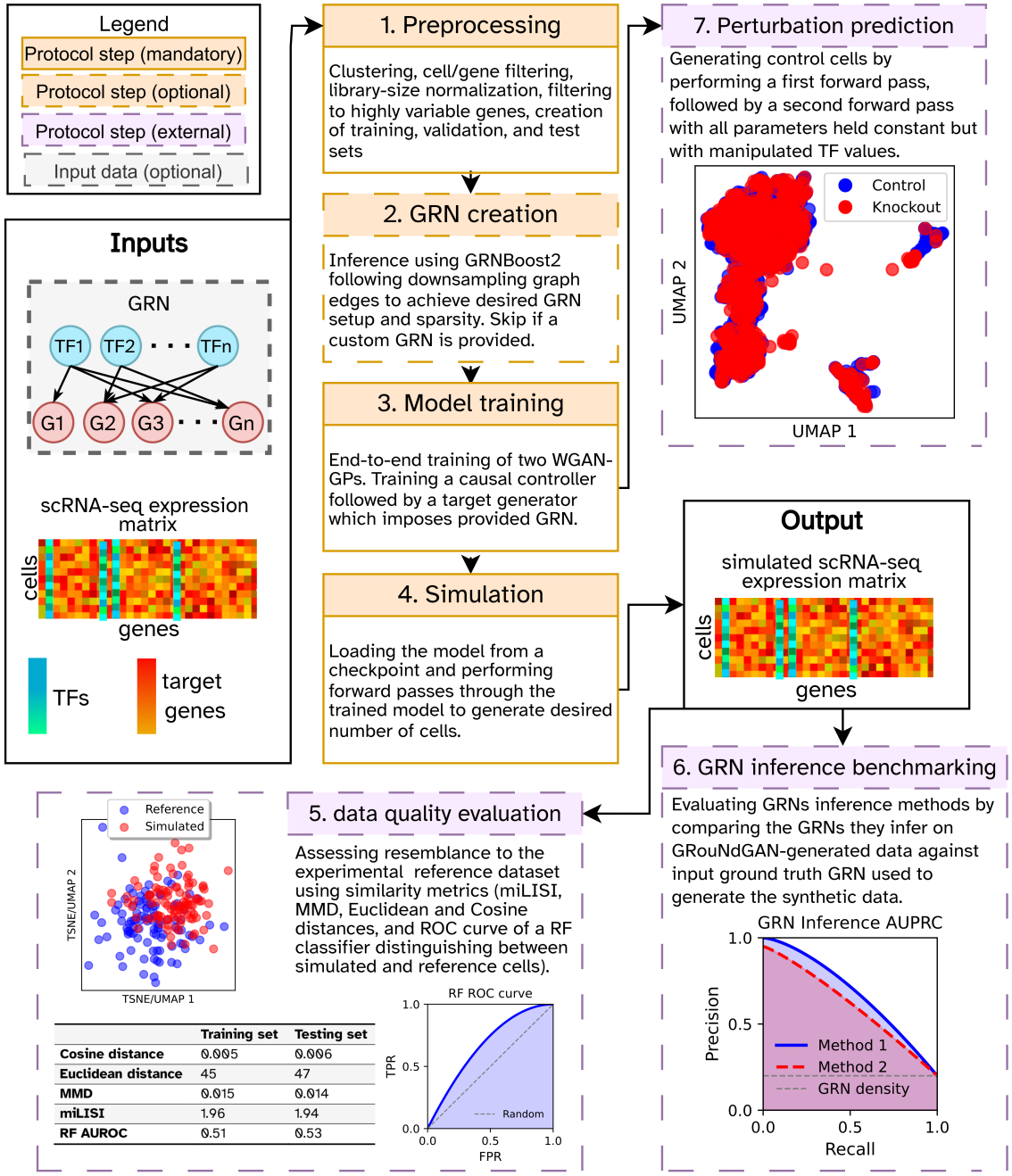


Figure 1: Schematic overview of the GRouNdGAN pipeline. Left: GRouNdGAN's input data consists of a scRNA-seq dataset and an optional user-provided GRN. A list of TF names must be provided to differentiate TFs from target genes. Middle (light orange): Main steps in the GRouNdGAN protocol. The GRN creation step can be skipped only if a GRN is provided by the user. Right and bottom (in light purple): External protocol steps that involve using GRouNdGAN's trained model or simulated data for secondary analysis and downstream tasks.

but cannot benchmark GRN inference algorithms. These simulators have shown to capture genegene correlations, but do not explicitly simulate data following a regulatory program that is either user-provided or extracted from the dataset, making them unsuitable for benchmarking GRN inference algorithms. Only a small number of simulators are explicitly guided by a user-defined GRN. One of the earliest models is GeneNetWeaver (GNW)[17] that was used to simulate bulk RNA-seq for benchmarking multiple GRN inference methods during Dialogue for Reverse Engineering Assessment and Methods (DREAM) challenges[17, 30, 31]. However, since it was originally designed for bulk RNA-seq data generation, its simulated datasets do not usually exhibit the statistical properties of single-cell data. Although there have been multiple attempts at adapting GNW to single-cell data, it falls short of modern simulators and cannot simulate data with multiple cell types, contexts, or trajectories[15].

SERGIO[15] and BoolODE[12] are single-cell GRN-guided simulators that use stochastic differential equations (SDEs) to model gene expression dynamics. BoolODE is a de novo simulator, meaning that it generates data entirely from scratch based solely on user-defined parameters and rules, without relying on a reference biological dataset. As a result, it cannot be used to benchmark GRN inference methods under specific contexts and conditions that are unique to an experimental dataset. Instead of a reference dataset, BoolODE relies on carefully selected user-defined parameters such as noise, dropout rate, and kinetic parameters to impose properties such as dropout, batch effect, variation across samples, population, and conditions. Although this gives the end-users greater control over the data generating process, it also places the responsibility of fully harnessing this flexibility on them, making it more challenging to generate new datasets. In other words, without extensive knowledge of single-cell transcriptomics and BoolODE's inner workings, one may risk creating simulations that do not reflect experimental datasets. SERGIO[15] is a hybrid de novo/reference based simulator that first generates a "clean" dataset, strictly following a GRN without aligning to a reference dataset. The user then refines this simulated data by iteratively adjusting three parameters (controlling technical noise) to match a reference dataset, by assessing similarity based on five statistical measures: library sizes, mean gene expression, gene expression variance, and zero counts per gene and per cell. This approach introducessubjectivity that affects reproducibility of the results. In our experiments, even after the addition of technical noise, data quality remained subpar compared to other simulators. Furthermore, since technical noise is introduced after the causal relationships have already been encoded, it may unintentionally alter or disrupt interactions that should remain unchanged for a GRN benchmarking experiment. This can make relationships unidentifiable in the final dataset, potentially explaining the significant drop in the performance of all GRN inference methods tested by the authors in the original study[15].

On the other hand, GRouNdGAN optimizes its parameters to simultaneously impose causal TFgene relationships while minimizing the distance between simulated and reference data distri-

butions. This process is fully automated, requiring no user intervention or post-processing to preserve technical and biological noise, and cellular dynamics (such as pseudotime and trajectories). Unlike SERGIO and BoolODE, which generate pseudo-genes without clear mappings to the reference dataset, GRouNdGAN ensures a direct one-to-one correspondence between genes and TFs in both simulated and reference data. Each generated gene and TF mimic the expression patterns of their real counterparts. In contrast, preserving gene/TF identities in BoolODE and SERGIO requires manual fine-tuning of SDE parameters to align generated gene expressions with reference data, placing a significant burden on users. Finally, GRouNdGAN learns elaborate non-linear co-regulatory dynamics through implicitly parameterized functions implemented as neural networks from the data and without making simplifying assumptions. This is in contrast to the simplifying assumption of SERGIO that assumes the combinatorial effect of multiple regulators is the sum of their individual effects, or BoolODE that relies on a userprovided Boolean truth table for TF co-regulation rules.

Experimental design

Besides configurations, GRouNdGAN requires only two inputs: (1) a scRNA-seq dataset (as the reference) and (2) a gene regulatory network (GRN). It produces a single output: a simulated scRNA-seq dataset that mimics the input reference dataset, while encoding the regulatory information from the input GRN within the simulated data. By preparing the inputs and using the output in a specific way (described in this manuscript), one can carry out different types of experiments, which we will explore in this section.

Input reference data and GRN considerations

GRouNdGAN's architecture and optimization function ensures that data simulation is guided by a structural causal model defined by the input GRN. Unlike general-purpose simulators that prioritize generating realistic data often at the cost of losing underlying regulatory information, GRouNdGAN optimizes for both GRN imposition and the realism of the simulated data. One should however note that while GRouNdGAN can faithfully encode any GRN into the data it simulates, the resemblance of its output to the reference dataset deteriorates when the input GRN deviates considerably from the patterns present in the reference. In such cases, optimizing for both objectives (realistic data generation and GRN imposition) becomes contradictory rather than complementary. For this reason, we suggest users to impose GRNs that are generally in line with the reference dataset, though strict matching is not required for most applications. Having access to regulatory programs underlying the data is especially important for predicting the result of interventions. GRouNdGAN's default approach involves building a scaffold GRN

by inferring regulatory edges using GRNBoost2[5], then imposing a subset by retaining the top k interactions for each gene. The value of k can be adjusted to control the sparsity of the graph, with a denser GRN typically providing more information and leading to higher-quality simulations.

Instead of the default approach, one can use a custom GRN. Custom GRNs can be constructed by selecting a subset of edges other than top k (for example top 1^{st}, 3^{rd}, 5^{th}, ...) or those inferred by an algorithm other than the default option. Additionally, they can be constructed using data sourced from literature, structured curated biological databases (e.g., KEEG[32], I2D[33], IID[34], ESCAPE[35], and GTRD[36]) or other modalities(e.g., TF ChIP-seq or ATAC-seq). These sources can also be combined to create a more complete GRN. The edges in the input GRN do not have to reflect the true causal relationships in the reference dataset. They could arise from spurious correlations or other observed associations. However, once imposed by GRouNdGAN, these edges become the underlying causal edges of the simulated dataset.

In our original study[18], we imposed a single shared GRN on a highly heterogeneous dataset of human peripheral blood mononuclear cells (PBMC) and obtained state-of-the-art performance. However, since regulatory programs are cell type- and context-specific, it may be beneficial to impose different GRNs on subpopulations of the dataset, rather than using a single shared GRN, depending on the application. This can be easily achieved by training multiple population-specific GRouNdGAN models, each using a subpopulation of the experimental dataset along with a GRN specific to that subpopulation. To demonstrate this experimental setup, here we trained five GRouNdGAN models using cell-type-specific GRNs for the five most populated cell types in the PBMC dataset[37]. We followed the procedure outlined in Figure 2-a where preprocessing is done on the entire dataset after which it is separated based on cell type and finally used to construct cell type-specific GRNs. These cell type-specific GRNs depict distinct regulatory programs as shown by Figure 2-b, while still sharing some regulatory edges. Table 1 highlights strong performance of cell type-specific models across five metrics in generating realistic data. While this approach offers some advantages, it cannot leverage the potential benefits of information sharing, nor the performance improvements that come from using a larger training sample. Nonetheless, the cell type-specific modelsstill achieved performance comparable to that of the non-cell-type specific model (PBMC-ALL in Table 1) despite being trained on fewer samples (See Table S1). By aggregating the outputs from cell type-specific models in proportion to the relative abundance of each cell type, one can effectively reconstruct the overall dataset simulation, similar to a non-cell-type specific model (see Figure 2-c).

When there is no need to integrate the simulation across cell types, the dataset may first be partitioned by cell type, followed by independent preprocessing and GRN construction for each subset. This approach may offer some advantages: for example, one can separately preprocess and identify highly variable genes specific to each cell type. As a result, selected genes and TFs

are more biologically relevant to specific cell types, leading to the inference and construction of more representative GRNs. Table S2 summarizes the performance of cell type-specific models under this alternative experimental setup.

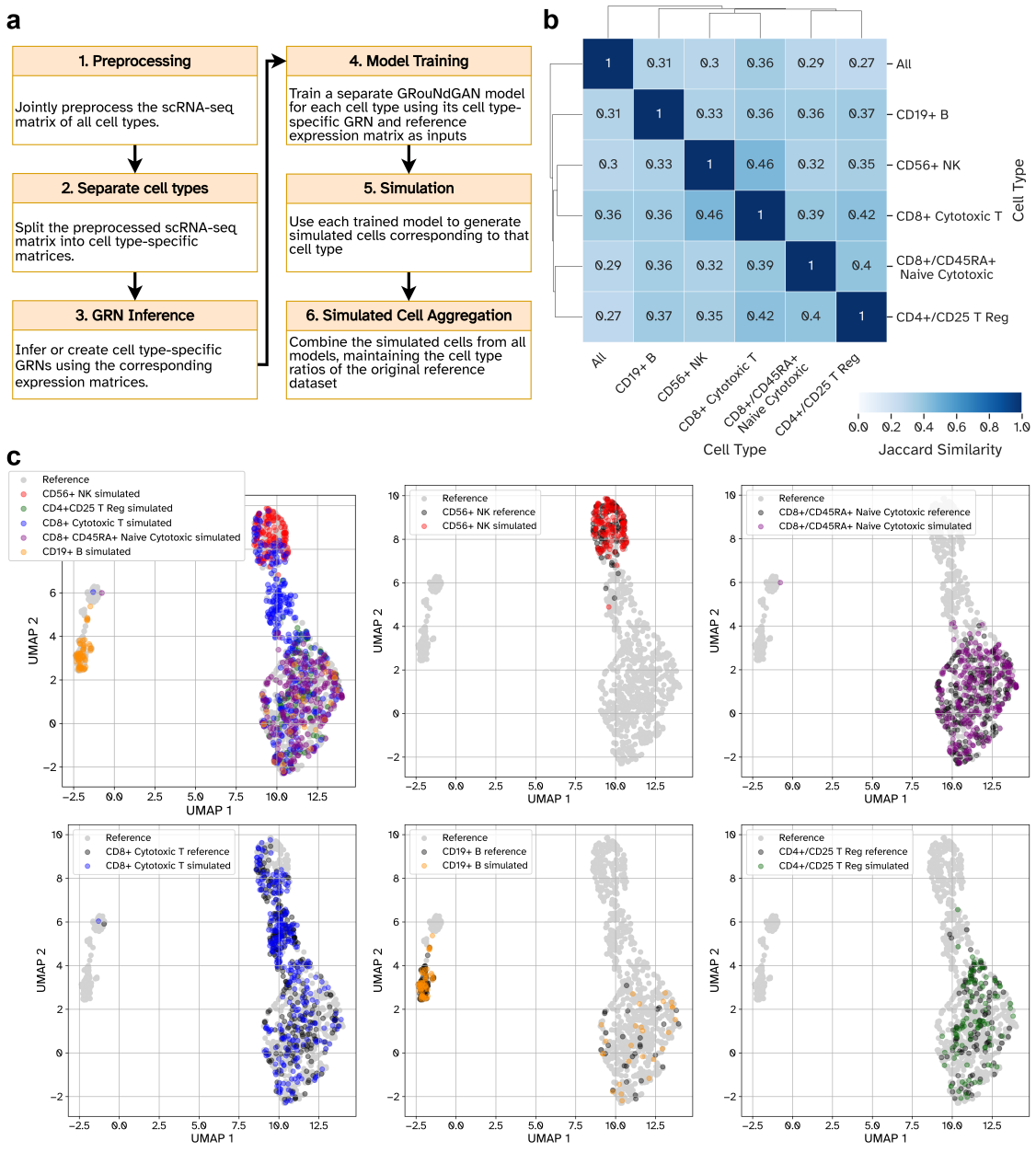


Figure 2: Cell type-specific simulation of cells using GRouNdGAN. Panel **a** illustrates an experimental setup designed for running simulations with cell type-specific GRNs. **b** Clustered heatmap showing pairwise Jaccard similarity between GRNs constructed for each cell type, based on shared TF-gene regulatory edges. Hierarchical clustering was done using average linkage and Euclidean distance. **c** Subplots show the UMAP embedding of cell type-specific simulations overlaid on reference (real) cells. The top left subfigure displays an aggregated UMAP where simulated cells from all cell type-specific models are combined in proportions matching the reference dataset (see Table S1 for relative abundance of cell types). The overlaid reference cells serve as a visual benchmark for simulation data quality.

For comprehensive benchmarking studies, we recommend that users generate multiple simu-

Table 1: Performance of GRouNdGAN in generating realistic scRNA-seq data using cell-typespecific GRNs using the PBMC dataset. The metrics are calculated by comparing a held-out test set of 1000 real cells and a simulated dataset of the same size. In the imposed GRN each gene is regulated by a maximum of 15 TFs. The default algorithm GRNBoost2 was used with the training experimental data to form the imposed GRN. For the cosine distance, Euclidean distance, and MMD, values closer to zero are desired. For RF AUROC a value closer to 0.5 and for miLISI a value closer to 2 reflect a more realistic simulation.

Simulator	Cell Type	Cosine distance	Euclidean distance	MMD	RF AUROC	miLISI
GRouNdGAN	CD8+ Cytotoxic T	0.00023	175	0.0137	0.53	1.889
Control	CD8+ Cytotoxic T	0.00017	129	0.0136	0.50	1.902
GRouNdGAN	CD8+/CD45RA+/Naïve Cytotoxic	0.00012	134	0.0161	0.51	1.902
Control	CD8+/CD45RA+/Naïve Cytotoxic	0.00011	135	0.0145	0.50	1.906
GRouNdGAN	CD56+ NK	0.00027	126	0.0181	0.59	1.885
Control	CD56+ NK	0.00022	116	0.0154	0.50	1.908
GRouNdGAN	CD4+/CD25 T Reg	0.00011	126	0.0113	0.54	1.895
Control	CD4+/CD25 T Reg	0.00011	126	0.0113	0.50	1.914
GRouNdGAN	CD19+ B	0.00018	117	0.0116	0.55	1.897
Control	CD19+ B	0.00023	125	0.0116	0.50	1.906
GRouNdGAN	PBMC-ALL	0.00018	175	0.0157	0.50	1.914
Control	PBMC-ALL	0.00019	138	0.0160	0.50	1.905

lated datasets with varying characteristics such as different GRN densities, different topologies, different number of TFs, and different number of target genes. Another approach that can enhance the rigor of a benchmarking study (previously demonstrated in our original study) involves creating two distinct GRNs. Each GRN would consist of half of the top k regulating TFs for each gene, with one serving as an unimposed control and the other used for simulation. The unimposed control GRN can serve as baseline for computing performance metrics.

Output data considerations

GRouNdGAN includes built-in functionalities to assess both data quality and GRN inference performance. Data quality is evaluated using the Area Under the Receiver Operating Characteristic Curve (AUROC) of a Random Forest (RF) distinguishing experimental from simulated cells, mean integration local inverse Simpson's index (miLISI)[38], Maximum Mean Discrepancy (MMD), and the Euclidean and Cosine distances between the centroids of experimental and simulated cells. GRN inference performance is measured with the Area Under the Precision-Recall Curve (AUPRC) and precision. For detailed explanations of each metric, please refer to our original publication.

A key advantage of using simulation for benchmarking GRN inference methods is the ability to generate an unlimited number of samples, which offers significant benefits over subsampling a finite experimental dataset. This enables comprehensive benchmarking by assessing GRN

inference algorithms in terms of computational resource usage (e.g., time and memory efficiency on large datasets), and enables quantifying their stability across multiple runs, evaluating their robustness to noise, and optimizing their performance for specific applications. To demonstrate this, we assessed the performance of PIDC[6] and PPCOR[7] on simulated datasets ranging from 100 to 100,000 cells, with the results presented in Figure 3. The reference dataset (PBMC-CTL) for this consists of the most abundant cell type of the PBMC dataset[37] (i.e., CD8+ Cytotoxic T cells). Figure 3-a, b, c show that GRouNdGAN-generated cells closely resemble real cells across multiple quality metrics and in low-dimensional t-SNE space. These results are based on 100 replicate comparisons, comparing a held-out test of 1000 real cells and 100 sets of simulated cells (each containing 1000 cells for a total of 100,000), demonstrating consistent and high data quality of GRouNdGAN simulations.

As expected, we observed an improvement in AUPRC for GRN recovery as the number of cells increased (Figure 3-d) with PIDC outperforming PPCOR. PPCOR fails on very small datasets (100– 500 cells) and PIDC becomes computationally prohibitive (taking longer than 12 hours to run) on large datasets (>80,000 cells). These results underscore that GRouNdGAN can not only be used for data augmentation to increase the stability of downstream computational tools, but also to systematically evaluate their robustness and scalability.

Expertise needed to implement the protocol

Some familiarity with the command line and basic understanding of the Python programming language are required to execute this protocol. Our protocol assumes no advanced bioinformatics expertise; though, such expertise may be necessary for downstream tasks and analysis involving the output of this protocol.

Discussion and Limitations

While GRouNdGAN is a powerful generative model, it has inherent limitations that need to be considered. Through adversarial training, GRouNdGAN learns to approximate the distribution of training samples. Once trained, it samples simulated cells from the learned probability distribution. As such, like most reference-based simulators, its ability to generalize and produce realistic outputs depends not only on the number but also the quality and diversity of training examples. Despite that, our previous study revealed that GRouNdGAN can introduce causality and capture properties such as pseudo-time ordering and cellular trajectories on relatively small datasets and excessively large datasets do not substantially improve performance beyond a certain point.

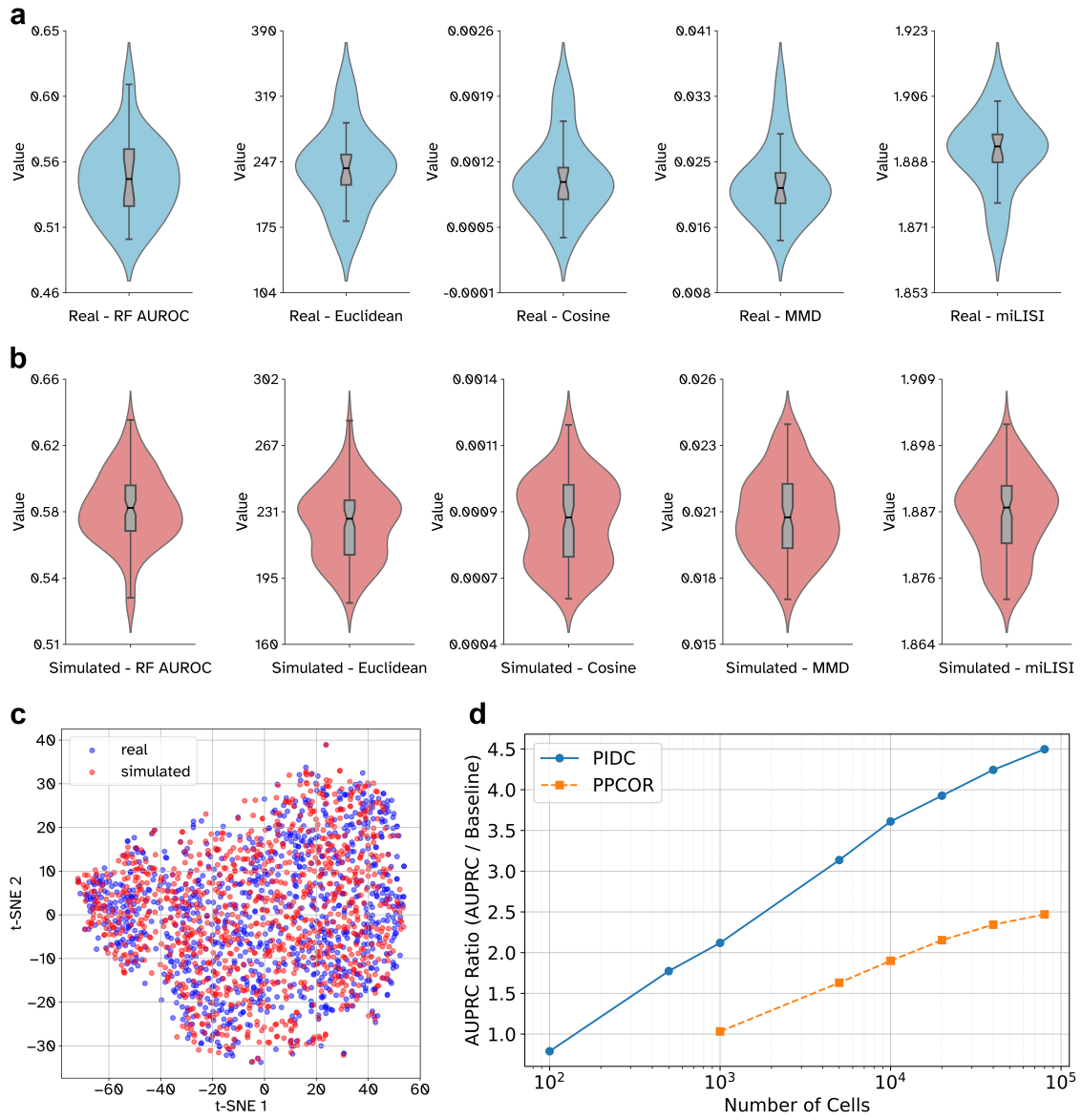


Figure 3: Robustness analysis of simulated data. **a** Violin plots showing five data quality metrics computed between partitions of 19000 reference training experimental cells (1000 cells each) and a held-out test set of 1000 experimental PBMC-CTL cells, serving as real vs real control benchmark. **b** Similar to **a** but computed between 100 distinct simulated datasets (each containing 1000 cells) and the same experimental test set. **c** t-SNE plot of 1000 simulated cells and the same 1000-cell test showing a visual assessment of the distributional similarity between generated and experimental cells. **d** AUPRC ratio (compared against the GRN density) for PIDC and PPCOR in inferring GRNs from simulated GRouNdGAN scRNA-seq data. The x-axis (log scale) represents the number of cells provided. The imposed GRN contains the top 15 regulating TFs for each target gene. Performance improves when increasing the number of provided cells, with PIDC outperforming PPCOR. PIDC takes over 12 hours to run on more than 80000 cells and PPCOR fails to output a GRN when provided with 100 and 500 cells.

Throughout all our analyses, we consistently used the same model architecture and hyperparameter settings, achieving state-of-the-art performance with different choices of GRN across diverse datasets spanning various cell-types, conditions, species, and sequencing technologies.

However, when training GRouNdGAN to a completely new dataset, especially one with a significantly different number of genes or transcription factors, we recommend fine-tuning hyperparameters using a validation set. GRouNdGAN's current architecture incorporates a library-size normalization (LSN) function as the last layer of its causal generator. The LSN layer ensures that all generated cells maintain a consistent fixed library size across selected genes and TFs. Our ablation study[18] confirmed that this layer strengthens GRN imposition and enhances the quality of generated cells. Additionally, our observations indicate that it accelerates model convergence and increases training stability, findings that align with previous GAN-based research in scRNA-seq[24]. While we offer the option to use GRouNdGAN without the LSN layer for flexibility in specific biological analyses requiring unnormalized count data, we advise against this unless necessary as it reduces the quality of the simulated data.

GRouNdGAN's main objective is two-fold: to generate realistic scRNA-seq data that aligns with the reference dataset and to impose causal relationships of a given GRN. If the TF-gene regulatory relationships described by the GRN are overly inconsistent with the patterns in the reference data, the two objectives become contradictory and impossible to optimize simultaneously. Conversely, providing a GRN consistent with the reference dataset can serve as additional knowledge, improving the quality of simulated cells compared to simulators that do not incorporate regulatory information. Hence, we suggest creating the input GRN from the dataset itself or using other modalities pertaining to the same biological system under study.

The model is currently confined to capturing gene regulation through a bipartite GRN (TF to gene). Gene regulation is inherently more complex, involving gene-gene interactions, multi-level regulation, feedback loops, and self-regulation—elements this model doesn't fully capture. However, we should mention that this simplified GRN model is widely used by GRN inference algorithms alike and avoids overcomplicating computations. We plan to relax this limitation as much as possible in future versions of GRouNdGAN.

Materials and Methods

Equipment

Hardware

This protocol was run on a high-performance compute cluster with 2 cores of an AMD EPYC 7413 (@ 2.65 GHz, 128M cache L3) processor, 16GB of RAM, and a single NVidia A100SXM4 (40 GB memory) GPU. Additionally, we conducted tests of the protocol on a desktop workstation equipped with an 11th Gen Intel Core i7-11700 processor (@ 2.50 GHz, 8 cores), 32GB

of RAM, and a GeForce RTX 3070 GPU (24GB memory). At minimum, we recommend a dual-core computer with 16GB of RAM, and a GPU with 24GB of video memory. However, it is still possible to run GRouNdGAN on devices with smaller VRAM than 24GB by making the input GRN sparser or reducing the model size through hyperparameters. An internet connection is required for setting up the compute environment and downloading GRouNdGAN and input data files.

Software

Operating system: Linux, Windows, or MacOS

- Python version 3.9.6
- Docker or Singularity, if running GRouNdGAN via containerized environments.
- GRouNdGAN: a free and open-source implementation maintained on GitHub at `https://github.com/Emad-COMBINE-lab/GRouNdGAN`.
- Basic command-line tools such as `curl`, `tar`, `git`, etc.
- (Optional) Jupyter interactive computing environment for secondary analysis
- (Optional) `pyenv` to install and switch between GRouNdGAN's required Python version and other installed versions, along with the `pyenv-virtualenv` plugin to manage Python virtual environments.
- (Optional) `SCANPY`, `Seurat`, or other single-cell processing pipelines for converting raw single-cell expression data into appropriate input formats.

Configuration file

Each GRouNdGAN command requires providing the path to a configuration file which specifies the protocol's hyperparameters and inputs. This configuration file (with a `.cfg` extension) employs a structure closely similar to that of Microsoft Windows INI files, which can be easily customized by the users. Aside from configurations regarding input and output parameters (discussed in each procedure step), the content of the base configuration file can remain unchanged. GRouNdGAN has consistently well-performed across datasets with the same set of hyperparameters specified in the base configuration file. Although GRouNdGAN is not overly sensitive to the choice of hyperparameters, we still advise to test different hyperparameters on a validation set when training on a new dataset with significantly different dimensions in terms of

number of TFs or target genes. A default base configuration file is available in Supplementary Notes and our code repository (path: `configs/causal_gan.cfg`) but can also be directly downloaded from the command line by running the following command:

```
curl -O https://raw.githubusercontent.com/Emad-COMBINElab/GRouNdGAN/
master/configs/causal_gan.cfg
```

Input data files

The protocol requires two user data inputs: raw single-cell RNA-seq count data and a list of known TFs belonging to the species being studied. GRouNdGAN accepts the scRNA-seq matrix as an AnnData object stored in the standard hierarchical `.h5ad` format. The AnnData object, at a minimum, requires the `X` and `var` fields to contain the expression data matrix and gene identifiers (e.g., Ensembl Gene ID, HGNC (HUGO Gene Nomenclature Committee) Symbol, or NCBI Entrez Gene ID), respectively. Note that the expression matrix needs to be stored as a dense (and not sparse) matrix. Alternatively, GRouNdGAN can accept 10x Genomics-formatted directories that include `matrix.mtx` (gene expression matrix in Matrix Market Exchange format), `genes.tsv` (list of features to be used as gene identifiers as discussed), and `barcodes.tsv` (list of cell barcodes representing unique cells) files. Users can optionally supplement that input with a `.tsv` file for additional cellular annotations and metadata such as cell type and cluster. To match TFs to their expression data, the list of known TFs must use the same identifier type that were chosen to uniquely name genes in the count matrix. Users can decide to omit GRouNdGAN's default causal GRN preparation procedure and instead define a custom causal graph. In that case, a causal graph should be supplied in lieu of the TF list. See Procedure – Step 2 for the causal graph's format.

Equipment setup

Software Installation

Depending on user preferences and requirements, GRouNdGAN can either be locally installed or run as a containerized application. Local installation is recommended for users that need greater control over GRouNdGAN's environment or decide to build new projects upon its framework as a foundation for new bioinformatics work. To reproduce our results or run the protocol using a pre-configured environment where installation is not needed, we suggest using our Docker or Singularity (where Docker is not allowed; e.g, HPC clusters) images, both of which are configured with CUDA support for GPU acceleration. GRouNdGAN's latest version

can be downloaded from our GitHub repository's release page: https://github.com/Emad-COMBINElab/GRouNdGAN/releases/.

Local Installation

- **TIMING ~12 min**

For local installation, python 3.9.6 is required. We recommend isolating GRouNdGAN's environment from the base environment to avoid breaking package dependencies either using python's virtualenv module or pyenv coupled with pyenv-virtualenv to manage multiple python versions. Here, we show how to set up GRouNdGAN with Python's own virtual environment module (venv) and package management system (pip).

1. Start with cloning GRouNdGAN's GitHub repository to a directory of your choosing.

   ```
   $ git clone https://github.com/Emad-COMBINE-lab/GRouNdGAN.git
   ```

   To fully reproduce our primary study[41], one can additionally include specific versions of scGAN, BEELINE[12], scDESIGN2[42], and SPARSIM[26] projects as submodules when cloning the repository.

   ```
   $ git clone –recurse-submodules https://github.com/Emad-COMBINElab/GRouNdGAN.git
   ```

2. Navigate to the cloned directory.

   ```
   $ cd GRouNdGAN
   ```

3. Initialize a new virtual environment for the project (here named `groundgan_env`).

   ```
   $ python -m venv groundgan_env
   ```

   If you chose to use pyenv and pyenv-virtualenv, visit their tutorial and installation guides for your operating system at https://github.com/pyenv/pyenv and https://github.com/pyenv/pyenvvirtualenv.

4. Activate the created virtual environment. Linus or MacOS:

   ```
   $ source groundgan_env/bin/activate
   ```

   Windows:

```
$ groundgan_env
```

5. Project dependencies can now be installed in the created virtual environment through pip from the Python package index (PyPi) using the `requirements.txt` file available at the root of the cloned directory.

```
(groundgan_env)$ pip install -r requirements.txt
```

Using the Docker Image

- **TIMING ~6 min**

GRouNdGAN's docker image is available on the DockerHub at https://hub.docker.com/r/yazdanz/groundgan.

1. Run the following command to pull the latest image from DockerHub:

```
$ docker pull yazdanz/groundgan:GRouNdGAN-toolkit
```

2. To run the Docker container, execute the following:

```
$ docker run –gpus all -it yazdanz/groundgan:latest /bin/bash
```

The `--gpus all` and `–it` flags enable GPU support within the container and start an interactive terminal session respectively.

Building Docker Image from Dockerfile

- **TIMING ~13 min**

We only recommend this option if you wish to further finetune the provided containerized environment to your specific requirements and maintain reproducibility.

1. Clone GRouNdGAN's GitHub repository to a directory of your choosing.

```
$ git clone https://github.com/Emad-COMBINE-lab/GRouNdGAN.git
```

2. Navigate to the project directory.

```
$ cd GRouNdGAN
```

3. Make intended updates and modifications to the Dockerfile (path: `docker/Dockerfile`)

4. Build the Docker image using the provided Dockerfile. The command below will build a docker image with the tag `yourusername/groundgan:custom` which you can change.

```
$ docker build -t yourusername/groundgan:custom -f
docker/Dockerfile .
```

5. Once the image is built, start the docker container interactively and pass GPU devices with the following command. You can also decide to publish it to DockerHub.

```
$ docker run -itd –name yourusername/groundgan:custom –gpus all
groundgan /bin/bash
```

Using Singularity

- **TIMING ~7 min**

1. Pull and convert the container image from Dockerhub into the Singularity Image Format (SIF). The following command creates a singularity image with the `groundgan.sif` filename.

```
$ singularity pull groundgan.sif docker://yazdanz/groundgan
:GRouNdGAN-toolkit
```

2. Start an interactive shell session within the groundgan.sif container. The –nv flag enables NVIDIA GPU hardware support and exposes the host system's drivers and libraries to the container.

```
$ singularity shell –nv groundgan.sif
```

   You can now execute shell commands inside the running container.

Verifying GPU Acceleration

- **TIMING ~<1 s**

You can ensure that your device's GPUs are recognized inside Docker and Singularity containers by running the `nvidia-smi` command. You should see detailed information regarding your GPU and installed drivers. See an example of the expected output below.

```
Tue May 27 16:06:35 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.163.01             Driver Version: 550.163.01     CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA A100-SXM4-40GB          Off |   00000000:C1:00.0 Off |                    0 |
| N/A   29C    P0             55W /  400W |       1MiB /  40960MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Example input data

Example input data comprising of the PBMC[37] and BoneMarrow[43] expression data and list of Homo sapiens and Mus musculus TFs (from AnimalTFDB3.0[44]) can downloaded from our webserver at https://nextcloud.computecanada.ca/index.php/s/pXKQ2isr47AwKEX. Alternatively, one can run the below command in the terminal to download, extract, and copy the raw input data files in the `data/raw` project directory. Make sure to run this command in the GRouNdGAN project directory after having cloned the repository.

```
$ curl https://nextcloud.computecanada.ca/index.php/s/WqrCqkH5zjYYMw9/download –output demo_data.tar &&
tar -xvf demo_data.tar -C data/raw/ &&
mv data/raw/demo/* data/raw &&
rm demo_data.tar &&
rm -rf data/raw/demo/
```

▲ **CRITICAL** Docker and Singularity images come prepackaged with the said example input data files and this step should be skipped.

Procedure

GRouNdGAN can be executed through a command-line interface (CLI) by running the `main.py` file in the `src/` directory via the Python interpreter. Running `main.py` with the `–help` (or the abbreviated `-h`) flag outputs a summary of the tool and a list of required and optional arguments. Commands are listed as optional arguments and the path to the configuration file, which needs to be inputted alongside each command, is a required argument. Each of the seven available commands is a step in the procedure: `–preprocess`, `–create_grn`, `–train`,

–generate, –evaluate, –benchmark_grn, and –perturb. Below is the output of the –help command

```
$ python src/main.py –help
usage: GRouNdGAN [-h] –config CONFIG [–preprocess] [–create_grn] [–train] [–generate]
[–evaluate] [–benchmark_grn] [–perturb]

GRouNdGAN is a gene regulatory network (GRN)-guided causal implicit generative model
for simulating single-cell RNA-seq data, in-silico perturbation experiments, and
benchmarking GRN inference methods. This program also contains cWGAN and unofficial
implementations of scGAN and cscGAN (with projection conditioning)

required arguments:
–config CONFIG Path to the configuration file

optional arguments:
–preprocess Preprocess raw data for GAN training
–create_grn Infer a GRN from preprocessed data using GRNBoost2 and appropriately
format as causal graph
–train Start or resume model training
–generate Simulate single-cells RNA-seq data in-silico
–evaluate Evaluate the data quality of the simulated dataset
–benchmark_grn Evaluate the performance of a GRN inference method in inferring the
ground truth GRN
–perturb Perform a perturbation experiment using a trained GRouNdGAN model
```

It is possible to execute the whole pipeline in one go through a single command by chaining multiple steps of the pipeline, as shown below.

```
$ python src/main.py –config configs/causal_gan.cfg –preprocess –
create_grn –train –generate –evaluate
```

Alternatively, one can run each step separately. For example, the below command only runs the preprocessing step.

```
$ python src/main.py –config configs/causal_gan.cfg –preprocess
```

▲ **CRITICAL** If you decide to chain two or more commands, the supplied configuration file must include required arguments of all invoked commands. We discuss each step of the pipeline and relevant configuration file arguments in the following sections.

▲ **CRITICAL** Avoid chaining commands that are not meant to be run consecutively (e.g., running –preprocess with –generate since this jumps over the –create_GRN and –train commands).

▲ **CRITICAL** Substitute python with python3.9 if you are running GRouNdGAN in a Docker or Singularity container.

Table 2: Arguments relevant to the preprocessing command in the configuration file.

Configuration File Section	Argument	Explanation
`[EXPERIMENT]/[Preprocessing]`	`10x` `[type: boolean]`	"True" if you are supplying 10x-formatted data files (`matrix.mtx`, `genes.tsv`, and `barcodes.tsv`). "False" if you are providing annotated data (anndata) objects in the `.h5ad` format.
`[EXPERIMENT]/[Preprocessing]`	`Raw` `[type: file path/directory]`	If 10x is set to "True", path to the directory containing 10x-formatted data files. Otherwise, path to the raw (unprocessed) `.h5ad` file.
`[EXPERIMENT]/[Preprocessing]`	`validation set size` `[type: integer]`	Number of cells to include in the validation set.
`[EXPERIMENT]/[Preprocessing]`	`test set size` `[type: integer]`	Number of cells to include in the test set.
`[EXPERIMENT]/[Preprocessing]`	`Annotations` `[type: directory, Optional]`	Path to additional annotations (such as cell-type, clusters, etc. for cells/samples).
`[EXPERIMENT]/[Preprocessing]`	`min cells` `[type: integer]`	Filter genes that are expressed in less than the specified "min cells" threshold of cells.
`[EXPERIMENT]/[Preprocessing]`	`min genes` `[type: integer]`	Filter cells with less than the specified threshold of "min genes" expressed.
`[EXPERIMENT]/[Preprocessing]`	`library size` `[type: integer]`	Library size used for library-size normalization.
`[EXPERIMENT]/[Preprocessing]`	`louvain res` `[type: float]`	Louvain clustering resolution (higher resolution means finding more and smaller clusters).
`[EXPERIMENT]/[Preprocessing]`	`highly variable number` `[type: integer]`	Number of highly variable genes to identify and keep for the rest of the pipeline.
`[EXPERIMENT]/[Data]`	`train` `[type: file path]`	Path to a `.h5ad` file to output the training set.
`[EXPERIMENT]/[Data]`	`validation` `[type: file path]`	Path to a `.h5ad` file to output the validation set.
`[EXPERIMENT]/[Data]`	`test` `[type: file path]`	Path to a `.h5ad` file to output the test set.
`[EXPERIMENT]/[Data]`	`number of genes` `[type: integer]`	Number of genes in the dataset.

Step 1. Preprocessing

• TIMING <1 min

Before running the preprocessing command, ensure that the configuration file contains the required arguments to the Preprocessing and Data sections of the configuration file, as outlined in Table 2.

Once the configuration file is ready, run the program with the `–preprocess` flag, supplying the path to the configuration file with the `–config` flag.

```
$ python src/main.py –config configs/causal_gan.cfg –preprocess
```

♦ **TROUBLESHOOTING**

Once completed, you will be shown a success message and the preprocessed train, validation, and test sets will be saved as separate `.h5ad` files in the paths defined under the `[Data]` section of the configuration file.

Example of the expected output:

```
Successfully preprocessed and saved dataset.
Train set: data/processed/PBMC/PBMC68k_train.h5ad
Validation set: data/processed/PBMC/PBMC68k_validation.h5ad
Test set: data/processed/PBMC/PBMC68k_test.h5ad
```

▲ **CRITICAL** Do not replace GRouNdGAN's preprocessing with a custom preprocessing step. GRouNdGAN specifically requires library-size normalized count data.

Step 2. GRN Creation

A) Imposing a GRN Inferred from Data

• TIMING ~8 min for a dataset of around 67000 cells with 4 CPU threads

GRouNdGAN by default uses GRNBoost2[5] to infer a GRN on the preprocessed training set. It then uses the inferred GRN as a scaffold to subsample edges in order to create a causal bipartite directed acyclic graph (DAG). This DAG is then converted and saved as a pickle file in a custom format (see Step 2 - Imposing Custom GRNs) readable to the program. The `––create_grn` command requires the arguments listed in Table 3, in addition to those previously discussed for Step 1 - Preprocessing. GRouNdGAN provides two ways to subsample the GRN. The first uses `[EXPERIMENT]/[GRN Preparation]/strategy = top`, which includes the top k most important TFs of each gene. The second involves generating two separate GRNs, each containing half of these top k TFs per gene. By setting `[EXPERIMENT]/[GRN Preparation]/strategy = pos ctr` and then `[EXPERIMENT]/[GRN Preparation]/strategy = neg ctr`, the TFs are split based on their parity in the list of each gene's ranked list of important TFs. Since this alternative method produces two GRNs of identical densities and relatively similar importances, it is useful for comparative and controlled analysis.

After completing the relevant sections of the configuration file, execute the following command

Table 3: Arguments relevant to the GRN creation command in the configuration file.

Configuration File Section	Argument	Explanation
`[EXPERIMENT]/[GRN Preparation]`	`TFs` `[type: file path]`	Path to the file containing TF names (accepts the AnimalTFDB `.csv` format).
`[EXPERIMENT]/[GRN Preparation]`	`k` `[type: integer]`	The number of most important TFs per gene to include in the GRN.
`[EXPERIMENT]/[GRN Preparation]`	`strategy` `[type: string]`	Defines the procedure used to subsample from the inferred GRN. The following options are available: "top": keep the top most important "k" TFs for every gene. "pos ctr:" keep the TFs with even parity in the list of top "k" TFs per gene (even indices 0, 2, 4... = top 1, 3, 5, ...). "neg ctr": keep the TFs with odd parity in the list of top "k" TFs per gene (odd indices 1, 3, 5... = top 2, 4, 6,...).
`[EXPERIMENT]/[GRN Preparation]`	`Inferred GRN` `[type: file path]`	File path to store the GRN inferred by GRNBoost2 (`.csv` file).
`[EXPERIMENT]/[Data]`	`causal graph` `[type: file path]`	File path to store the GRouNdGAN subsampled GRN (`.pkl` file).

to run this step:

```
$ python src/main.py –config configs/causal_gan.cfg –create_grn
```

♦ TROUBLESHOOTING

Once finished, you will be shown an output detailing the properties of the subsampled GRN and the causal graph will be written in the location specified by `[EXPERIMENT]/[Data]/causal graph` in the configuration file.

Example of the expected output:

```
Using 63 TFs for GRN inference.
preparing dask client
parsing input
creating dask graph
4 partitions
computing dask graph
shutting down client and local cluster
finished
Successfully saved GRN inferred by GRNBoost2 GRN to
data/processed/PBMC/inferred_grnboost2.csv

Causal Graph
_____ ____
TFs 63
Targets 937
Genes 1000
Possible Edges 59031
Imposed Edges 14055
GRN density Edges 0.238095
_____ ____
Successfully saved GRouNdGAN causal graph to
data/processed/PBMC/causal_graph.pkl
```

B) Imposing a Custom GRN

▲ **CRITICAL STEP** Instead of creating causal graphs through the described subsampling process of a GRNBoost2 inferred GRN, it is possible to impose custom GRNs for greater control over exact edges to include and exclude from the causal graph. Nonetheless, we still advise users to impose causal edges that are biologically meaningful to the reference dataset (through evidence found from literature, computational methods, ChiP-seq datasets, etc.).

▲ **CRITICAL** Since GRouNdGAN's two-fold objective is to impose a causal structure while generating realistic data points, drastic inconsistencies between the reference data and the imposed causal graph can deteriorate the quality of simulated cells.

GRouNdGAN reads the causal graph from a pickle file storing a python dictionary. In this dictionary, each key corresponds to a gene index, and each value is a Python set containing the indices of its regulating TFs.

Before saving the dictionary, gene and TF names must be converted to their corresponding indices based on their order in the input dataset (`.var_names` attribute in the Anndata file). For example, a dictionary structured with gene/TF names of the form

```
causal_graph = {
 "G2": {"TF2", "TFn"},
 "G1": {"TF1"},
 "Gn": {"TF2", "TF1"},
 "G3": {"TFn", "TF1"}
}
```

must first be converted into index-based format after resolving each name to its datasetspecific index below:

```
causal_graph = {
 1: {4, 5},
 3: {0},
 6: {4, 0},
 2: {5, 0}
}
```

Finally, the causal graph dictionary can be saved as a pickle file in python as shown below.

```
import pickle

with open("path/to/write/causal_graph.pkl", "wb") as fp:
pickle.dump(causal_graph, fp, protocol=pickle.HIGHEST_PROTOCOL)
```

▲ **CRITICAL** If you're imposing a custom GRN, skip the `–create_grn` command.

▲ **CRITICAL** The GRN must be a directed bipartite graph. It cannot include cycles.

▲ **CRITICAL** The python dictionary must include all TFs and genes present in the preprocessed dataset. Every target gene must appear as a key in the dictionary, and every TF must appear at least once in the list of values associated with those keys.

▲ **CRITICAL** Make sure the `[EXPERIMENT]/[Data]/causal graph]` argument in the configuration file points to the newly-created pickle file.

Step 3. Model Training

• TIMING ~75 h on a single NVidia A100SXM4 GPU

When running training command, three folders will be created in the output directory specified by the configuration file (`[EXPERIMENT]/output directory`). The configuration file will also be copied over into this directory.

Table 4: Arguments relevant to the training command in the configuration file.

Configuration File Section	Argument	Explanation
`[EXPERIMENT]/[Training]/[Logging]`	`summary frequency [type: integer]`	Specifies the frequency at which TensorBoard logs are generated for the GRouNdGAN model.
`[EXPERIMENT]/[Training]/[Logging]`	`plot frequency [type: integer]`	Specifies the frequency at which t-SNE plots are generated for the GRouNdGAN model.
`[EXPERIMENT]/[Training]/[Logging]`	`save frequency [type: integer]`	Specifies the frequency at which checkpoints are saved for the GRouNdGAN model.
`[EXPERIMENT]/[CC Training]/[Logging]`	`summary frequency [type: integer]`	Specifies the frequency at which TensorBoard logs are generated for the causal controller.
`[EXPERIMENT]/[CC Training]/[Logging]`	`plot frequency [type: integer]`	Specifies the frequency at which t-SNE plots are generated for the causal controller.
`[EXPERIMENT]/[CC Training]/[Logging]`	`save frequency [type: integer]`	Specifies the frequency at which checkpoints are saved for the causal controller.

- `checkpoints/`: contains the `.pth` state dictionaries, including model's weights, biases, and other parameters.
- `TensorBoard/`: contains stored TensorBoard logs (.tfevent files) for visualizing the model's training progress and neural graph.
- `TSNE/`: contains t-SNE plots of reference and simulated data.

The frequency at which these directories are updated can be configured through parameters specified in the configuration file, as detailed in Table 4.

To begin training, use the `–train` flag and along with the path to the configuration file as shown below.

```
$ python src/main.py –config configs/causal_gan.cfg –train
```

♦ TROUBLESHOOTING

Example of the expected output:

```
Done training causal controller step 0
Done training causal controller step 1
Done training causal controller step 2
...
Done training causal controller step 10000
Saved logs
Saved t-SNE plot
Done training causal controller step 10001
...
Done training causal controller step 200000
Saved logs
Saved t-SNE plot
Saved checkpoint
Done training GRouNdGAN step 0
Done training GRouNdGAN step 1
Done training GRouNdGAN step 2
...
Done training GRouNdGAN step 10000
Saved logs
Saved t-SNE plot
Done training GRouNdGAN step 10001
...
Done training GRouNdGAN step 100000
Saved logs
Saved t-SNE plot
Saved checkpoint
...
Done training GRouNdGAN step 1000000
Saved logs
Saved t-SNE plot
Saved checkpoint
Finished training
```

▲ **CRITICAL** While GRouNdGAN supports muti-GPU training, we recommend using a single GPU to minimize overhead. By default, GRouNdGAN trains for 1000000 steps. Changing this setting in the configuration file is not recommended.

▲ **CRITICAL** GRouNdGAN also provides two slurm scripts for training and monitoring on HPC clusters in the `scripts/` directory of its code repository.

□ **PAUSE POINT** Users can stop and resume training from a checkpoint by setting `[EXPERIMENT]/checkpoint` in the configuration file to the latest saved checkpoint (`.pth` file in `[EXPERIMENT]/output directory/checkpoints/`) and rerunning the `–train` command.

Training Monitoring using TensorBoard

TensorBoard is a visualization tool that GRouNdGAN uses to provide its users with an intuitive monitoring and visualization platform. It keeps track of several key scalar values such as generator and critic losses, learning rates, and gradient penalty (Figure 4-b). At regular intervals, it also logs t-SNE plots comparing cells from the validation set with an equal number of simulated cells (Figure 4-a). Furthermore, TensorBoard provides a detailed view of GRouNdGAN's architecture by rendering the computational graphs of the Wasserstein critic and generator (comprised of causal controller and target generators) at varying levels of complexity as shown in Figure 4-c,d.

GRouNdGAN saves TensorBoard event files in `[EXPERIMENT]/output directory_CC/Tensorboard` for the causal controller training and `[EXPERIMENT]/output directory/Tensorboard` for the second training step.

Run the following command to start TensorBoard.

```
tensorboard –logdir="Tensorboard Event Directory " –host 0.0.0.0
–load_fast false
```

▲ **CRITICAL** Users can replace the `Tensorboard Event Directory` in the above command with a parent directory to compare different training runs of the model.

Once Tensorboard is running, it will output a local URL (by default: `http://localhost:6006/`) that can be opened in a web browser to access the TensorBoard dashboard. In the case you are running GRouNdGAN on a remote server, you might need to set up port forwarding to access TensorBoard on your local machine. The command below forwards the remote machine's port 6006 (where TensorBoard is running) to port 6007 on your local machine.

```
ssh -N -f -L localhost:6007:computenode:6006
userid@remote_machine_ip
```

▲ **CRITICAL** The above command might need to be modified depending on your remote server setup, network configuration, and port availability. Ensure that the port on which TensorBoard is running is not blocked by a firewall on the remote server. Some servers might be configured to restrict port forwarding.

Step 4. Simulation

- **TIMING <30 s**

Once training is finished, set the `[EXPERIMENT]/checkpoint` path in the configuration file to the desired model checkpoint (`.pth` file in `[EXPERIMENT]/output directory/checkpoints/`).

Figure 4: TensorBoard visualizations during GRouNdGAN training. a The "images" tab displays a t-SNE plot of reference and generated cells. **b** The "scalars tab" displays key training metrics over time, including gradient penalty, learning rate, and losses for both the critic and generator. **c** and **d** The "graphs" tab visualise the computational graph for the critic and generator networks of GRouNdGAN, showing the model architecture and the flow of gradients during backpropagation.

In most cases, you would want to use the latest checkpoint corresponding to the greatest number of steps. Furthermore, you can control the generation path and number of cells to generate through the configuration file (refer to Table 5).

Running the following command will save the generated cells as a `.h5ad` file (`simulated.h5ad`).

Table 5: Arguments relevant to the generation command in the configuration file.

Configuration File Section	Argument	Explanation
`[EXPERIMENT]/checkpoint`	`model checkpoint [type: file path]`	Path to the checkpoint from which to load the trained model.
`[EXPERIMENT]/[generation]`	`number of cells to generate [type: integer]`	Number of cells to simulate.
`[EXPERIMENT]/[generation]`	`generation path [type: directory, Optional]`	Path where to save the simulated dataset (`simulated.h5ad`). If left undefined, `simulated.h5ad` will be saved to `[Experiment]/output directory/`.

```
$ python src/main.py –config configs/causal_gan.cfg ––generate
```

Example of the expected output:

```
Simulated cells saved to results/GRouNdGAN/simulated.h5ad
```

□ **PAUSE POINT** Remaining steps are optional. Users can stop here and use the saved simulated dataset for benchmarking, data augmentation, and other external analyses.

Step 5. Assessing simulated data quality (Optional)

• TIMING <30 s

This step provides a simple command for assessing the quality of simulated scRNA-seq data compared to its reference experimental dataset using the following qualitative and quantitative metrics:

- t-SNE plot visualization of jointly embedded experimental and simulated cells. Greater overlap indicates better alignment.
- Euclidean distance between the mean expression profiles of real and simulated cells. A lower value indicates higher similarity.
- Cosine distance between the mean expression profiles of real and simulated cells. A lower value indicates higher directional similarity and better alignment.
- AUROC of a random forest classifier trained to distinguish simulated cells from experimental cells. An AUROC closer to 0.5 indicates that the classifier fails to tell the two classes of cells apart, implying more realistic simulation.

Table 6: Arguments relevant to the evaluation command in the configuration file.

Configuration File Section	Argument	Explanation
`[EXPERIMENT]/[Evaluation]`	`simulated data path [type: directory, Optional]`	Path to the simulated data you wish to assess. Defaults to `[EXPERIMENT]/[Generation]/generation path` if left undefined.
`[EXPERIMENT]/[Evaluation]`	`plot tsne [type: boolean]`	Whether to plot the t-SNE plot.
`[EXPERIMENT]/[Evaluation]`	`compute euclidean distance [type: boolean]`	Whether to compute the Euclidean distance metric.
`[EXPERIMENT]/[Evaluation]`	`compute cosine distance [type: boolean]`	Whether to compute the Cosine distance metric.
`[EXPERIMENT]/[Evaluation]`	`compute rf auroc [type: boolean]`	Whether to compute the RF AUROC metric and plot the ROC curve.
`[EXPERIMENT]/[Evaluation]`	`compute MMD [type: boolean]`	Whether to compute the MMD metric.
`[EXPERIMENT]/[Evaluation]`	`compute miLISI [type: boolean]`	Whether to compute the miLISI metric.

- Maximum Mean Discrepancy (MMD) to quantify distributional similarity of experimental and simulated cells. A lower value is desired.
- Mean integration local inverse Simpson's index (miLISI) to measure local mixing of real and simulated cells in the t-SNE space. A value closer to 2 indicates good integration and mixing.

All metrics are computed by comparing the test set to an equal number of cells sampled from the simulated data. As a control, we also compute the Euclidean distance, Cosine distance, MMD, and miLISI between two halves of the reference test set to establish a baseline for comparison. You can specify which evaluation metrics to compute by setting the corresponding flags in the configuration file (see Table 6). Additionally, you can use GRouNdGAN's evaluation framework to assess simulated datasets generated by external simulators by updating `[EXPERIMENT]/ [Evaluation]/simulated data path` in the configuration file to point to the desired dataset.

To run the evaluation pipeline with the specified configuration, execute the following command.

```
$ python src/main.py –config configs/causal_gan.cfg –evaluate
```

Output plots (following the format of Figure 5-a,b) will be saved to [EXPERIMENT]/output directory/

Example of the expected output:

```
t-SNE plot saved to results/GRouNdGAN/tSNE.png

Euclidean distance (real vs fake): 171.218017578125
Euclidean distance (control): 242.84738159179688

Cosine distance (real vs fake): 0.0003756880760192871
Cosine distance (control): 0.000784814357757568 4

RF ROC plot saved to results/GRouNdGAN/RF.png
RF AUROC: 0.5211342726343393

MMD (real vs fake): 0.03008744777663175
MMD (control): 0.06244054076970418

miLISI (real vs fake): 1.89692891595357
miLISI (control): 1.8900128524031679
```

Step 6. GRN inference benchmarking (Optional)

• TIMING <20 s

One of GRouNdGAN's main goals is to provide tools and ground truth for benchmarking GRN inference methods. This command evaluates how faithful GRNs inferred on GRouNdGAN-generated data are to the one imposed onto the model and used for simulation. To run this command, you must provide the path to the inferred GRN through the configuration file using the `[EXPERIMENT]/[GRN Benchmarking]/grn to benchmark` field (refer to Table 7). The GRN should be a tab-separated `.csv` file with three columns in the following order:

1. TF
2. Gene
3. Importance

Headers are optional; only the column order matters.

Example format:

```
Gata1 Tbb-bs 0.89
Tal1 Tbb-bs 0.74
Gata2 Gfi1b 0.52
```

Table 7: Arguments relevant to the benchmarking command in the configuration file.

Configuration File Section	Argument	Explanation
`[EXPERIMENT]/[GRN Benchmarking]`	`grn to benchmark` `[type: file path]`	Path to the inferred GRN you wish to benchmark.
`[EXPERIMENT]/[GRN Benchmarking]`	`ground truth save path` `[type: directory, Optional]`	If specified, the ground truth GRN (GRouNdGAN's causal graph) will be exported to the provided path.
`[EXPERIMENT]/[GRN Benchmarking]`	`plots save path` `[type: path]`	Folder in which to save generated plots.
`[EXPERIMENT]/[GRN Benchmarking]`	`compute precision at k` `[type: boolean]`	Whether to compute precision at k.
`[EXPERIMENT]/[GRN Benchmarking]`	`k` `[type: integer]`	How many of the top k regulators to consider when computing precision at k.
`[EXPERIMENT]/[GRN Benchmarking]`	`compute pr` `[type: boolean]`	Whether to plot the PR curve and compute AUPRC of GRN recovery.

When enabled in the configuration file, the GRN benchmarking command generates one or both of the following plots, which are saved to the directory specified in `[EXPERIMENT]/[GRN Benchmarking]/plots save path`:

- Precision at k: Evaluates the proportion of the top-k inferred TFs for each gene are true regulators. The value of k is configurable.
- Precision-Recall (PR) curve: Computed across all predicted edges in the inferred GRN.

Run this step using the command below.

```
$ python src/main.py –config configs/causal_gan.cfg –benchmark_grn
```

Both plots (see Figure 5-c, d) are saved as `.png` files, and relevant evaluation metrics such as precision at k scores and the area under the precision recall curve (AUPRC) are printed to the console.

Example of the expected output:

Table 8: Arguments relevant to the perturbation command in the configuration file.

Configuration File Section	Argument	Explanation
`[EXPERIMENT]/[Perturbation]`	`save dir [type: directory]`	Directory in which to save cells before and after perturbation, as well as generated plot.
`[EXPERIMENT]/[Perturbation]`	`tfs to perturb [type: list[string]]`	List of space-separated TF names to perturb. Example: `IRF8 CEBPA`.
`[EXPERIMENT]/[Perturbation]`	`perturbation values [type: list[float]]`	List of space-separated float values.

```
Saved ground truth GRN to data/generated/ground truth GRN.csv

Inferred GRN precision at k: [1.0, 0.9983425414364641,
0.9918968692449356, 0.9820441988950276, 0.9703867403314917]
Baseline precision at k: [0.010526315789473684, 0.021052631578947368,
0.031578947368421054, 0.042105263157894736, 0.05263157894736842]
Saved precision at k plot at results/GRouNdGAN/Top_TF_precision.png

Inferred GRN AURPC: 0.71
Baseline AUPRC (random predictor): 0.15789473684210525
Saved PR curve at results/GRouNdGAN/PR_curve.png
```

□ **PAUSE POINT** The ground truth GRN can be exported as a `.csv` file to be used for additional analysis or use in external benchmarking tools. Simply specify `[EXPERIMENT]/[GRN Benchmarking]/ground truth save path` in the configuration file before running the command.

Step 7. Perturbation prediction (Optional)

• TIMING <1 min

Once trained, GRouNdGAN can sample from interventional distributions and perform in silico perturbation experiments at inference time. It enables the creation of matched case/control comparison as perturbations are applied to the same batch of cells (batch size of 1024 by default). To simulate a perturbation, provide a list of TFs to perturb using `[EXPERIMENT]/[Perturbation]/tfs to perturb`, along with the corresponding target values in `[EXPERIMENT]/[Perturbation]/perturbation values`. Each TF is matched to a value by its position in the list, with entries separated by spaces (as shown in Table 8).

▲ **CRITICAL** Ensure all TFs specified in `[EXPERIMENT]/[Perturbation]/tfs to perturb` exist in the causal graph and that their count matches the number of entries in `[EXPERIMENT]/[Perturbation]/perturbation`.

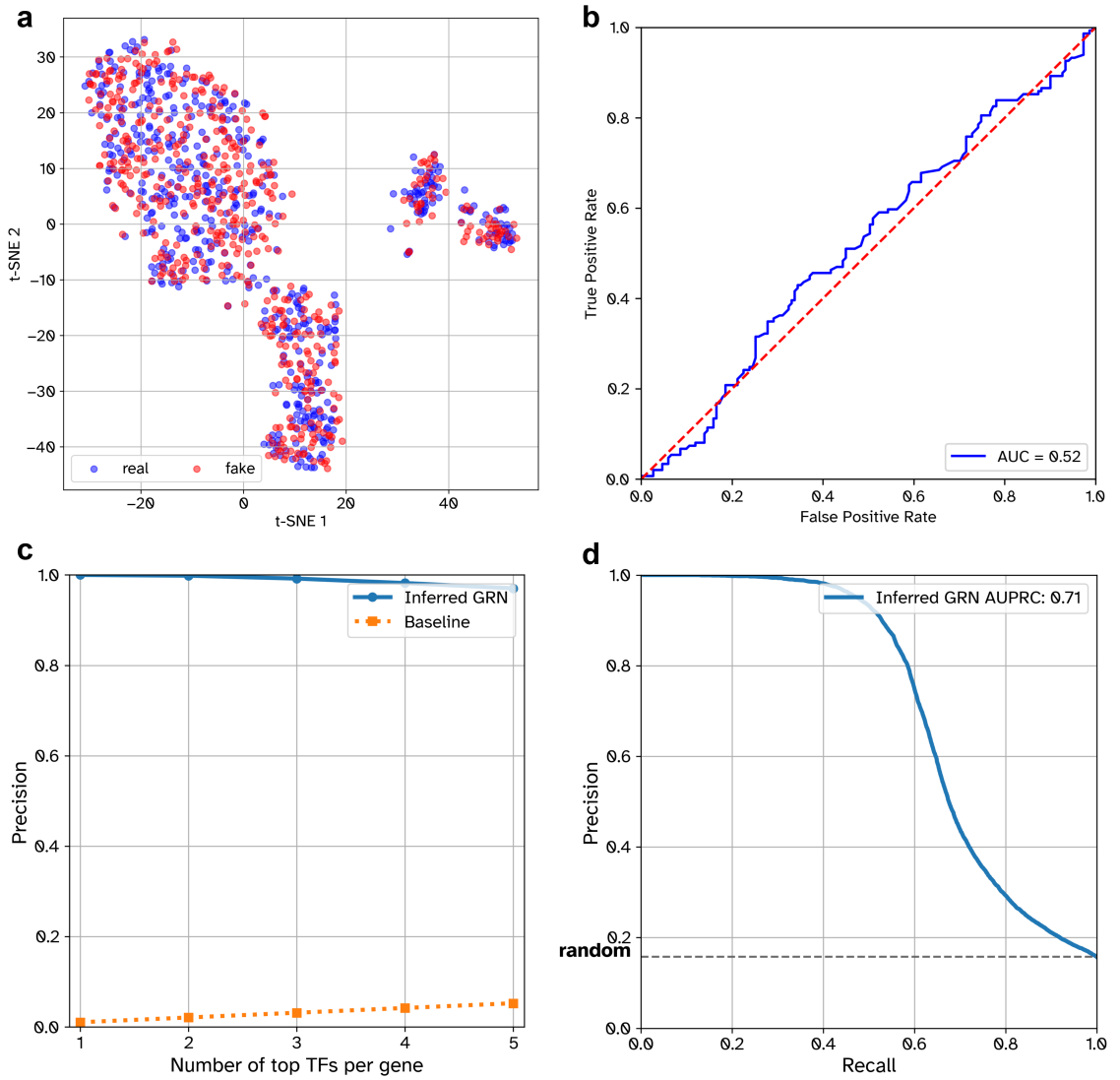


Figure 5: Example of expected plot outputs saved by the evaluation and benchmarking commands. **a** t-SNE plot of simulated and test set experimental cells. **b** Receiver Operating Characteristic (ROC) curve of a random forest classifier distinguishing between simulated cells and test set experimental cells. **c** Precision at k plot showing the proportion of true regulators among the top-k predicted transcription factors for each gene. **d** Precision-Recall (PR) curve illustrating global GRN reconstruction performance. The baseline corresponds to the performance of a random predictor which also corresponds to the density of the ground truth GRN.

Once the experiment parameters are defined, you can run the perturbation experiment with the following command.

```
$ python src/main.py –config configs/causal_gan.cfg –perturb
```

♦ TROUBLESHOOTING

The cell states before and after perturbation will be saved as `before_perturbation.h5ad` and `after_perturbation.h5ad`, respectively, in the specified output directory. There is a

one-to-one correspondence between cells in the two simulated datasets (row and column indices match). These files can be used for downstream or secondary analysis. In addition to the data files, a series of plots, similar to Figure 6, will be generated and saved to visualize the effects of the perturbation.

Example of the expected output:

```
Loaded GAN
Using checkpoint at results/GRouNdGAN/checkpoints/step_1000000.pth
Saved cells before and after perturbation to data/generated/
```

Timing

The reported execution times correspond to running GRouNdGAN on the PBMC dataset, which comprises 67579 cells, using the hardware configuration detailed in the Equipment section. The duration of step 1 (Preprocessing) and step 2 (GRN creation) primarily scale with the dataset size. In contrast, the execution of step 3 (Model training) is largely dependent on the density of the imposed GRN and GPU specification. The remaining steps exhibit comparatively lower sensitivity to these factors. Installation times are predominantly affected by download speed and server loads.

Software installation - local: ~12 min
Software installation - running through the Docker image: ~6 min
Software installation - building Docker image from Dockerfile: ~13 min
Software installation - using Singularity: ~7 min

Procedure - Step 1, Preprocessing: <1 min
Procedure - Step 2, GRN creation: ~8 min
Procedure - Step 3, Model training: ~75 h
Procedure - Step 4, Simulation: <30 s
Procedure - Step 5, Data quality evaluation: <30 s
Procedure - Step 6, GRN inference benchmarking: <20 s
Procedure - Step 7: Perturbation prediction: <1 min
Procedure - End to end (Steps 1-7): ~75 h, 12 min

General notes and troubleshooting

Table 9 contains common issues and how to troubleshoot them.

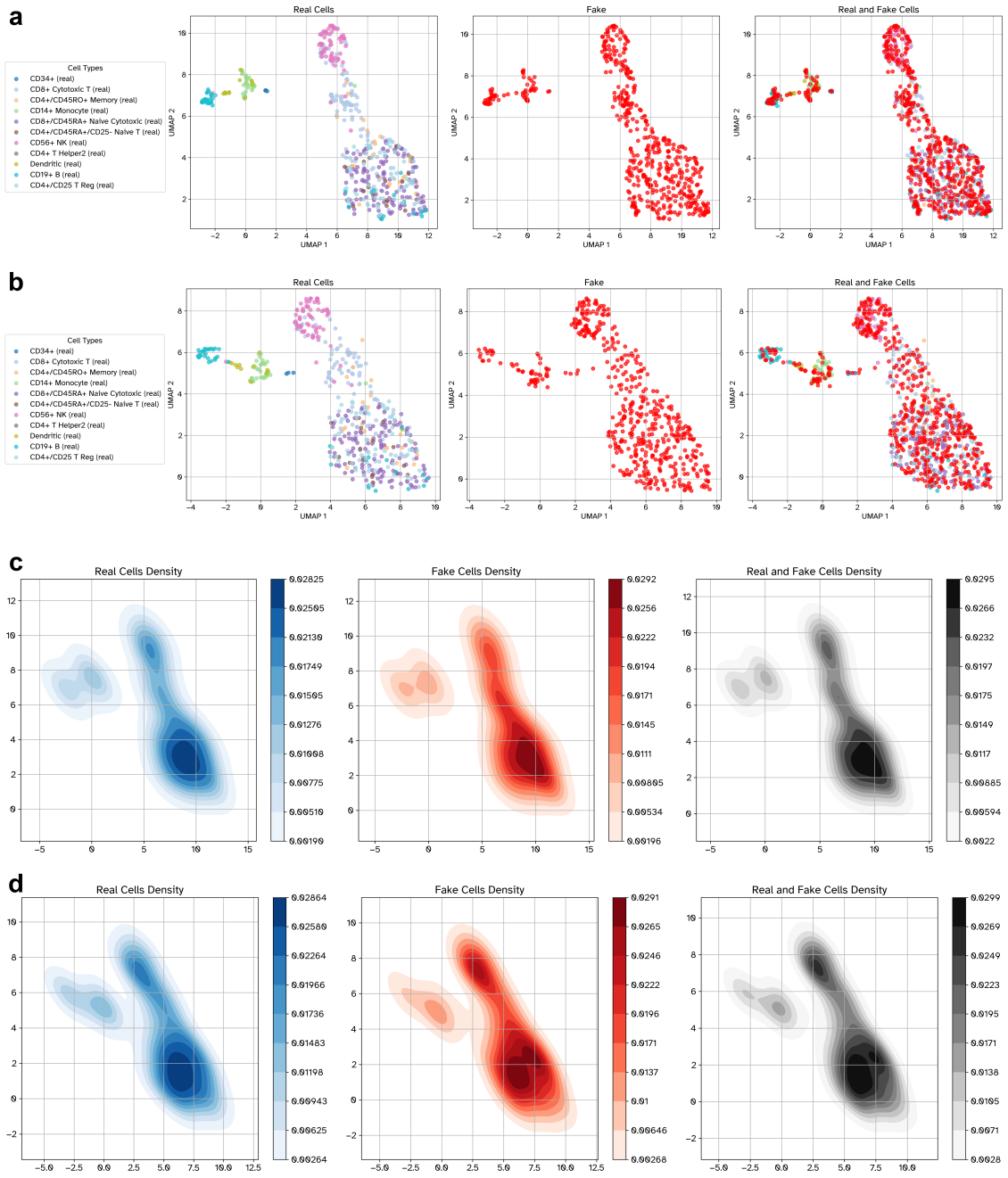


Figure 6: Example of expected plot outputs saved by the perturbation command. **a** UMAP embedding of experimental cells (left), unperturbed simulated cells (middle), and overlayed on top of each other (right). **b** similar to a but simulated cells were generated under perturbation. **c** Cell density in the UMAP embedding for experimental cells (left), unperturbed simulated cells (middle), and together (right). **d** similar to **c** but simulated cells are perturbed.

Expected results

Running GRouNdGAN's GRN-guided simulation procedure (Steps 1 to 4) produces a single final output: a `simulated.h5ad` file containing simulated scRNA-seq data. The file preserves the original gene annotations from the input data and assigns sequential labels to generated

Table 9: Troubleshooting table.

Step	Problem	Possible reason	Solution
1, 2	Error message: `check_values_indices_shape_match`	Mismatch between the number of genes in `train`, `validation`, or `test` sets and configuration parameters or `GRN`.	Verify `[EXPERIMENT]/[Preprocessing]/highly variable number` and `[EXPERIMENT]/[Data]/number of genes` parameters in the configuration file against the input dataset.
		The `.X` attribute in the `anndata` input is a sparse expression matrix.	If the input expression matrix is sparse, convert it into a dense matrix using `adata.X.toarray()`.
3	Error message: `torch.cuda.OutOfMemoryError: CUDA out of memory`	Insufficient video memory on `GPU`.	Switching to a `GPU` with a larger `VRAM`.
			Making the input `GRN` sparser. If the `–create_grn` command is used to construct the input `GRN`, making the hyperparameter `[GRN Preparation]/k` in the configuration file smaller will result in a sparser `GRN`. Reducing `GRN` size might result in a lower quality simulation.
			Reducing the `[EXPERIMENT]/[Model]/depth per gene` and `[EXPERIMENT]/[Model]/width per gene]` hyperparameters in the configuration file to 2 and 1, respectively. This will make `GRouNdGAN's` neural network smaller to fit in a smaller `VRAM`. This option might result in a lower quality simulation.
3	Error message: `what(): mat1 and mat2 shapes cannot be multiplied`	The specified number of genes is inconsistent with the `GRN`.	Ensure that the number of nodes in the `GRN` correspond to the `[EXPERIMENT]/[Data]/number of genes` parameter in the configuration file.
7	Error message: `ValueError: 'TF' is not in list`	The `TF` targeted for perturbation does not exist in the `GRN`.	Ensure that all the `TFs` listed in the `[EXPERIMENT]/[Perturbation]/tfs to perturb` parameter of the configuration file exist in the dataset and are regulators (not targets) in the `GRN`.

cells (e.g., Cell1, Cell2, ...). The simulated expression matrix is stored in the `.X` attribute, where rows correspond to cells and columns to genes. Furthermore, `.var` and `.obs` store gene and cell metadata, respectively. This structured output format ensures compatibility with downstream python and R single-cell analysis workflows. A collection of such output datasets paired with the GRN used for simulation can be found on GRouNdGAN's website at https://emad-combinelab.github.io/GRouNdGAN/benchmarking.html.

The perturbation prediction procedure generates two .h5ad files: `before_perturbation.h5ad` and `after_perturbation.h5ad`. They contain expression matrices (in the `.X` attribute) of identical dimensions, representing cellular states before and after perturbation, respectively. Both include the same set of genes and annotations. Additionally, cells are matched one-to-one between the two files, meaning that each cell (e.g., "Cell 1") in `before_perturbation.h5ad` corresponds directly to its perturbed counterpart in `after_perturbation.h5ad`.

Given a reasonably sized training set and a sufficiently representative GRN, we expect the simulated output data to be realistic (i.e., mimicking experimental training data) and true to the imposed GRN. Under the same conditions, we also expect that the perturbation results will align with biological expectations, though perturbation prediction relies more heavily on the

quality of the input GRN. We provide guidelines for evaluating the quality of the output data using five metrics (see the Procedure – Step 5) and comparing it to a control baseline, where the control is derived from the data itself. Furthermore, it is expected that the generator and critic losses, which can be monitored using TensorBoard (see Procedure – Step3), will decrease as the model undergoes training, showing better performance over time.

Supplementary Information

The online version contains supplementary material available in Supplementary File S1.

Authors' contributions

All authors contributed to the design of the protocol, data analyses, and writing the manuscript. A.E. supervised the study. Y.Z. implemented the protocol, conducted computational analyses, developed the website, and managed releases across various formats.

Funding

This work was supported by a Discovery grant from Natural Sciences and Engineering Research Council of Canada (NSERC) [RGPIN-2019-04460] (AE), a NOVA grant funded by NSERC and Fonds de recherche du Québec – Nature et technologies (FRQNT) [2024-NOVA-344140], and by a grant from Canada Foundation for Innovation (CFI) JELF [project 40781] (AE). This work was enabled in part by support provided by Calcul Québec (www.calculquebec.ca) and the Digital Research Alliance of Canada (alliancecan.ca)

Data Availability

The dataset used to demonstrate the protocol is publicly available. The PBMC-ALL scRNA-seq dataset (healthy donor A) can be downloaded through the 10x Genomics website at https://support.10xgenomics.com/single-cell-geneexpression/datasets/1.1.0/fresh_68k_pbmc_donor_a?. The GRouNdGAN-simulated PBMC-CTL dataset (100k cells by 1000 genes ,including TFs), along with the imposed ground truth GRN, is available on GRouNdGAN's website at https://emad-combinelab.github.io/GRouNdGAN/benchmarking.html#pbmc-ctl.

Code Availability

An implementation of GRouNdGAN is freely available under the terms of the GNU Affero General Public License v3.0 on GitHub at https://github.com/Emad-COMBINE-lab/GRouNdGAN. The software is written in Python 3.9.6 using the PyTorch deep learning framework (v. 1.13.1). To ensure ease of use and reproducibility, a pre-build container image is also available on DockerHub (https://hub.docker.com/r/yazdanz/groundgan) and an archived version can be found on Zenodo (https://doi.org/10.5281/zenodo.11068245). Visit the GRouNdGAN website (https://emad-combine-lab.github.io/GRouNdGAN) to explore an expanding collection of readily available simulated datasets for benchmarking GRN inference algorithms, along with the latest updates on running the protocol.

Competing interests

The authors have no conflict of interest to declare.

References

1. Pardee AB: **Regulatory molecular biology.** *Cell Cycle* 2006, **5**(8):846-852.

2. Jacob F, Monod J: **Genetic regulatory mechanisms in the synthesis of proteins.** *Journal of molecular biology* 1961, **3**(3):318-356.

3. Afonja T, Sheth I, Binkyte R, Hanif W, Ulas T, Becker M, Fritz M: **LLM4GRN: Discovering Causal Gene Regulatory Networks with LLMs–Evaluation through Synthetic Data Generation.** *arXiv preprint arXiv:241015828* 2024.

4. Emad A, Sinha S: **Inference of phenotype-relevant transcriptional regulatory networks elucidates cancer type-specific regulatory mechanisms in a pan-cancer study.** *NPJ systems biology and applications* 2021, **7**(1):9.

5. Moerman T, Aibar Santos S, Bravo González-Blas C, Simm J, Moreau Y, Aerts J, Aerts S: **GRNBoost2 and Arboreto: efficient and scalable inference of gene regulatory networks.** *Bioinformatics* 2019, **35**(12):2159-2161.

6. Chan TE, Stumpf MP, Babtie AC: **Gene regulatory network inference from single-cell data using multivariate information measures.** *Cell systems* 2017, **5**(3):251-267. e253.

7. Kim S: **ppcor: an R package for a fast calculation to semi-partial correlation coefficients.** *Communications for statistical applications and methods* 2015, **22**(6):665.

8. Su C, Pastor WA, Emad A: **Deciphering lineage-relevant gene regulatory networks during endoderm formation by InPheRNo-ChIP.** *Briefings in Bioinformatics* 2024, **25**(6):bbae592.

9. Kamimoto K, Stringa B, Hoffmann CM, Jindal K, Solnica-Krezel L, Morris SA: **Dissecting cell identity via network inference and in silico gene perturbation.** *Nature* 2023, **614**(7949):742-751.

10. Bravo González-Blas C, De Winter S, Hulselmans G, Hecker N, Matetovici I, Christiaens V, Poovathingal S, Wouters J, Aibar S, Aerts S: **SCENIC+: single-cell multiomic inference of enhancers and gene regulatory networks.** *Nature Methods* 2023:1-13.

11. Emmert-Streib F, Dehmer M, Haibe-Kains B: **Gene regulatory networks and their applications: understanding biological and medical problems in terms of networks.** *Frontiers in cell and developmental biology* 2014, **2**:38.

12. Pratapa A, Jalihal AP, Law JN, Bharadwaj A, Murali T: **Benchmarking algorithms for gene regulatory network inference from single-cell transcriptomic data.** *Nature methods* 2020, **17**(2):147-154.

13. McCall MN: **Estimation of gene regulatory networks.** *Postdoc journal: a journal of postdoctoral research and postdoctoral affairs* 2013, **1**(1):60.

14. Erbe R, Gore J, Gemmill K, Gaykalova DA, Fertig EJ: **The use of machine learning to discover regulatory networks controlling biological systems.** *Molecular cell* 2022, **82**(2):260-273.

15. Dibaeinia P, Sinha S: **SERGIO: a single-cell expression simulator guided by gene regulatory networks.** *Cell systems* 2020, **11**(3):252-271. e211.

16. Li H, Zhang Z, Squires M, Chen X, Zhang X: **scMultiSim: simulation of multimodality single cell data guided by cell-cell interactions and gene regulatory networks.** *Research Square* 2023:rs. 3. rs-2675530.

17. Schaffter T, Marbach D, Floreano D: **GeneNetWeaver: in silico benchmark generation and performance profiling of network inference methods.** *Bioinformatics* 2011, **27**(16):2263-2270.

18. Zinati Y, Takiddeen A, Emad A: **GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks.** *Nature Communications* 2024, **15**(1):4055.

19. Marouf M, Machart P, Bansal V, Kilian C, Magruder DS, Krebs CF, Bonn S: **Realistic in silico generation and augmentation of single-cell RNA-seq data using generative adversarial networks.** *Nature communications* 2020, **11**(1):166.

20. Kocaoglu M, Snyder C, Dimakis AG, Vishwanath S: **Causalgan: Learning causal implicit generative models with adversarial training.** *arXiv preprint arXiv:170902023* 2017.

21. Arjovsky M, Chintala S, Bottou L: **Wasserstein generative adversarial networks.** In: *International conference on machine learning: 2017.* PMLR: 214-223.

22. Gulrajani I, Ahmed F, Arjovsky M, Dumoulin V, Courville AC: **Improved training of wasserstein gans.** *Advances in neural information processing systems* 2017, **30**.

23. Zandigohar M, Rehman J, Dai Y: **scRegulate: Single-Cell Regulatory-Embedded Variational Inference of Transcription Factor Activity from Gene Expression.** *bioRxiv* 2025:2025.2004.2017.649372.

24. Marouf M, Machart P, Bansal V, Kilian C, Magruder DS, Krebs CF, Bonn S: **Realistic in silico generation and augmentation of single-cell RNA-seq data using generative adversarial networks.** *Nature communications* 2020, **11**(1):1-12.

25. Xu Y, Zhang Z, You L, Liu J, Fan Z, Zhou X: **scIGANs: single-cell RNA-seq imputation using generative adversarial networks.** *Nucleic acids research* 2020, **48**(15):e85-e85.

26. Baruzzo G, Patuzzi I, Di Camillo B: **SPARSim single cell: a count data simulator for scRNA-seq data.** *Bioinformatics* 2020, **36**(5):1468-1475.

27. Song D, Wang Q, Yan G, Liu T, Sun T, Li JJ: **scDesign3 generates realistic in silico data for multimodal single-cell and spatial omics.** *Nature Biotechnology* 2024, **42**(2):247-252.

28. Zappia L, Phipson B, Oshlack A: **Splatter: simulation of single-cell RNA sequencing data.** *Genome biology* 2017, **18**(1):174.

29. Zhang X, Xu C, Yosef N: **Simulating multiple faceted variability in single cell RNA sequencing.** *Nature communications* 2019, **10**(1):2611.

30. Marbach D, Prill RJ, Schaffter T, Mattiussi C, Floreano D, Stolovitzky G: **Revealing strengths and weaknesses of methods for gene network inference.** *Proceedings of the national academy of sciences* 2010, **107**(14):6286-6291.

31. Marbach D, Costello JC, Küffner R, Vega NM, Prill RJ, Camacho DM, Allison KR, Kellis M, Collins JJ: **Wisdom of crowds for robust gene network inference.** *Nature methods* 2012, **9**(8):796-804.

32. Kanehisa M, Goto S: **KEGG: kyoto encyclopedia of genes and genomes.** *Nucleic acids research* 2000, **28**(1):27-30.

33. Brown KR, Jurisica I: **Unequal evolutionary conservation of human protein interactions in interologous networks.** *Genome biology* 2007, **8**:1-11.

34. Kotlyar M, Pastrello C, Sheahan N, Jurisica I: **Integrated interactions database: tissue-specific view of the human and model organism interactomes.** *Nucleic acids research* 2016, **44**(D1):D536-D541.

35. Xu H, Baroukh C, Dannenfelser R, Chen EY, Tan CM, Kou Y, Kim YE, Lemischka IR, Ma'ayan A: **ESCAPE: database for integrating high-content published data collected from human and mouse embryonic stem cells.** *Database* 2013, **2013**:bat045.

36. Kolmykov S, Yevshin I, Kulyashov M, Sharipov R, Kondrakhin Y, Makeev VJ, Kulakovskiy IV, Kel A, Kolpakov F: **GTRD: an integrated view of transcription regulation.** *Nucleic acids research* 2021, **49**(D1):D104-D111.

37. Zheng GX, Terry JM, Belgrader P, Ryvkin P, Bent ZW, Wilson R, Ziraldo SB, Wheeler TD, McDermott GP, Zhu J: **Massively parallel digital transcriptional profiling of single cells.** *Nature communications* 2017, **8**(1):1-12.

38. Korsunsky I, Millard N, Fan J, Slowikowski K, Zhang F, Wei K, Baglaenko Y, Brenner M, Loh P-r, Raychaudhuri S: **Fast, sensitive and accurate integration of single-cell data with Harmony.** *Nature methods* 2019, **16**(12):1289-1296.

39. Wolf FA, Angerer P, Theis FJ: **SCANPY: large-scale single-cell gene expression data analysis.** *Genome biology* 2018, **19**:1-5.

40. Satija R, Farrell JA, Gennert D, Schier AF, Regev A: **Spatial reconstruction of single-cell gene expression data.** *Nature biotechnology* 2015, **33**(5):495-502.

41. Zinati Y, Takiddeen A, Emad A: **GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks.** *bioRxiv* 2023:2023.2007. 2025.550225.

42. Sun T, Song D, Li WV, Li JJ: **scDesign2: a transparent simulator that generates high-fidelity single-cell gene expression count data with gene correlations captured.** *Genome biology* 2021, **22**(1):1-37.

43. Paul F, Arkin Ya, Giladi A, Jaitin DA, Kenigsberg E, Keren-Shaul H, Winter D, Lara-Astiaso D, Gury M, Weiner A: **Transcriptional heterogeneity and lineage commitment in myeloid progenitors.** *Cell* 2015, **163**(7):1663-1677.

44. Hu H, Miao Y-R, Jia L-H, Yu Q-Y, Zhang Q, Guo A-Y: **AnimalTFDB 3.0: a comprehensive resource for annotation and prediction of animal transcription factors.** *Nucleic acids research* 2019, **47**(D1):D33-D38.

Chapter 5

Discussion

Chapter 6

Conclusion

Bibliography

[1] Murat Kocaoglu, Christopher Snyder, Alexandros G Dimakis, and Sriram Vishwanath. Causalgan: Learning causal implicit generative models with adversarial training. *arXiv preprint arXiv:1709.02023*, 2017.

[2] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of the IEEE international conference on computer vision*, pages 3730–3738, 2015.